REACT

CHEAT SHEETS

A Complete Reference for Modern React Development

Components · Hooks · State Management · Routing · Data Fetching · Forms · Performance · Styling · Testing · Deployment

45 REFERENCE SHEETS

CONTENTS

REACT

Table of Contents

45 sheets · syntax · examples · notes

Sheet 01

Introduction to React

component model · virtual DOM · declarative UI · o

WHAT IS REACT?

React is a **JavaScript library** for building user interfaces, created by Facebook in 2013 and open-sourced the same year. It introduced a **component-based model** where UIs are built from small, reusable pieces — each managing its own state and rendering its own output. React does not prescribe a router, data layer, or build tool; it is deliberately a view-layer library that composes with other tools.

React's core insight is the **virtual DOM**: instead of directly manipulating the browser DOM on every change, React maintains a lightweight in-memory copy, diffs it against the previous version, and applies only the minimal set of real DOM updates needed. This makes frequent re-renders fast enough to drive interactive UIs at 60 fps.

CORE CONCEPTS

- `Component` — a function (or class) that accepts props and returns JSX describing the UI
- `JSX` — HTML-like syntax in JS files; compiled to `React.createElement()` calls by Babel/Vite
- `Virtual DOM` — React's in-memory UI tree; React diffs it and batches real DOM updates
- `Props` — read-only inputs passed from parent to child; the primary data channel
- `State` — mutable data local to a component; changing it triggers a re-render
- **One-way data flow** — data flows down (parent → child via props); events flow up (callbacks)
- **Declarative** — describe *what* the UI looks like for a given state; React handles *how* to update it

NOTES

- `createRoot` — React 18+ API; replaces legacy `ReactDOM.render()`
- `StrictMode` — development wrapper that double-invokes renders to surface side effects; no-op in production
- `import App from './App.jsx'` — default import; the component name is the local alias
- `export default function App()` — every component file exports one root component; named exports for additional helpers
- `{ name }` in JSX — curly braces evaluate any JS expression: variables, calls, ternaries
- Component names **must start with a capital letter** — lowercase tags are treated as native HTML elements

MINIMAL REACT APP

```jsx
// index.html (just a mount point)
// <div id="root"></div>

// main.jsx — application entry point
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
);

// App.jsx — root component
export default function App() {
  return <h1>Hello, React!</h1>;
       // JSX returned from a function
}

// A component with props
function Greeting({ name }) {
  return <p>Welcome, {name}!</p>;
       // {name} evaluates the JS expression
}
// Usage: <Greeting name="Ada" />   → Welcome, Ada!
```

REACT VS VANILLA JS

CONCERN	VANILLA JS	REACT
UI Updates	Manual DOM queries & mutations	Re-render component; React patches DOM
State	Variables + DOM as source of truth	`useState` — JS is the source of truth
Reuse	Copy-paste HTML; utility functions	Composable components with props
Events	`addEventListener`	Inline `onClick`, `onChange` props
Templating	Template literals / innerHTML	JSX — typed, linted, co-located

FILE TYPES & VERSIONS

ITEM	DETAIL
`.jsx`	Component files with JSX syntax (most common)
`.tsx`	TypeScript component files
`.js / .ts`	Utility modules, hooks, constants (no JSX)
`React 18`	Current stable — concurrent rendering, `useTransition`, automatic batching
`React 19`	2024+ — Server Components stable, `use()` hook, Actions API
`react-dom`	Browser renderer — separate package from `react` core
`react-native`	Mobile renderer — same component model, different host elements

COMMON MISTAKES

- Lowercase component names (`<app />`) — React treats them as unknown HTML tags and renders nothing useful
- Using old `ReactDOM.render()` in React 18+ — causes a console warning and disables concurrent features
- Confusing `react` and `react-dom` — `useState` comes from `react` ; `createRoot` comes from `react-dom/client`

TIPS

- Keep `StrictMode` on during development — it intentionally double-fires effects to expose bugs early
- One component per file is not enforced, but it scales better and makes imports predictable
- Think in components first: sketch the UI as a box-within-boxes hierarchy before writing any code

SUMMARY

- `Component` — function returning JSX; the fundamental building block
- `virtual DOM` — React diffs in-memory trees and minimizes real DOM writes
- `Props` down, events up — the one-way data flow contract
- `createRoot` — React 18+ entry point; mount once, render the component tree

Project Setup & Tooling

Vite · create-react-app · npm · project structure

SCAFFOLDING A REACT PROJECT

The two main ways to bootstrap a React project are **Vite** (recommended for new projects) and **Create React App** (CRA, now in maintenance mode). Vite uses native ES modules for near-instant dev server startup and hot module replacement; CRA uses Webpack and is significantly slower but has a larger existing ecosystem of guides.

For production apps that need server-side rendering or file-system routing, frameworks like **Next.js** or **Remix** scaffold React for you — skip Vite and use `npx create-next-app` instead.

KEY CONFIG FILES

- `package.json` — dependencies, scripts, project metadata; committed to git
- `vite.config.js` — Vite plugin config (e.g. `@vitejs/plugin-react` for JSX transform)
- `index.html` — Vite entry point; contains `<div id="root">` and a `<script>` to `main.jsx`
- `.env` — local env vars; never committed. Vite exposes vars prefixed `VITE_` to client code
- `.env.example` — committed template listing required env var keys (values omitted)
- `node_modules/` — auto-generated; always in `.gitignore`

NOTES

- `index.html` in root — Vite's entry point lives at the project root, not inside `/public` (unlike CRA)
- `src/main.jsx` — the only file that calls `createRoot`; all other files are components or utilities
- `components/` vs `pages/` — convention only; Vite doesn't enforce it. Pages are route-level; components are shared
- `hooks/` — custom hooks must start with `use` (e.g. `useFetch.js`) for React's linter rules to apply
- `VITE_` prefix — required; env vars without it are not injected into browser bundles (security guard)
- `import.meta.env` — Vite's env object; CRA uses `process.env.REACT_APP_*` instead

VITE PROJECT STRUCTURE & ENV VARS

```
my-app/
├── index.html          # entry point (not in /public)
├── vite.config.js      # Vite + React plugin config
├── package.json
├── .env                # local secrets — git-ignored
├── .env.example        # committed key template
├── public/             # static assets (copied as-is to dist)
│   └── favicon.svg
└── src/
    ├── main.jsx        # createRoot — mounts <App />
    ├── App.jsx         # root component
    ├── App.css
    ├── components/     # shared/reusable components
    ├── pages/          # route-level components
    ├── hooks/          # custom hooks (use*.js)
    └── utils/          # pure helper functions
```

```
# .env — Vite only exposes VITE_
                        # prefixed vars to the browser
VITE_API_URL=https://api.example.com
VITE_APP_TITLE=My App

# Accessing in component code:
# import.meta.env.VITE_API_URL  → "https://api.example.com"
# import.meta.env.MODE          → "development" | "production"
```

SCAFFOLD COMMANDS

COMMAND	WHAT IT DOES
`npm create vite@latest my-app -- --template react`	Vite + React (JS) scaffold — recommended
`npm create vite@latest my-app -- --template react-ts`	Vite + React + TypeScript scaffold
`npx create-react-app my-app`	CRA scaffold (Webpack; maintenance mode)
`npx create-next-app@latest`	Next.js (SSR/SSG + React)
`cd my-app && npm install`	Install dependencies after scaffold
`npm run dev`	Start Vite dev server (default: localhost:5173)

COMMON NPM SCRIPTS

SCRIPT	PURPOSE
`npm run dev`	Dev server with HMR
`npm run build`	Production bundle → `dist/`
`npm run preview`	Serve the `dist/` build locally
`npm run lint`	ESLint check (if configured)
`npm test`	Run test suite (Vitest / Jest)
`npm install pkg`	Add a runtime dependency
`npm install -D pkg`	Add a dev-only dependency

COMMON MISTAKES

- Committing `.env` to git — it contains secrets; add it to `.gitignore` immediately and commit `.env.example` instead
- Missing `VITE_` prefix on env vars — they silently resolve to `undefined` in the browser with no build error
- Running `npm run build` and serving `index.html` directly — use `npm run preview` or a static host; opening the file in a browser breaks asset paths

TIPS

- Prefer Vite over CRA for new projects — HMR is near-instant and the config is far simpler
- Use `npm run preview` before deploying to catch asset-path bugs that only appear in production builds
- Install `eslint-plugin-react-hooks` immediately — it catches Rules of Hooks violations and missing `useEffect` dependencies

SUMMARY

- `npm create vite@latest` — recommended scaffold; fastest dev server
- `npm run dev / build / preview` — the three core lifecycle scripts
- `VITE_` prefix — required for env vars to be available in browser code
- `src/` structure — components, pages, hooks, utils; convention, not enforced

JSX Basics

WHAT IS JSX?

JSX (JavaScript XML) is a syntax extension that lets you write HTML-like markup directly inside JavaScript. It is not valid JS on its own — a compiler (Babel, the Vite React plugin, or the TypeScript compiler) transforms it into `React.createElement()` calls before the browser sees it.

JSX looks like HTML but follows **JavaScript scoping rules**: attribute names are camelCase, reserved words like `class` and `for` are replaced with `className` and `htmlFor`, and any JavaScript expression can be embedded in **curly braces** `{ }`. Every JSX element must have a single root node — or use a Fragment to wrap siblings without adding a DOM element.

JSX RULES

- Return a **single root element** — wrap siblings in a `<div>` or `<>…</>` Fragment
- All tags must be **closed** — `<img />`, `<br />`, `<input />` (not `<img>`)
- Use `className` instead of `class` — `class` is a reserved JS keyword
- Use `htmlFor` instead of `for` on `<label>` elements
- Attribute names are **camelCase**: `onClick`, `tabIndex`, `strokeWidth`
- `{ }` evaluates any JS **expression** (not statements — no `if` blocks, no `for` loops)
- JSX comments: `{/* comment */}` — HTML-style `<!-- -->` does not work inside JSX

NOTES

- `<>…</>` — shorthand for `React.Fragment`; groups siblings without adding a DOM node
- `style={{ }}` — outer braces are the JSX expression; inner braces are the JS object literal; keys are camelCase strings
- `onClick={handleClick}` — pass the function reference, not a call; `onClick={handleClick()}` fires immediately on render
- `{/* comment */}` — only way to comment inside JSX; must be inside curly braces
- `React.createElement` — what every JSX element compiles to; you never call it directly in modern React
- Strings render fine; `null`, `undefined`, and `false` render nothing — useful for conditional rendering

JSX IN PRACTICE

```jsx
// ── Single root requirement ──────────────────────────
function Bad()  { return <h1>Hi</h1><p>Oops</p> }
            // ✖ error
function Good() { return <><h1>Hi</h1><p>OK</p></> }
            // ✔ Fragment

// ── Attributes ──────────────────────────────────────
<button className="btn" onClick={handleClick}>Save</button>
<input type="text" tabIndex={0} />
<label htmlFor="email">Email</label>
<input id="email" />

// ── Inline styles (object, camelCase keys) ──────────
<div style={{ color: 'red', fontSize: 16 }}>Alert</div>

// ── Expressions in curly braces ─────────────────────
const user = 'Ada';
const score = 42;
<p>{user} scored {score * 2} points</p>
{/* → Ada scored 84 points */}

// ── Self-closing tags ───────────────────────────────
<img src="/logo.svg" alt="Logo" />
<br />
<MyComponent />  {/* component with no children */}

// ── What JSX compiles to ────────────────────────────
// <h1 className="title">Hello</h1>
// React.createElement('h1', { className: 'title' }, 'Hello')
```

HTML → JSX ATTRIBUTE CHANGES

HTML	JSX	NOTE
`class`	`className`	JS reserved word
`for`	`htmlFor`	JS reserved word
`onclick`	`onClick`	camelCase events
`tabindex`	`tabIndex`	camelCase attrs
`stroke-width`	`strokeWidth`	SVG attrs also camelCase
`style="color:red"`	`style={{color:'red'}}`	Object, not string
`colspan`	`colSpan`	Table attrs camelCase

VALID JSX EXPRESSIONS

TYPE	EXAMPLE
Variable	`{name}`
Arithmetic	`{count + 1}`
Function call	`{formatDate(date)}`
Ternary	`{isOn ? 'On' : 'Off'}`
Logical AND	`{show && <Tag />}`
Array.map	`{items.map(i => <li>{i}</li>)}`
String template	`{`Hello ${name}`}`

COMMON MISTAKES

- Using `class=` instead of `className=` — silently applies in the browser but React logs a warning and it breaks linting
- `onClick={doSomething()}` — calling the function instead of passing it; the function fires on every render, not on click
- Using an `if` statement directly inside JSX — only expressions are allowed; use a ternary or move the logic above the return

TIPS

- Use Fragments (`<>…</>`) instead of wrapper `<div>` s to avoid polluting the DOM with extra nodes
- Move complex logic above the `return` statement and assign results to variables — JSX expressions should stay readable
- Install the **ES7+ React/Redux/React-Native snippets** VS Code extension for fast component scaffolding (`rafce` snippet)

SUMMARY

- `className / htmlFor` — replace HTML's `class` / `for` ; all attrs camelCase
- `{ }` — embed any JS expression; not statements
- `<>…</>` — Fragment; single root with no extra DOM node
- `self-closing` — all tags must close: `<img />`, `<input />`

Sheet 04

JSX — Lists, Conditionals & Fragments

key prop · Array.map · conditional rendering · &&

RENDERING LISTS & CONDITIONAL UI

Because JSX only accepts **expressions** (not statements), dynamic rendering relies on two JavaScript patterns: `Array.map()` for lists, and either the **ternary operator** or **logical AND** (&&) for conditional output.

When rendering a list with `map()`, every item must have a **key prop** — a stable, unique string or number identifying that item within its list. React uses keys to match elements between renders during reconciliation; without them, React falls back to index-based diffing, which causes subtle bugs when items are reordered, added, or removed.

KEY RULES

- `key` must be **unique among siblings** — not globally unique; same key is fine in a different list
- `key` must be **stable** — do not use `Math.random()` or the array index if the list can be reordered
- `key` is **not a prop** — you cannot read it inside the component via `props.key`; pass it as a separate prop if needed
- `&&` pitfall: `{count && <Tag />}` renders `0` when count is 0 — use `{count > 0 && …}` or a ternary instead
- `null`, `undefined`, `false`, `true` all render **nothing** — safe to return from conditional branches
- `<Fragment key={id}>` — use the long-form Fragment when you need a `key` prop on a fragment wrapper

LISTS & CONDITIONALS IN JSX

```
const fruits = [
  { id: 1, name: 'Apple' },
  { id: 2, name: 'Banana' },
  { id: 3, name: 'Cherry' },
];

// ── Rendering a list ──────────────────────────────
function FruitList() {
  return (
    <ul>
      {fruits.map(fruit => (
        <li key={fruit.id}>{fruit.name}</li>  // key on outermost element
      ))}
    </ul>
  );
}

// ── Conditional: logical AND ──────────────────────
{isLoggedIn && <Dashboard />}           // shows Dashboard or nothing
{count && <Badge />}                    // ✖ renders "0" when count=0
{count > 0 && <Badge count={count} />} // ✔ safe guard

// ── Conditional: ternary ──────────────────────────
{isLoggedIn ? <Dashboard /> : <LoginPage />}

// ── Early return guard ────────────────────────────
function UserCard({ user }) {
  if (!user) return null;               // renders nothing
  return <div>{user.name}</div>;
}

// ── Fragment with key (in a map) ──────────────────
import { Fragment } from 'react';
{items.map(item => (
  <Fragment key={item.id}>
    <dt>{item.term}</dt>
    <dd>{item.def}</dd>
  </Fragment>
))}
```

NOTES

- `key={fruit.id}` — goes on the outermost element returned from `map()`, not on an inner element
- `fruits.map(…)` — map returns an array of JSX nodes; React renders arrays automatically
- `count > 0 &&` — converting count to a boolean before `&&` prevents the `0` render pitfall
- `return null` — a component returning null renders nothing and unmounts its children cleanly
- `<Fragment key={item.id}>` — shorthand `<>` cannot take props; use the named import when you need `key`
- Nesting `map()` calls — each level needs its own `key`; key uniqueness is scoped to the sibling list at each level

CONDITIONAL RENDERING PATTERNS

PATTERN	USE WHEN	RENDERS NOTHING IF…
`{cond && <A />}`	Show A or nothing	cond is falsy (watch: 0 renders!)
`{cond ? <A /> : <B />}`	Swap between two UIs	Never — always renders one branch
`{cond ? <A /> : null}`	Show A or nothing (safe)	cond is falsy
`if/else before return`	Complex multi-branch logic	return null explicitly
`Early return null`	Guard clause in component	Condition triggers the return

GOOD VS BAD KEY CHOICES

KEY SOURCE	SAFE?	WHY
`item.id (DB primary key)`	✔	Stable, unique by definition
`item.slug or item.email`	✔	Stable unique identifier
`crypto.randomUUID()`	✖	New key every render → remounts
`Math.random()`	✖	Same — causes full remount
`array index (static list)`	~	OK if list never reorders/filters
`array index (dynamic list)`	✖	Bugs on add/remove/sort

COMMON MISTAKES

- Omitting the `key` prop in a `map()` — React logs a warning and falls back to slow index-based diffing
- `{count && <Badge />}` when count can be 0 — renders the literal digit `0` in the DOM; use `count > 0 &&`
- Putting the `key` on an inner element instead of the outermost returned node — React cannot track the item correctly

TIPS

- When your list items are multi-element groups, reach for `<Fragment key={id}>` rather than a wrapper `<div>` to keep the DOM clean
- Prefer ternary over `&&` when the falsy path should render an explicit fallback UI rather than nothing
- If you don't have a natural unique ID, generate stable IDs server-side or at data-creation time — never at render time

SUMMARY

- `Array.map()` — primary pattern for rendering lists in JSX
- `key` — required on each list item; must be stable and unique among siblings
- `&&` — show or hide; use `count > 0 &&` to avoid rendering `0`
- `ternary` — swap between two UIs; `Fragment key` for keyed multi-element groups

Function Components

THE FUNCTION COMPONENT MODEL

A **function component** is a plain JavaScript function that accepts a single `props` object and returns JSX (or `null`). Since React 16.8, hooks give function components full access to state, side effects, and context — making class components unnecessary for new code.

React calls your function on every render. The function must be **pure with respect to its inputs**: given the same props and state, it must return the same JSX. Side effects (data fetching, subscriptions, timers) belong in `useEffect`, not in the function body directly.

COMPONENT RULES

- Name must start with a **capital letter** — React uses this to distinguish components from native HTML tags
- Must return a **single root** — one element, a Fragment, or `null`
- Must be **pure** — no direct DOM mutations, no side effects in the render body
- `props` are **read-only** — never mutate `props` or `props.children`
- Can call **hooks** at the top level — never inside loops, conditions, or nested functions
- One component per file is convention; named + default exports both work

COMPONENT PATTERNS

```jsx
// ── Basic function declaration ──────────────────────
function Greeting({ name }) {
  return <h1>Hello, {name}!</h1>;
}
export default Greeting;

// ── Arrow component (implicit return) ───────────────
const Badge = ({ label }) => <span className="badge">{label}</span>;

// ── Arrow component (explicit return, multi-line JSX) ──
const Card = ({ title, children }) => {
  return (
    <div className="card">
      <h2>{title}</h2>
      <div className="body">{children}</div>
    </div>
  );
};

// ── Named exports (multiple from one file) ──────────
export { Badge, Card };

// ── Consuming components ────────────────────────────
import Greeting from './Greeting';
import { Badge, Card } from './ui';

function App() {
  return (
    <>
      <Greeting name="Ada" />
      <Card title="Status">
        <Badge label="Active" />
      </Card>
    </>
  );
}
```

NOTES

→ `{ name }` — destructuring props inline; cleaner than `props.name` everywhere

→ `children` — special prop; whatever is nested between the opening and closing tags

→ Implicit arrow return — wrap multi-line JSX in `( )` to avoid ASI issues; omit braces entirely

→ `export default` — one per file; the import alias can be anything: `import Foo from './Bar'`

→ `export { Badge, Card }` — named exports; import must use the exact name in braces

→ Parentheses around multi-line JSX after `return` are required — bare newline after return returns `undefined`

DECLARATION STYLES

STYLE	SYNTAX	NOTES
`Function declaration`	`function Foo() {}`	Hoisted; most common in tutorials
`Arrow (const)`	`const Foo = () => {}`	Not hoisted; preferred by many teams
`Arrow (implicit)`	`const Foo = () => <div />`	Implicit return — only for single expr
`Default export`	`export default function Foo`	One per file; imported without braces
`Named export`	`export function Foo`	Multiple per file; imported with braces

VALID RETURN VALUES

RETURN	EFFECT
`JSX element`	Renders the element
`<>…</> Fragment`	Renders children, no wrapper node
`Array of JSX`	Renders each item (each needs key)
`String / number`	Renders as text node
`null`	Renders nothing; component unmounts children
`undefined`	Error in dev; avoid — use null explicitly

COMMON MISTAKES

- Lowercase component name (`function card()`) — React renders it as an unknown HTML tag instead of calling the function
- `return` on its own line without parentheses — JS inserts a semicolon and the component returns `undefined`
- Mutating props directly (`props.count++`) — props are read-only; mutating them doesn't trigger a re-render and corrupts parent state

TIPS

- Destructure props in the parameter list — `({ name, age })` keeps the JSX body clean and makes the API obvious at a glance
- Keep components small and focused — if a component needs a scroll bar to read, it should be split
- Co-locate small helper components in the same file; extract to their own file once they're reused elsewhere

SUMMARY

- `function Foo(props)` — accepts props object, returns JSX or null
- `Capital name` — required; lowercase = HTML tag to React
- `export default` — one per file; `export {}` for named multi-exports
- `children` — built-in prop for nested JSX content

Props

WHAT ARE PROPS?

Props (short for properties) are the mechanism for passing data *down* the component tree — from parent to child. They are passed as JSX attributes and received as a plain JavaScript object in the component function. Props are **immutable from the child's perspective**: a child component must never modify the props it receives.

Any JavaScript value can be a prop: strings, numbers, booleans, arrays, objects, and **functions** (including event handlers and callbacks). The special `children` prop receives whatever JSX is nested between a component's opening and closing tags.

PROPS IN PRACTICE

```
// ── Passing and receiving props ──────────────────────
function Avatar({ src, alt, size = 40 }) {    // default size=40
  return <img src={src} alt={alt} width={size} height={size} />;
}
<Avatar src="/ada.jpg" alt="Ada" size={80} />
<Avatar src="/ada.jpg" alt="Ada" />   // size defaults to 40

// ── children prop ────────────────────────────────────
function Panel({ title, children }) {
  return (
    <div className="panel">
      <h2>{title}</h2>
      {children}
    </div>
  );
}
<Panel title="Info">
  <p>Any JSX goes here as children.</p>
</Panel>

// ── Spread props (forwarding to DOM element) ─────────
function Input({ label, ...rest }) {
            // rest captures remaining props
  return (
    <div>
      <label>{label}</label>
      <input {...rest} />
            // forwards type, value, onChange…
    </div>
  );
}
<Input label="Email" type="email" value={val} onChange={setVal} />

// ── Callback prop (child → parent communication) ─────
function Counter({ onReset }) {
  return <button onClick={onReset}>Reset</button>;
}
<Counter onReset={() => setCount(0)} />
```

PROP PATTERNS

- **Default values** — set in destructuring: `function Btn({ label = 'OK' })`
- **Spread props** — `<Input {...inputProps} />` forwards all keys; use carefully to avoid unintended attrs
- **Rest props** — `({ id, ...rest })` captures remaining props for forwarding to a DOM element
- `children` — implicit prop; set by nesting JSX between open/close tags of the component
- **Boolean shorthand** — `<Modal isOpen />` is identical to `<Modal isOpen={true} />`
- **Callbacks** — pass functions as props to let children communicate events upward to the parent

NOTES

- `size = 40` — default value in destructuring; only applies when the prop is `undefined`, not when it's `null`
- `{children}` — render anywhere inside the component; wrap in a container or scatter across the layout
- `...rest` — collects all remaining props; forward to a native element to let callers use any standard HTML attribute
- `{...rest}` on a DOM element — JSX spread; be careful not to forward React-specific props (like `isOpen`) to the DOM
- `onReset` — naming convention: `on + EventName` for callback props mirrors React's native event naming
- Props can be any JS value, including other components or render functions (render props pattern)

PROP VALUE SYNTAX BY TYPE

TYPE	JSX SYNTAX	EXAMPLE
`String`	Quotes or braces	`name="Ada"` or `name={'Ada'}`
`Number`	Braces required	`count={42}`
`Boolean (true)`	Attribute shorthand	`disabled` or `disabled={true}`
`Boolean (false)`	Braces required	`disabled={false}`
`Array`	Braces required	`items={[1, 2, 3]}`
`Object`	Braces required	`style={{ color: 'red' }}`
`Function`	Braces required	`onClick={handleClick}`

CHILDREN PROP FORMS

USAGE	CHILDREN VALUE
`<Btn>Save</Btn>`	String: `"Save"`
`<Btn><Icon /></Btn>`	Single React element
`<Btn><A /><B /></Btn>`	Array of elements
`<Btn /> (self-closing)`	`undefined`
`<Btn>{fn}</Btn>`	Function (render prop pattern)

COMMON MISTAKES

- Mutating a prop (`props.items.push(x)`) — props are read-only; mutation doesn't trigger a re-render and causes subtle bugs
- Using `null` as a prop value expecting the default to kick in — defaults only apply for `undefined`; `null` is passed as-is
- Spreading props onto a native DOM element without filtering — unknown attributes cause React warnings and noisy HTML output

TIPS

- Use `...rest` + spread to build wrapper components around native elements — they stay fully compatible with all HTML attributes
- Name callback props `onVerb` (e.g. `onSave`, `onDelete`) — consistent with React's own event naming
- If you find yourself passing many props through several layers, consider `useContext` (Sheet 12) to avoid prop drilling

SUMMARY

- `props` — read-only data from parent; never mutate
- `{ prop = default }` — default value applies only when prop is `undefined`
- `children` — implicit prop for nested JSX content
- `{...rest}` — spread remaining props onto a child element

PropTypes & Type Checking

RUNTIME VS COMPILE-TIME TYPE CHECKING

React ships with no built-in prop type enforcement. The **prop-types** package adds *runtime* warnings in development: when a component receives a prop of the wrong type, React logs a console warning. Prop-types checks are stripped in production for performance.

For *compile-time* checking, use **TypeScript** (`.tsx` files) with an interface or type alias for props — this catches errors before the code runs. TypeScript is generally preferred for new projects; prop-types remains useful in pure JavaScript codebases or when migrating legacy code.

PROP-TYPES CHECKERS

- `PropTypes.string` · `.number` · `.bool` · `.func` · `.symbol` — primitives
- `PropTypes.array` · `.object` — any array / any object (prefer `arrayOf` / `shape`)
- `PropTypes.node` — anything renderable: string, number, element, array, fragment
- `PropTypes.element` — exactly one React element
- `PropTypes.arrayOf(type)` — array where every item matches `type`
- `PropTypes.shape({ })` — object with specific key types
- `PropTypes.oneOf([…])` — enum: one of the listed literal values
- `.isRequired` — chain onto any checker; warns if prop is `undefined` or `null`

PROP-TYPES & TYPESCRIPT EXAMPLES

```jsx
// ── prop-types (JavaScript) ──────────────────────────
import PropTypes from 'prop-types';

function UserCard({ name, age, role, tags, address, onSave }) {
  return <div>{name}</div>;
}

UserCard.propTypes = {
  name:    PropTypes.string.isRequired,
  age:     PropTypes.number,
  role:    PropTypes.oneOf(['admin', 'editor', 'viewer']),
  tags:    PropTypes.arrayOf(PropTypes.string),
  address: PropTypes.shape({
    street: PropTypes.string,
    city:   PropTypes.string.isRequired,
  }),
  onSave:  PropTypes.func.isRequired,
};

UserCard.defaultProps = {   // legacy default props API
  age: 0,
  tags: [],
};

// ── TypeScript equivalent (.tsx) ─────────────────────
interface UserCardProps {
  name: string;                          // required
  age?: number;                          // optional
  role?: 'admin' | 'editor' | 'viewer'; // union
  tags?: string[];
  address?: { street?: string; city: string; };
  onSave: () => void;                    // required function
}

function UserCard({ name, age = 0, tags = [], ...rest }: UserCardProps) {
  return <div>{name}</div>;
}
```

NOTES

- `.isRequired` — chained last; warns if the prop is missing or null; do not use with `defaultProps` for the same prop
- `PropTypes.node` — use for `children`; accepts strings, numbers, elements, and arrays of those
- `PropTypes.shape` — only checks the listed keys; extra keys on the object are ignored
- `defaultProps` — legacy API; preferred modern approach is default values in destructuring (`age = 0`)
- `interface` vs `type` in TS — both work for props; `interface` is extendable, `type` supports unions; pick one and be consistent
- TS props: `?` marks optional; omitting `?` makes it required — enforced at compile time with no runtime cost

PROPTYPES VS TYPESCRIPT PROPS

FEATURE	PROP-TYPES	TYPESCRIPT
When it runs	Runtime (dev only)	Compile time (always)
Catches errors	After the page loads	Before running the code
IDE autocomplete	✖	✔
Production cost	Stripped by bundler	Zero (erased at compile)
Setup needed	`npm i prop-types`	TypeScript + `.tsx` files
Best for	JS codebases, quick projects	New projects, team code

COMMON MISTAKES

- Using `PropTypes.object` or `PropTypes.array` — too loose; use `shape({})` and `arrayOf()` to document the actual structure
- Adding `.isRequired` and a `defaultProps` entry for the same prop — the default means it's never undefined, making `isRequired` a no-op
- Expecting prop-types to catch errors in production — checks only run in development; production builds strip them entirely

TIPS

- Prefer TypeScript over prop-types for new projects — compile-time errors and IDE autocomplete are far more useful than runtime warnings
- Use `PropTypes.node` for `children` — it's the most permissive type that still rejects non-renderable values like plain objects
- In TypeScript, define the props interface in the same file as the component; only extract it to a shared types file when it's reused across multiple components

SUMMARY

- `PropTypes` — runtime dev-only warnings; install `prop-types` package
- `.isRequired` — chain to warn when prop is missing or null
- `shape / arrayOf / oneOf` — specific structural validators
- `interface Props` — TypeScript compile-time alternative; preferred for new projects

Sheet 08

Component Composition

children · slot pattern · compound components · re

COMPOSITION OVER INHERITANCE

React favors **composition** over inheritance: instead of extending a base component class, you build complex UIs by *nesting* simpler components inside each other. The `children` prop is the primary tool — a parent "wraps" output around whatever the caller places inside it.

The **slot pattern** extends this idea: instead of one `children` slot, a component accepts multiple named props that each receive a JSX value, giving the caller control over several distinct regions of the layout. **Compound components** — a group of related components sharing context — allow complex widgets (tabs, accordions, selects) to be assembled from parts while keeping state internal.

COMPOSITION PATTERNS

- **Containment** — component uses `children` to render whatever is nested inside it (Panel, Modal, Card)
- **Slot pattern** — named JSX props (`header=` , `footer=`) give callers control over specific regions
- **Specialization** — a general component (`Button`) configured via props to make a specific one (`DangerButton`)
- **Compound components** — a family of components (e.g. `Tabs.Root` , `Tabs.List` , `Tabs.Panel`) sharing internal context
- **Render props** — a prop whose value is a function returning JSX; the parent calls it with data; useful for sharing logic
- Prefer **composition** over wrapper HOCs — easier to read, type, and test

COMPOSITION PATTERNS IN CODE

```
// ── Containment (children) ──────────────────────────
function Card({ children }) {
  return <div className="card">{children}</div>;
}
<Card><h2>Title</h2><p>Body</p></Card>

// ── Slot pattern (named JSX props) ──────────────────
function Layout({ header, sidebar, children }) {
  return (
    <div className="layout">
      <header>{header}</header>
      <aside>{sidebar}</aside>
      <main>{children}</main>
    </div>
  );
}
<Layout
  header={<TopNav />}
  sidebar={<SideMenu />}
>
  <Page />
</Layout>

// ── Specialization ──────────────────────────────────
function Button({ variant = 'primary', ...rest }) {
  return <button className={`btn btn-${variant}`} {...rest} />;
}
const DangerButton = (props) => <Button variant="danger" {...props} />;

// ── Compound components ─────────────────────────────
const TabsCtx = React.createContext(null);

function Tabs({ children, defaultTab }) {
  const [active, setActive] = React.useState(defaultTab);
  return <TabsCtx.Provider value={{ active, setActive }}>{children}</TabsCtx.Provider>;
}
Tabs.Tab = function Tab({ id, children }) {
  const { active, setActive } = React.useContext(TabsCtx);
  return <button className={active === id ? 'active' : ''} onClick={() =>
        setActive(id)}>{children}</button>;
};
```

NOTES

- `children` — the most common slot; accepts any renderable value; use `React.Children.count` if you need to inspect it
- `header={<TopNav />}` — JSX passed as a prop value; evaluated in the caller's scope, rendered inside the callee
- `variant="danger"` — specialization via partial application of props; the wrapping component hard-codes one or more defaults
- `Tabs.Tab` — attaching sub-components as static properties is the classic compound component pattern; keeps the API namespaced
- `TabsCtx` — compound components share state through context; callers don't manage the internal active-tab state
- Avoid deep prop drilling by using the slot pattern or compound components before reaching for global state

PATTERN COMPARISON

PATTERN	HOW	BEST FOR
`children`	Nest JSX between tags	Generic wrappers, layouts
`Slot props`	Named JSX props (`header={}`)	Multi-region layouts
`Specialization`	Wrap + hard-code some props	Themed / variant components
`Compound`	Shared context among siblings	Complex widgets (Tabs, Select)
`Render prop`	Prop is a `() => JSX` function	Logic sharing without HOCs

REACT.CHILDREN UTILITIES

API	PURPOSE
`React.Children.count(children)`	Number of child nodes
`React.Children.map(children, fn)`	Map over children safely
`React.Children.forEach(children, fn)`	Iterate without returning
`React.Children.toArray(children)`	Flatten to array with stable keys
`React.Children.only(children)`	Assert exactly one child (throws otherwise)

COMMON MISTAKES

- Reaching for inheritance (`class AdminPage extends Page`) — React components don't compose well through class inheritance; use props and composition instead
- Cloning children with `React.cloneElement` to inject props — fragile and hard to type; prefer compound components with context
- Making every layout decision a prop — too many slot props turns a component into a configuration object; split into smaller components instead

TIPS

- Start with `children` ; only add named slot props when you have two or more distinct regions that callers need to customize independently
- Compound components shine for UI primitives like Tabs, Accordion, Dialog — keep internal state inside the root and expose sub-components via static properties
- Specialization is the cleanest way to build a design system: one generic `Button` , several named variants wrapping it

SUMMARY

- `children` — primary composition slot; any nested JSX
- `slot props` — named JSX props for multi-region layouts
- `Specialization` — wrap + pre-configure a generic component
- `Compound` — family of components sharing context for complex widgets

Class Components (Legacy)

CLASS COMPONENTS IN MODERN REACT

Before React 16.8 (2019), **class components** were the only way to use state and lifecycle methods. They extend `React.Component` and implement a mandatory `render()` method. They are still fully supported and appear in most pre-2019 codebases — you will encounter them when maintaining legacy code.

New features like concurrent mode, `useTransition`, and Server Components are designed around function components. Hooks (`useState`, `useEffect`) cannot be used inside class components. The recommended migration path is to **rewrite class components as function components** when touching them for other reasons.

CLASS COMPONENT ANATOMY

```jsx
import React from 'react';

class Counter extends React.Component {
  // 1. Initialize state in constructor
  constructor(props) {
    super(props);                          // always first
    this.state = { count: 0 };
    this.handleClick = this.handleClick.bind(this);
              // bind or use arrow
  }

  // 2. Lifecycle — after first paint
  componentDidMount() {
    document.title = `Count: ${this.state.count}`;
  }

  // 3. Lifecycle — after every re-render
  componentDidUpdate(prevProps, prevState) {
    if (prevState.count !== this.state.count) {
              // guard required!
      document.title = `Count: ${this.state.count}`;
    }
  }

  // 4. Lifecycle — before removal
  componentWillUnmount() {
    document.title = 'App';                // cleanup
  }

  handleClick() {
    this.setState(prev => ({ count: prev.count + 1 }));
  }

  // 5. render() is required
  render() {
    return (
      <div>
        <p>{this.state.count}</p>
        <button onClick={this.handleClick}>+</button>
      </div>
    );
  }
}
```

LIFECYCLE METHODS

- `constructor(props)` — initialize `this.state`; call `super(props)` first
- `render()` — required; must be pure; returns JSX or null
- `componentDidMount()` — runs after first render; start data fetching, subscriptions here
- `componentDidUpdate(prevProps, prevState)` — runs after every re-render; compare prev to avoid infinite loops
- `componentWillUnmount()` — cleanup: cancel timers, unsubscribe, abort fetches
- `componentDidCatch(error, info)` — only class components can be Error Boundaries
- `static getDerivedStateFromProps` — rare; derive state from props before render

NOTES

- `super(props)` — required first line of constructor; without it, `this.props` is undefined inside the constructor
- `this.state = {}` — only assign directly in the constructor; everywhere else use `setState`
- `.bind(this)` — class methods lose `this` context when passed as callbacks; bind in constructor or use arrow class fields
- `prev => ({ count: prev.count + 1 })` — functional update; safe when multiple setStates batch together
- `prevState.count !== this.state.count` — guard in `componentDidUpdate` is mandatory; omitting it causes an infinite render loop
- `componentDidCatch` — the only thing class components still do that hooks cannot; keep one class Error Boundary per app

CLASS ↔ HOOK EQUIVALENTS

CLASS	FUNCTION / HOOK EQUIVALENT
`this.state / setState`	`useState`
`componentDidMount`	`useEffect(() => {}, [])`
`componentDidUpdate`	`useEffect(() => {}, [deps])`
`componentWillUnmount`	`useEffect cleanup return fn`
`this.props`	`function parameters / props`
`this.forceUpdate`	`useState setter (no direct equiv)`
`componentDidCatch`	No hook — keep as class for Error Boundaries

SETSTATE BEHAVIOUR

CALL	EFFECT
`this.setState({ count: 1 })`	Merges patch into state (shallow)
`this.setState(prev => ({...}))`	Functional update — safe for batched calls
`this.setState({}, callback)`	Callback runs after state is applied
`setState in render()`	Infinite loop — never do this
`Reads this.state after setState`	May be stale — use functional form

COMMON MISTAKES

- Missing `super(props)` in constructor — `this.props` is `undefined` for the rest of the constructor body
- Omitting the comparison guard in `componentDidUpdate` — causes an infinite re-render loop every time the component updates
- Reading `this.state` immediately after `setState` — state updates are batched and asynchronous; use the functional form or the callback argument

TIPS

- Use arrow class fields instead of constructor binding: `handleClick = () => {}` — cleaner and avoids forgetting `.bind`
- Keep one class component in your app as an **Error Boundary** — there is no hooks equivalent for `componentDidCatch`
- When migrating, convert lifecycle methods to hooks: `componentDidMount` + `componentWillUnmount` → single `useEffect` with cleanup return

SUMMARY

- `render()` — required method; returns JSX; must be pure
- `this.setState()` — triggers re-render; use functional form for safety
- `componentDidMount/Update/WillUnmount` — lifecycle hooks replaced by `useEffect`
- `componentDidCatch` — Error Boundaries only; no hooks equivalent

useState

LOCAL COMPONENT STATE

`useState` adds a reactive state variable to a function component. Every time the setter is called with a new value, React schedules a re-render of the component and its children. The hook returns a **tuple**: the current value and a setter function. The setter replaces state entirely — it does *not* merge like class component `setState`.

State is **local and private** to the component instance. Two instances of the same component each have their own independent state. To share state between components, *lift it up* to their nearest common ancestor and pass it down as props.

USESTATE RULES

- Call only at the **top level** of a component — never inside loops, conditions, or nested functions
- Setter is **asynchronous** — reading state immediately after calling the setter gives the old value
- Setter with the **same value** (by reference equality) skips re-render (`Object.is` comparison)
- **Functional update** form `set(prev => …)` — safe when new state depends on old state
- **Lazy initializer** — pass a function as initial value: `useState(() => expensiveCalc())` ; runs only once
- Objects and arrays must be **replaced, not mutated** — spread to create a new reference

USESTATE IN PRACTICE

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);       // initial value = 0

  return (
    <div>
      <p>{count}</p>
      <button onClick={() => setCount(c => c + 1)}>+</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}

// ── Object state ──────────────────────────────────────
const [user, setUser] = useState({ name: 'Ada', age: 28 });
// ✔ Replace with spread — new object reference triggers re-render
setUser(prev => ({ ...prev, age: 29 }));
// ✖ Mutation — same reference, React skips re-render
user.age = 29; setUser(user);

// ── Array state ───────────────────────────────────────
const [todos, setTodos] = useState([]);
const add    = (todo) => setTodos(prev => [...prev, todo]);
const remove = (id) => setTodos(prev => prev.filter(t => t.id !== id));
const update = (id, text) =>
  setTodos(prev => prev.map(t => t.id === id ? { ...t, text } : t));

// ── Lazy initializer (runs only on mount) ─────────────
const [theme, setTheme] = useState(() =>
    localStorage.getItem('theme') ?? 'dark');
```

NOTES

- → `useState(0)` — initial value used only on the first render; ignored on subsequent renders
- → `setCount(c => c + 1)` — functional update; always reads the latest value even in batched updates
- → `setCount(0)` — passing the same value as current state skips re-render (React uses `Object.is`)
- → `{ ...prev, age: 29 }` — spread creates a new object reference; React sees a different reference and re-renders
- → `prev.filter / prev.map` — both return new arrays; never use `push` , `splice` , or direct index assignment on state arrays
- → `() => localStorage.getItem(…)` — lazy initializer avoids running the expensive/slow function on every render

COMMON STATE PATTERNS

PATTERN	CODE
Primitive	`const [count, setCount] = useState(0)`
String	`const [name, setName] = useState('')`
Boolean toggle	`setOpen(prev => !prev)`
Object update	`setUser(p => ({ ...p, age: 30 }))`
Array append	`setItems(p => [...p, newItem])`
Array remove	`setItems(p => p.filter(i => i.id !== id))`
Lazy init	`useState(() => JSON.parse(stored))`

MUTATION VS REPLACEMENT

	CODE	WORKS?
✖ Mutate array	`items.push(x); setItems(items)`	✖
✔ Replace array	`setItems([...items, x])`	✔
✖ Mutate object	`user.name = 'x'; setUser(user)`	✖
✔ Replace object	`setUser({ ...user, name: 'x' })`	✔
✖ Direct assign	`count = count + 1`	✖
✔ Setter	`setCount(c => c + 1)`	✔

COMMON MISTAKES

- Mutating state in place (`arr.push(x); setArr(arr)`) — same reference means React skips the re-render; always produce a new array/object
- Reading state right after setting it — the setter schedules a future render; the current variable still holds the old value until the next render
- Using `useState` for derived data — if a value can be computed from existing state or props, compute it during render instead of storing it in state

TIPS

- Use the **functional update** form whenever new state depends on old state — it's safe during React 18's automatic batching
- Group related state values into an object if they always change together; keep unrelated values in separate `useState` calls
- For complex state transitions with many actions, consider upgrading to `useReducer` (Sheet 15)

SUMMARY

- `const [val, setVal] = useState(init)` — declare reactive state
- `setVal(prev => …)` — functional update; safe for batched calls
- `{ ...prev, key: val }` — always replace objects/arrays, never mutate
- `useState(() => fn())` — lazy initializer; runs once on mount only

useEffect

SYNCHRONISING WITH THE OUTSIDE WORLD

`useEffect` lets you run code *after* React has painted the screen — the right place for side effects: data fetching, subscriptions, timers, and direct DOM manipulation. It runs **after every render** by default; the optional dependency array controls when it re-runs.

The effect can return a **cleanup function** that React calls before the next effect run and before the component unmounts. Use cleanup to cancel fetches, clear timers, or unsubscribe — otherwise you risk memory leaks and state updates on unmounted components. In **Strict Mode** (development only), React intentionally runs each effect twice to help surface missing cleanups.

DEPENDENCY ARRAY BEHAVIOUR

- `useEffect(fn)` — no array: runs after *every* render
- `useEffect(fn, [])` — empty array: runs once after the first render (mount only)
- `useEffect(fn, [a, b])` — runs when `a` or `b` changes (by `Object.is`)
- All values from the component scope used inside the effect must be in the dependency array
- The `eslint-plugin-react-hooks` rule `exhaustive-deps` enforces this automatically
- Objects and functions as deps change reference every render — memoize with `useMemo` / `useCallback` first

USEEFFECT PATTERNS

```
import { useState, useEffect } from 'react';

// ── Run once on mount (fetch data) ──────────────────
function Posts() {
  const [posts, setPosts] = useState([]);

  useEffect(() => {
    const controller = new AbortController();  // for cleanup
    fetch('/api/posts', { signal: controller.signal })
      .then(r => r.json())
      .then(setPosts)
      .catch(err => { if (err.name !== 'AbortError') console.error(err); });
    return () => controller.abort();          // cleanup on unmount
  }, []);                                     // [] = mount only

  return {posts.map(p => {p.title})};
}

// ── Re-run when a dependency changes ────────────────
function Search({ query }) {
  const [results, setResults] = useState([]);
  useEffect(() => {
    if (!query) return;
    const id = setTimeout(() => {
      fetch(`/api/search?q=${query}`).then(r => r.json()).then(setResults);
    }, 300);                                  // debounce
    return () => clearTimeout(id);            // cancel on next keystroke
  }, [query]);                                // re-run when query changes
  return {results.map(r => {r.name})};
}

// ── Sync with an external system ────────────────────
useEffect(() => {
  window.addEventListener('resize', handleResize);
  return () => window.removeEventListener('resize', handleResize);
}, []);                                       // stable ref, empty deps OK
```

NOTES

- → `AbortController` — cancel in-flight fetch on cleanup; prevents setting state after unmount
- → `return () => …` — cleanup function; runs before next effect and on unmount; must return a function or nothing
- → `[], [query]` — deps control re-running; missing a dep is a bug (stale closure); extra deps cause needless re-runs
- → `setTimeout` debounce — cancel with `clearTimeout` in cleanup so rapid changes only fire one fetch
- → Strict Mode double-invoke — React mounts → unmounts → remounts to verify cleanup works; effects running twice in dev is expected
- → Do not set state unconditionally inside an effect without deps — it creates an infinite render loop

EFFECT TIMING

HOOK	RUNS	USE FOR
`useEffect`	After paint (async)	Fetch, subscriptions, timers
`useLayoutEffect`	After DOM, before paint (sync)	DOM measurements, scroll position
`useInsertionEffect`	Before DOM mutations	CSS-in-JS libraries only

WHAT NEEDS CLEANUP

EFFECT	CLEANUP
`fetch()`	`AbortController.abort()`
`setTimeout`	`clearTimeout(id)`
`setInterval`	`clearInterval(id)`
`addEventListener`	`removeEventListener`
`WebSocket / EventSource`	`.close()`
`Pub/sub subscription`	`unsubscribe()`

COMMON MISTAKES

- Omitting a dependency from the array — creates a stale closure; the effect reads an old value of the variable forever
- Async function directly as the effect: `useEffect(async () => …)` — the effect must return a cleanup function or nothing; async returns a Promise. Declare the async function inside and call it.
- Setting state in an effect with no dependency array — runs after every render, setting state causes another render, infinite loop

TIPS

- Install `eslint-plugin-react-hooks` and enable `exhaustive-deps` — it catches missing dependencies automatically
- Think of effects as "synchronise with X" not "run code at time Y" — this framing makes the dependency array obvious
- For data fetching, consider TanStack Query or SWR (Sheets 33–34) — they handle loading, caching, and error states far more robustly than raw `useEffect`

SUMMARY

- `useEffect(fn, [])` — run once on mount; `[deps]` re-runs on change
- `return () => cleanup()` — runs before next effect and on unmount
- `AbortController` — cancel fetches in cleanup to avoid state-on-unmount errors
- All scope values used inside effect must be listed as dependencies

useContext

SHARING STATE WITHOUT PROP DRILLING

Context provides a way to pass data through the component tree without threading props through every intermediate component. You create a context with `createContext`, wrap a subtree in its `Provider`, and any descendant can read the current value with `useContext`.

Context is ideal for **truly global or subtree-wide data**: current user, theme, locale, or feature flags. It is *not* a general-purpose state management solution — every component that calls `useContext` re-renders whenever the context value changes, so placing frequently-updating data in context (like a form's input value) can cause widespread unnecessary re-renders.

CONTEXT API

- `createContext(defaultValue)` — creates the context object; `defaultValue` used only when there is *no* matching Provider above
- `<Ctx.Provider value={…}>` — supplies the value to all descendants; re-renders them when value changes
- `useContext(Ctx)` — reads the nearest Provider's value; subscribes the component to updates
- Multiple Providers of the same context can nest — the nearest ancestor Provider wins
- Split large contexts — separate `ThemeContext` from `UserContext` to avoid over-rendering
- Pair with `useReducer` for context that involves complex state transitions (Sheet 15)

CONTEXT PATTERN

```jsx
// ── 1. Create the context (its own file: ThemeContext.jsx) ─
import { createContext, useContext, useState } from 'react';

const ThemeContext = createContext('light');
    // 'light' = default (no Provider)

// ── 2. Build a custom Provider component ──────────────────
export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');
  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      {children}
    </ThemeContext.Provider>
  );
}

// ── 3. Export a custom hook for consumers ─────────────────
export function useTheme() {
  const ctx = useContext(ThemeContext);
  if (!ctx) throw new Error('useTheme must be used inside ThemeProvider');
  return ctx;
}

// ── 4. Wrap the app (or a subtree) ────────────────────────
// main.jsx
<ThemeProvider>
  <App />
</ThemeProvider>

// ── 5. Consume anywhere in the tree ───────────────────────
function Toolbar() {
  const { theme, setTheme } = useTheme();
  return (
    <button onClick={() => setTheme(t => t === 'light' ? 'dark' : 'light')}>
      Current: {theme}
    </button>
  );
}
```

NOTES

- `createContext('light')` — default value is only used when a component reads context with *no* Provider anywhere above it in the tree
- `ThemeProvider` — wrapping the Provider in a custom component encapsulates the state and hides the raw context from consumers
- `useTheme()` — custom hook adds the null-check guard and gives consumers a clean, named API instead of raw `useContext(ThemeContext)`
- `value={{ theme, setTheme }}` — creating a new object on every render can cause all consumers to re-render; memoize with `useMemo` if performance matters
- Throwing in the custom hook — ensures the hook is never accidentally called outside its Provider with a cryptic error
- Context does not replace Redux/Zustand — for complex state logic or high-frequency updates, a dedicated state library is more efficient

PROPS VS CONTEXT

SCENARIO	USE
`1–2 levels deep`	Props — explicit, easy to trace
`3+ levels deep (prop drilling)`	Context or state manager
`Theme / locale / auth user`	Context — read-mostly global data
`High-frequency updates (typing)`	Local state or Zustand/Redux
`Component library internal state`	Context (compound components)

AVOIDING RE-RENDER STORMS

TECHNIQUE	HOW
`Split contexts`	Separate frequently-changing data into its own context
`Memoize value`	`useMemo` on the value object so reference stays stable
`Memoize consumers`	Wrap child components in `React.memo`
`State + dispatch split`	Two contexts: one for state (rare change), one for dispatch (stable)

COMMON MISTAKES

- Putting high-frequency state in context (e.g. form input value) — every consumer re-renders on every keystroke; keep frequently-changing state local
- Forgetting to wrap the app in the Provider — `useContext` silently returns the `createContext` default value, which is often `undefined`
- One giant context for all global state — unrelated components re-render when any part changes; split into focused contexts

TIPS

- Always export a custom `useFoo()` hook rather than the raw context — it adds null-check guards and lets you swap the implementation later
- Split state and dispatch into two contexts — dispatch is stable (never changes reference) so components that only dispatch never re-render when state changes
- For auth: store the user object in context but fetch it once in the Provider; avoid re-fetching on every tree re-render

SUMMARY

- `createContext(default)` — creates context; default only used without Provider
- `<Ctx.Provider value={…}>` — supplies value to the subtree
- `useContext(Ctx)` — reads nearest Provider value; subscribes to changes
- Wrap in custom hook — adds guard, cleaner API, easier to refactor

useRef

REFS: MUTABLE, NON-REACTIVE STORAGE

`useRef` returns a plain mutable object `{ current: initialValue }` that persists across renders. Unlike state, **mutating `ref.current` does not trigger a re-render**. This makes refs ideal for two distinct use cases: holding a reference to a **DOM node** so you can call imperative APIs on it (focus, scroll, measure), and storing a **mutable value** that you need to read in an effect or callback without causing re-renders.

To attach a ref to a DOM element, pass it as the `ref` prop. React sets `ref.current` to the DOM node after mount and sets it back to `null` on unmount. To forward a ref into a child component, the child must use `forwardRef`.

REF VS STATE

- `ref.current` — mutable; changing it does *not* trigger re-render
- `useState` — immutable snapshot; changing it triggers re-render
- Use **ref** for: timers IDs, previous-value tracking, DOM imperative APIs, interval handles
- Use **state** for: anything the UI must reflect when it changes
- `ref.current` is available immediately; state reads are snapshots of the render
- Do not read or write `ref.current` during rendering — only in effects and event handlers

USEREF PATTERNS

```
import { useRef, useEffect } from 'react';

// ── DOM ref: auto-focus an input on mount ──────────────
function SearchBox() {
  const inputRef = useRef(null);
  useEffect(() => { inputRef.current.focus(); }, []);
  return ;
}

// ── Mutable value: store interval ID ──────────────────
function Timer() {
  const [seconds, setSeconds] = useState(0);
  const intervalRef = useRef(null);

  const start = () => {
    intervalRef.current = setInterval(() => setSeconds(s => s + 1), 1000);
  };
  const stop = () => clearInterval(intervalRef.current);

  return <>StartStop{seconds}s;
}

// ── Track previous value ──────────────────────────────
function usePrevious(value) {
  const ref = useRef(undefined);
  useEffect(() => { ref.current = value; });   // runs after render
  return ref.current;                          // returns last render's value
}

// ── forwardRef: expose DOM ref to parent ──────────────
import { forwardRef } from 'react';

const FancyInput = forwardRef(function FancyInput(props, ref) {
  return ;
});

// Usage: parent can call ref.current.focus()
const ref = useRef(null);
```

NOTES

- `useRef(null)` — initialise to null for DOM refs; React fills it after mount
- `ref={inputRef}` — the special `ref` JSX prop; not passed as a normal prop to the component function
- `intervalRef.current = setInterval(…)` — storing the ID in a ref so it's readable in `stop` without causing re-renders
- `usePrevious` — reads ref *before* the effect runs (previous render's value), then the effect updates it after
- `forwardRef(fn)` — wraps the component so its parent can pass a `ref` that attaches to an inner DOM element
- Never read `ref.current` during render — the value is mutable and not part of React's snapshot model; it can change between renders

COMMON DOM REF METHODS

METHOD / PROPERTY	PURPOSE
`ref.current.focus()`	Focus an input or button
`ref.current.blur()`	Remove focus
`ref.current.scrollIntoView()`	Scroll element into viewport
`ref.current.getBoundingClientRect()`	Measure position and size
`ref.current.value`	Read uncontrolled input value
`ref.current.play() / .pause()`	Control video/audio elements

MUTABLE VALUE USE CASES

STORE IN REF	WHY NOT STATE
`setTimeout / setInterval ID`	Clearing doesn't need to trigger UI update
`Previous prop/state value`	Comparison only; no UI change needed
`WebSocket instance`	Stable reference across renders
`Render count (debugging)`	Counting renders without causing more renders
`Flag (isMounted, hasFired)`	Guard condition checked in effects/callbacks

COMMON MISTAKES

- Using a ref when state is needed — if the UI must reflect the value, use `useState`; refs are invisible to React's rendering
- Accessing `ref.current` inside the render function body — it may not yet be set (null on first render) and it's mutable so reads are unreliable
- Passing `ref` to a custom component without `forwardRef` — ref is not forwarded by default; React logs a warning and the ref stays null

TIPS

- Prefer **controlled state** for inputs; only reach for `ref.current.value` (uncontrolled) for file inputs or when integrating with non-React libraries
- Use `useImperativeHandle` (with `forwardRef`) to expose only a curated API to the parent instead of the whole DOM node
- Store any stable object (WebSocket, third-party class instance) in a ref — it survives re-renders without triggering them

SUMMARY

- `useRef(null)` — mutable box; `.current` change does not re-render
- `ref={el}` — attach to JSX; React sets `.current` to the DOM node
- `forwardRef` — required to pass a ref into a custom component
- Read/write refs only in effects and event handlers, never during render

useMemo & useCallback

MEMOISING VALUES AND FUNCTIONS

`useMemo` caches a *computed value* between renders. `useCallback` caches a *function definition*. Both accept a dependency array; they return the cached version until a dependency changes. Their primary purpose is **referential stability**: React's reconciler uses `Object.is` to compare props, so passing a new object or function on every render breaks `React.memo` and `useEffect` dep checks even when the content is identical.

Both hooks add overhead — the comparison itself has a cost. **Do not memoize by default**. Reach for them only when a profiler shows a measurable problem, a computation is genuinely expensive, or a stable reference is required as a `useEffect` dependency.

WHEN TO USE EACH

- `useMemo` — expensive calculation (sorting/filtering large arrays, heavy transforms)
- `useMemo` — stable object/array reference needed as a `useEffect` dep or prop to a memoised child
- `useCallback` — stable function reference passed as a prop to a `React.memo` child
- `useCallback` — function used as a `useEffect` dependency to avoid infinite loops
- `useCallback(fn, deps)` is equivalent to `useMemo(() => fn, deps)`
- Profile first — memoization adds complexity; only useful if the re-render cost exceeds the memo overhead

USEMEMO & USECALLBACK EXAMPLES

```
import { useState, useMemo, useCallback, memo } from 'react';

// ── useMemo: expensive filter ──────────────────────────
function ProductList({ products, minPrice }) {
  const filtered = useMemo(
    () => products.filter(p => p.price >= minPrice),
    [products, minPrice]           // only recompute when these change
  );
  return {filtered.map(p => {p.name})};
}

// ── useCallback: stable handler for memoised child ─────
const Row = memo(function Row({ item, onDelete }) {
  console.log('Row rendered:', item.id);
  return {item.name}  onDelete(item.id)}>×;
});

function List({ items }) {
  const [items_, setItems] = useState(items);

  /*
  * Without useCallback:
  *     new function ref each render → Row always re-renders
  * With useCallback:
  *     same function ref → Row skips re-render when items_ unchanged
  */
  const handleDelete = useCallback(
    (id) => setItems(prev => prev.filter(i => i.id !== id)),
    []                             // setItems is stable — empty deps OK
  );

  return {items_.map(i => )};
}

// ── Stable context value ───────────────────────────────
const value = useMemo(() => ({ user, logout }), [user, logout]);
…
```

NOTES

- `useMemo(() => …, [deps])` — the factory function runs once; result cached until deps change
- `memo(Component)` — skips re-rendering if props are shallowly equal; ineffective if any prop is a new object/function reference each render
- `useCallback(fn, [])` — empty deps means the function is created once and reused; safe when the function only calls stable setters
- `setItems` from `useState` is guaranteed stable — it never changes reference, so it does not need to be in a `useCallback` dep array
- `useMemo` on context value — the `{ user, logout }` object would be a new reference every render without memoisation, causing all consumers to re-render
- React may discard the memo cache in low-memory situations — memoization is an optimisation hint, not a guarantee

USEMEMO VS USECALLBACK

	USEMEMO	USECALLBACK
Caches	A computed value	A function definition
Returns	Result of calling fn	The fn itself (not called)
Syntax	`useMemo(() => val, [deps])`	`useCallback(fn, [deps])`
Use for	Expensive computations	Stable event handler references

MEMOIZE OR NOT?

SITUATION	MEMOIZE?
`Simple arithmetic / string concat`	✖ too cheap
`Filtering 10 000+ item array`	✔ expensive
`Object passed as useEffect dep`	✔ stable ref needed
`Handler not passed to memo child`	✖ no benefit
`Function passed to React.memo child`	✔ prevents re-render
`Context value object`	✔ prevents all consumers re-rendering

COMMON MISTAKES

- Memoizing everything by default — adds cognitive overhead and a small performance cost without benefit when renders are already cheap
- Using `useCallback` without `React.memo` on the child — if the child re-renders anyway, the stable function reference provides no benefit
- Putting an object or array literal in the dep array of `useMemo` — the literal creates a new reference every render, breaking the memo every time

TIPS

- Profile with React DevTools Profiler before memoizing — most renders are fast enough that memoization adds complexity without payoff
- `useCallback` + `React.memo` work as a pair — one without the other rarely helps; apply them together to expensive child components
- Setter functions from `useState` and `dispatch` from `useReducer` are always stable — never include them in dependency arrays

SUMMARY

- `useMemo(() => val, [deps])` — cache computed value; re-run on dep change
- `useCallback(fn, [deps])` — cache function reference; stable across renders
- Both require `React.memo` on the child to actually prevent re-renders
- Profile first — premature memoization adds complexity without benefit

useReduce

MANAGING COMPLEX STATE TRANSITIONS

`useReducer` is an alternative to `useState` for state that involves **multiple sub-values** or **complex transition logic**. It follows the Redux pattern: state transitions are described as plain **action objects**, and a pure **reducer function** computes the next state from the current state and the action.

The hook returns the current state and a `dispatch` function. `dispatch` is **always stable** (never changes reference), making it safe to pass to child components or use as a `useEffect` dependency without memoization. Pair `useReducer` with context to build a lightweight global store without Redux.

USESTATE VS USEREDUCER

- Use `useState` for: independent primitive values, simple toggle, single-field updates
- Use `useReducer` for: multiple related fields that change together, next state depends on action type, complex update logic
- `dispatch` is **always stable** — safe in dep arrays and as props without `useCallback`
- Reducer must be a **pure function** — no side effects, no mutations, same input → same output
- Action objects: convention is `{ type: 'ACTION_NAME', payload: … }`
- Third arg `init` — lazy initializer: `useReducer(reducer, arg, initFn)` calls `initFn(arg)` on mount

USEREDUCER IN PRACTICE

```
import { useReducer } from 'react';

// ── 1. Define initial state ──────────────────────
const initialState = { count: 0, step: 1 };

// ── 2. Write a pure reducer ──────────────────────
function reducer(state, action) {
  switch (action.type) {
    case 'increment':
      return { ...state, count: state.count + state.step };
    case 'decrement':
      return { ...state, count: state.count - state.step };
    case 'reset':
      return initialState;
    case 'setStep':
      return { ...state, step: action.payload };
    default:
      throw new Error(`Unknown action: ${action.type}`);
  }
}

// ── 3. Use in a component ────────────────────────
function Counter() {
  const [state, dispatch] = useReducer(reducer, initialState);

  return (

      Count: {state.count} (step: {state.step})
       dispatch({ type: 'increment' })}>+
       dispatch({ type: 'decrement' })}>-
       dispatch({ type: 'reset' })}>Reset
       dispatch({ type: 'setStep', payload: Number(e.target.value) })}
      />

  );
}
```

NOTES

- `useReducer(reducer, initialState)` — returns `[state, dispatch]`; same destructuring pattern as `useState`
- `{ ...state, count: … }` — reducer returns a new state object; never mutate the existing state argument
- `switch (action.type)` — standard pattern; the `default` throw catches typos in dispatch calls during development
- `action.payload` — convention for carrying data; not a React requirement; any shape works as long as the reducer handles it
- `dispatch` is stable — unlike `useState` setters it is guaranteed to be the same reference across renders; safe as a dep
- Reducer lives outside the component — easy to unit test in isolation with plain function calls

USEREDUCER API

PART	DESCRIPTION
`reducer(state, action)`	Pure function; returns next state
`initialState`	Starting value for state
`[state, dispatch]`	Current state + dispatch function
`dispatch(action)`	Sends action to reducer; triggers re-render
`action.type`	String identifying the transition
`action.payload`	Optional data the reducer uses

USEREDUCER + CONTEXT PATTERN

CONTEXT	VALUE
`StateContext`	The state object — changes frequently
`DispatchContext`	The dispatch fn — always stable; never changes

CONSUMER READS	RE-RENDERS WHEN
`StateContext only`	State changes
`DispatchContext only`	Never (dispatch is stable)

COMMON MISTAKES

- Mutating state inside the reducer (`state.count++; return state`) — returns the same reference; React bails out of the re-render
- Adding side effects in the reducer (fetches, timers) — reducers must be pure; side effects belong in `useEffect` or event handlers
- Missing `default` case — unhandled action types silently return `undefined` , wiping all state

TIPS

- Define the reducer outside the component — keeps the component clean and makes the reducer trivially unit-testable
- Use string constants for action types (`const INCREMENT = 'increment'`) or a TypeScript union — eliminates typo bugs
- Pair `useReducer` + context for app-wide state (Sheet 25): put state in one context and dispatch in another for minimal re-renders

SUMMARY

- `useReducer(reducer, init)` — returns `[state, dispatch]`
- `reducer(state, action) => newState` — pure function, no mutation
- `dispatch({ type, payload })` — triggers the reducer; dispatch is always stable
- Prefer over `useState` when multiple values change together or logic is complex

Custom Hooks

use* naming · extracting logic · useFetch · useLoc

EXTRACTING REUSABLE HOOK LOGIC

A **custom hook** is a plain JavaScript function whose name starts with `use` and that calls other hooks inside it. Custom hooks let you extract stateful logic from a component and share it across multiple components — each call to the hook gets its own isolated state and effect lifecycle.

The `use` prefix is **required**: it signals to React's linter and runtime that the function follows the Rules of Hooks. Any function that calls `useState`, `useEffect`, or other hooks must start with `use`, or the `exhaustive-deps` lint rule and Strict Mode double-invoke won't apply correctly.

CUSTOM HOOK RULES

- Name must start with `use` — e.g. `useFetch`, `useDebounce`, `useWindowSize`
- Can call any other hooks — built-in or other custom hooks
- Each component that calls the hook gets **independent state** — hooks are not shared singletons
- Return anything useful — a value, an array, an object, or nothing
- Live in `src/hooks/` — one hook per file is convention, filename matches hook name
- Document the return shape with JSDoc or TypeScript types — callers can't see inside the hook

CUSTOM HOOK EXAMPLES

```
// ── useFetch: encapsulate data-fetching pattern ──────────
function useFetch(url) {
  const [data, setData]       = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError]     = useState(null);

  useEffect(() => {
    const controller = new AbortController();
    setLoading(true);
    fetch(url, { signal: controller.signal })
      .then(r => { if (!r.ok) throw new Error(r.status); return r.json(); })
      .then(setData)
      .catch(e => { if (e.name !== 'AbortError') setError(e.message); })
      .finally(() => setLoading(false));
    return () => controller.abort();
  }, [url]);

  return { data, loading, error };   // named object — easy to destructure
}

// Usage
function Posts() {
  const { data, loading, error } = useFetch('/api/posts');
  if (loading) return Loading…;
  if (error)   return Error: {error};
  return {data.map(p => {p.title})};
}

// ── useLocalStorage: state persisted across page loads ────
function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    try { return JSON.parse(localStorage.getItem(key)) ?? initialValue; }
    catch { return initialValue; }
  });
  const set = (v) =>
    { setValue(v); localStorage.setItem(key, JSON.stringify(v)); };
  return [value, set];
}
```

NOTES

- → `{ data, loading, error }` — returning a named object scales better than a tuple when there are 3+ values
- → `controller.abort()` — cleanup in `useFetch` cancels the in-flight request when `url` changes or component unmounts
- → `[url]` as dep — `useFetch` re-fires whenever the url changes, just like a component re-renders when props change
- → `useState(() => JSON.parse(…))` — lazy initializer so localStorage is only read on mount, not every render
- → Each caller of `useFetch` gets its own `data/loading/error` state — custom hooks are not shared singletons
- → Hooks can compose other custom hooks — `useFetch` could internally call `useDebounce` to debounce its URL

COMMON CUSTOM HOOKS

HOOK	WHAT IT ENCAPSULATES
`useFetch(url)`	Fetch + loading + error state
`useLocalStorage(key, init)`	State synced to localStorage
`useDebounce(value, delay)`	Delays updating until input settles
`useWindowSize()`	Tracks window width/height on resize
`useOnClickOutside(ref, fn)`	Calls fn when click outside element
`useMediaQuery(query)`	Returns boolean for CSS media query
`usePrevious(value)`	Returns the value from the previous render

RETURN VALUE CONVENTIONS

PATTERN	EXAMPLE	USE WHEN
`Tuple`	`[value, setValue]`	Mirrors useState; caller picks names
`Object`	`{ data, loading, error }`	Multiple named values; scales well
`Single value`	`return windowWidth`	Only one thing to return
`Nothing`	`return`	Side-effect only (event listener)

COMMON MISTAKES

- Naming a hook without the `use` prefix — React's linter won't enforce Rules of Hooks and Strict Mode won't double-invoke it
- Sharing state across components by calling the hook once at the top level — each hook call is isolated; to share state, lift it up or use context
- Missing cleanup inside the hook's `useEffect` — the memory leak is now hidden inside the hook and harder to diagnose

TIPS

- Extract a custom hook as soon as you copy a `useState` + `useEffect` pattern into a second component — that's the signal
- Return a named object (`{ data, loading, error }`) rather than a positional tuple when the hook returns more than two values
- Check **usehooks.com** or **ahooks** before building common hooks from scratch — battle-tested implementations handle edge cases you might miss

SUMMARY

- `use* naming` — required; enables React lint rules and Strict Mode checks
- Each call = isolated state — not a shared singleton across components
- Return object for 3+ values; tuple for 2; single value for 1
- Always clean up effects inside the hook — leaks are hidden from callers

useLayoutEffect & useDebugValue

TWO SPECIALISED HOOKS

`useLayoutEffect` has the same signature as `useEffect` but fires **synchronously after DOM mutations and before the browser paints**. Use it when you must read layout (element dimensions, scroll position) and immediately update state or style before the user sees the initial render — otherwise you get a visible flicker. For everything else, prefer `useEffect`.

`useDebugValue` is for **custom hooks only**. It adds a label to the hook in React DevTools so you can see the hook's current value without opening the component. It accepts an optional formatter function that is only called when DevTools is open — use it for expensive formatting.

USELAYOUTEFFECT NOTES

- Runs **synchronously** after DOM update, before paint — can block the browser if slow
- Same signature as `useEffect` — accepts cleanup return and dependency array
- Use for: measuring DOM elements, syncing external libraries with DOM, preventing flicker on initial render
- Avoid on the server — `useLayoutEffect` does not run during SSR; React warns. Use `useEffect` or wrap in a check
- `useDebugValue` — call inside a custom hook, not a component
- Formatter arg — `useDebugValue(value, v => format(v))` — only called when DevTools is open

USELAYOUTEFFECT & USEDEBUGVALUE

```
import { useRef, useState, useLayoutEffect, useDebugValue } from 'react';

// ── useLayoutEffect: measure width before first paint ──────
function useElementWidth(ref) {
  const [width, setWidth] = useState(0);

  useLayoutEffect(() => {
    if (!ref.current) return;
    const observer = new ResizeObserver(([entry]) => {
      setWidth(entry.contentRect.width);
    });
    observer.observe(ref.current);
    setWidth(ref.current.getBoundingClientRect().width); // initial read
    return () => observer.disconnect();
  }, [ref]);

  return width;
}

// If we used useEffect here, the component would paint with
// width=0 first, then repaint with the real width → visible flicker.

function Banner() {
  const ref = useRef(null);
  const width = useElementWidth(ref);
  return Width: {width}px;
}

// ── useDebugValue: label in React DevTools ─────────────────
function useOnlineStatus() {
  const [online, setOnline] = useState(navigator.onLine);

  useDebugValue(online ? 'Online' : 'Offline');   // shows in DevTools

  useEffect(() => {
    const on  = () => setOnline(true);
    const off = () => setOnline(false);
    window.addEventListener('online', on);
    window.addEventListener('offline', off);
    return () => {
      window.removeEventListener('online', on);
      window.removeEventListener('offline', off);
    };
  }, []);

  return online;
}
```

NOTES

- → `getBoundingClientRect()` — reads the layout (requires DOM); only accurate after the DOM has been mutated; safe inside `useLayoutEffect`
- → `ResizeObserver` — observes size changes; disconnect in cleanup to avoid memory leaks
- → Flicker scenario — if the width affects rendering (e.g. truncation), using `useEffect` would paint the wrong layout first; `useLayoutEffect` blocks paint until the correct width is set
- → `useDebugValue(online ? 'Online' : 'Offline')` — React DevTools shows "useOnlineStatus: Online" next to the hook in the component tree
- → Formatter arg — pass as second arg to avoid expensive computation when DevTools is closed: `useDebugValue(date, d => d.toISOString())`
- → SSR guard — if using `useLayoutEffect` in a library, suppress the server warning with: `const useIsomorphicLayoutEffect = typeof window !== 'undefined' ? useLayoutEffect : useEffect`

EFFECT TIMING COMPARISON

HOOK	WHEN IT FIRES	BLOCKING?	USE FOR
`useEffect`	After paint	✖	Fetch, subscriptions, timers
`useLayoutEffect`	After DOM, before paint	✔	DOM measurement, tooltip position
`useInsertionEffect`	Before any DOM mutations	✔	CSS-in-JS style injection only

USEDEBUGVALUE USAGE

CALL	DEVTOOLS SHOWS
`useDebugValue(value)`	The value directly
`useDebugValue(date, d => d.toISOString())`	Formatted string (lazy)
`useDebugValue('Online')`	Static label string
`(not called)`	Shows hook name only

COMMON MISTAKES

- Using `useLayoutEffect` for data fetching — it blocks paint; fetches should use `useEffect` which runs asynchronously after the frame is displayed
- Using `useLayoutEffect` in an SSR app without a guard — React warns in production and the effect is silently skipped on the server
- Calling `useDebugValue` inside a component — it's for custom hooks only; calling it in a component does nothing useful

TIPS

- Default to `useEffect`; only switch to `useLayoutEffect` when you observe a visible flicker caused by a layout read
- Create `useIsomorphicLayoutEffect` in a shared file for any hook that uses layout reads — safe in both SSR and CSR environments
- Add `useDebugValue` to every non-trivial custom hook — it costs nothing in production and makes debugging with DevTools much faster

SUMMARY

- `useLayoutEffect` — fires before paint; use for DOM measurements to avoid flicker
- Same API as `useEffect` — cleanup return + dependency array
- `useLayoutEffect` + SSR — needs a guard; use isomorphic pattern
- `useDebugValue(label)` — custom hooks only; shows value in React DevTools

useTransition & useId

REACT 18 CONCURRENT HOOKS

`useTransition` marks a state update as **non-urgent**. React can interrupt and deprioritise transitions to keep the UI responsive — typing in an input stays snappy even if a heavy list re-render is triggered by the same interaction. `isPending` lets you show a loading indicator while the transition is in progress.

`useId` generates a **stable, unique ID** that is consistent across server and client renders. Use it to link `<label>` + `<input>` pairs or for ARIA attributes when you need an ID that won't collide across multiple component instances but must match between SSR and hydration.

USETRANSITION & USEID EXAMPLES

```
import { useState, useTransition, useId } from 'react';

// ── useTransition: keep input responsive during heavy render ─
function SearchPage() {
  const [query, setQuery]     = useState('');
  const [results, setResults] = useState([]);
  const [isPending, startTransition] = useTransition();

  function handleChange(e) {
    setQuery(e.target.value);              // urgent — updates instantly

    startTransition(() => {
      setResults(heavyFilter(e.target.value)); // non-urgent — can be interrupted
    });
  }

  return (
    <div>
      <input value={query} onChange={handleChange} />
      {isPending && <span>Filtering…</span>}
      <ul>{results.map(r => <li key={r.id}>{r.name}</li>)}</ul>
    </div>
  );
}

// ── useId: stable, SSR-safe IDs ──────────────────────────────
function EmailField() {
  const id = useId();        // e.g. ":r3:"
  return (
    <div>
      <label htmlFor={id}>Email</label>
      <input id={id} type="email" />
    </div>
  );
}

// ── Multiple fields from one useId ───────────────────────────
function ProfileForm() {
  const id = useId();
  return (
    <>
      <label htmlFor={id + '-name'}>Name</label>
      <input id={id + '-name'} />
      <label htmlFor={id + '-bio'}>Bio</label>
      <textarea id={id + '-bio'}></textarea>
    </>
  );
}
```

KEY POINTS

- `startTransition` — wraps the state update you want to deprioritise; the update can be interrupted by urgent updates
- `isPending` — `true` while the transition hasn't committed; use for spinner/skeleton UI
- Transitions only work with state setters and `dispatch` — not with timeouts or Promises directly
- `startTransition` (top-level import) — same as hook version but without `isPending`
- `useId()` — returns a string like `:r0:`; stable across SSR + hydration
- Do not use `useId` as a list `key` — use your data's ID instead

NOTES

- → `setQuery` outside `startTransition` — urgent; the input value updates on every keystroke without delay
- → `startTransition(() => setResults(…))` — React can pause this update if the user types another character before it finishes
- → `isPending` — true while the transition is computing; use to show a spinner or dim the results list
- → `heavyFilter` — must be synchronous; transitions only defer rendering, not JavaScript execution time; very slow JS still blocks
- → `useId()` — each component instance gets its own unique ID; calling it three times in one component returns three different IDs
- → `id + '-name'` — suffix one `useId` value for multiple related fields instead of calling `useId` repeatedly

URGENT VS TRANSITION UPDATES

UPDATE TYPE	EXAMPLE	BEHAVIOUR
Urgent	Typing, clicking, pressing	Interrupts transitions; always syncs immediately
Transition	Tab switch, search results	Can be interrupted; React shows stale UI while pending

USEID USE CASES

USE CASE	EXAMPLE
label + input pair	id={id} htmlFor={id}
ARIA describedby	aria-describedby={id}
Multiple fields in one component	id + '-name', id + '-email'
SSR-safe unique ID	Consistent across server/client render

COMMON MISTAKES

- Wrapping urgent updates in `startTransition` — user-typed input must update instantly; putting `setQuery` in a transition makes the input feel laggy
- Using `useId()` as a list `key` — the ID is stable per instance but not per item; use your data's unique ID for keys
- Expecting `startTransition` to speed up slow JavaScript — it only defers the render; a 500ms filter function still blocks for 500ms

TIPS

- Use `useTransition` for tab switches and search result updates — the stale content is better than a blank screen while recalculating
- Combine `useTransition` with `Suspense` — React can show a fallback while the transition resolves a lazy component
- Always use `useId` instead of incrementing counters or `Math.random()` for ARIA and label IDs — it's SSR-safe and collision-free

SUMMARY

- `startTransition(() => setState(…))` — marks update as non-urgent; can be interrupted
- `isPending` — true while transition is in progress; use for loading UI
- `useId()` — stable unique ID; consistent across SSR and hydration
- Suffix one `useId` for multiple related fields: `id + '-field'`

use() & React 19 Hooks

REACT 19 NEW PRIMITIVES

React 19 (stable 2024) introduced several new APIs. `use()` is a special function — not a traditional hook — that can be called **conditionally and inside loops**. It unwraps a `Promise` (suspending until resolved) or reads a `Context` value. It must still be called inside a component or hook.

`useOptimistic` applies a UI update immediately while an async action is in-flight, then reverts to the server-confirmed value when the action settles. `useFormStatus` reads the submission state of the nearest parent `<form>`. `useActionState` manages the state cycle of a form action function.

REACT 19 HOOKS IN PRACTICE

```
import { use, useOptimistic, useFormStatus, useActionState, Suspense } from 'react';

// ── use(Promise): data fetching with Suspense ──────────────
function UserCard({ userPromise }) {
  const user = use(userPromise);      // suspends until resolved
  return <p>{user.name}</p>;
}
// Wrap in Suspense:
<Suspense fallback={<p>Loading…</p>}>
  <UserCard userPromise={fetchUser(1)} />
</Suspense>

// ── useOptimistic: instant list update ─────────────────────
function TodoList({ todos, addTodo }) {
  const [optimisticTodos, addOptimistic] = useOptimistic(
    todos,
    (state, newTodo) => [...state, { ...newTodo, pending: true }]
  );
  async function handleAdd(text) {
    addOptimistic({ id: Date.now(), text }); // instant UI update
    await addTodo(text);               // server action — may revert on error
  }
  return <ul>{optimisticTodos.map(t =>
    <li key={t.id}>{t.text}{t.pending ? ' …' : ''}</li>)}</ul>;
}

// ── useFormStatus: pending-aware submit button ─────────────
function SubmitButton() {
  const { pending } = useFormStatus();  // must be inside a <form> descendant
  return <button type="submit" disabled={pending}>
  {pending ? 'Saving…' : 'Save'}</button>;
}

// ── useActionState: manage form action result ──────────────
async function saveAction(prevState, formData) {
  const name = formData.get('name');
  if (!name) return { error: 'Name required' };
  await api.save(name);
  return { success: true };
}
function NameForm() {
  const [state, formAction, isPending] = useActionState(saveAction, null);
  return (
    <form action={formAction}>
      <input name="name" />
      {state?.error && <p>{state.error}</p>}
      <SubmitButton />
    </form>
  );
}
```

REACT 19 API SUMMARY

- `use(promise)` — suspends until the promise resolves; must be wrapped in `<Suspense>`
- `use(Context)` — reads context; can be called conditionally unlike `useContext`
- `useOptimistic(state, reducer)` — returns `[optimisticState, addOptimistic]`; optimistic state reverts after action
- `useFormStatus()` — returns `{ pending, data, method, action }`; must be inside a `<form>` descendant
- `useActionState(action, initialState)` — returns `[state, formAction, isPending]`
- Server Actions — async functions marked `'use server'`; can be passed as `form action`

NOTES

- `use(userPromise)` — the component suspends (throws the promise) until it resolves; the nearest `<Suspense>` shows the fallback
- `addOptimistic` — immediately applies the optimistic update; React reverts to `todos` (the real state) after the async action completes
- `pending: true` — a flag on the optimistic item so you can show a spinner or dim colour while the server is confirming
- `useFormStatus` must be in a *child* of the form — it cannot be in the same component as the `<form>` element
- `formAction` from `useActionState` — pass to `<form action={formAction}>`; React calls your action with `(prevState, formData)`
- Server Actions (`'use server'`) — can be passed directly to `form action` prop; works with both `useActionState` and progressive enhancement

USE() VS USECONTEXT / SUSPENSE

FEATURE	USE()	USECONTEXT / USEEFFECT
Conditionally callable	✔	✖
Inside loops	✔	✖
Unwrap Promise	✔	✖
Read Context	✔	✔
Requires Suspense boundary	✔ (Promise)	✖

FORM HOOKS AT A GLANCE

HOOK	RETURNS	USE FOR
useFormStatus	{ pending, data }	Disable submit button while pending
useActionState	[state, action, isPending]	Handle form action result & errors
useOptimistic	[optimistic, addOptimistic]	Instant UI feedback before server confirms

COMMON MISTAKES

- Calling `use(promise)` without a `<Suspense>` boundary — the component throws and React has no fallback to show; the app crashes
- Placing `useFormStatus` in the same component as the `<form>` — it only reads status from a *parent* form; put it in a child component
- Creating the promise inside the component body without memoizing — a new promise on every render suspends on every render; create promises outside or memoize with `useMemo`

TIPS

- Use `useOptimistic` for any mutation that should feel instant — likes, todo adds, cart updates — and let the server confirm or revert
- Extract the submit button into its own component to use `useFormStatus` cleanly — it's a natural split anyway for reuse
- These React 19 APIs shine with Next.js 14+ Server Actions — they were designed together; check Next.js docs for the full integration pattern

SUMMARY

- `use(promise)` — suspends until resolved; needs `<Suspense>`
- `useOptimistic` — instant UI update; reverts when async action settles
- `useFormStatus` — reads parent form's pending state; use in a child component
- `useActionState` — manages form action state cycle and errors

Event Handling

REACT SYNTHETIC EVENTS

React wraps native browser events in a **SyntheticEvent** object that normalises differences across browsers. The API mirrors the DOM event interface (`event.target`, `event.preventDefault()`, `event.stopPropagation()`) so existing DOM knowledge transfers directly.

Event handlers are passed as **camelCase props** (`onClick`, `onChange`, `onSubmit`) and receive the event object as their first argument. React attaches a single listener at the root (event delegation) rather than on every element — this is automatic and transparent to you.

COMMON EVENT PROPS

- `onClick` — any element; fires on left-click and Enter/Space on focused buttons
- `onChange` — inputs, selects, textareas; fires on every value change (not on blur like native)
- `onSubmit` — form element; always call `e.preventDefault()` to prevent page reload
- `onKeyDown` / `onKeyUp` — keyboard events; check `e.key` (e.g. `'Enter'`, `'Escape'`)
- `onFocus` / `onBlur` — focus events; bubble in React unlike native DOM
- `onMouseEnter` / `onMouseLeave` — do not bubble; use for hover states

EVENT HANDLING EXAMPLES

```
// ── Basic click handler ──────────────────────────────
function Counter() {
  const [count, setCount] = useState(0);
  function handleClick() { setCount(c => c + 1); }
  return <button onClick={handleClick}>{count}</button>;
}

// ── Passing arguments via closure ────────────────────
function TodoList({ todos, onDelete }) {
  return (
    <ul>
      {todos.map(todo => (
        <li key={todo.id}>
          {todo.text}
          <button onClick={() => onDelete(todo.id)}>×</button>
        </li>
      ))}
    </ul>
  );
}

// ── Form submit ──────────────────────────────────────
function LoginForm() {
  const [email, setEmail] = useState('');

  function handleSubmit(e) {
    e.preventDefault();                    // stop page reload
    console.log('Submitted:', email);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="email"
        value={email}
        onChange={e => setEmail(e.target.value)}
      />
      <button type="submit">Login</button>
    </form>
  );
}

// ── Keyboard event ───────────────────────────────────
<input onKeyDown={e => { if (e.key === 'Enter') handleSubmit(); }} />
```

NOTES

- → `onClick={handleClick}` — pass the reference; `onClick={handleClick()}` calls it immediately on render
- → `() => onDelete(todo.id)` — closure captures `todo.id`; necessary when you need to pass an argument to the handler
- → `e.preventDefault()` on form submit — required to stop the browser from doing a full-page POST request
- → `e.target.value` — read the input's current value; this is the controlled-input pattern when paired with `useState`
- → `e.key === 'Enter'` — use `e.key` (string); avoid the deprecated `e.keyCode` (number)
- → Inline arrow functions — create a new function reference every render; fine for most cases but can break `React.memo` on children

SYNTHETICEVENT PROPERTIES

PROPERTY / METHOD	DESCRIPTION
`e.target`	Element that triggered the event
`e.target.value`	Current value of an input
`e.target.checked`	Checked state of a checkbox
`e.currentTarget`	Element the handler is attached to
`e.preventDefault()`	Stop default browser action (form submit, link navigate)
`e.stopPropagation()`	Stop event bubbling to parent elements
`e.key`	Key pressed as a string: 'Enter', 'ArrowDown', 'a'

HANDLER PATTERNS

PATTERN	EXAMPLE
Named handler	`onClick={handleClick}`
Inline arrow	`onClick={() => setOpen(true)}`
With argument	`onClick={() => del(item.id)}`
Event + argument	`onChange={e => setVal(e.target.value)}`
Prevent default	`onSubmit={e => { e.preventDefault(); … }}`

COMMON MISTAKES

- `onClick={doSomething()}` — calling the function instead of passing it; the function fires on render, not on click
- Forgetting `e.preventDefault()` on form submit — the page reloads and all React state is lost
- Using `e.keyCode` instead of `e.key` — `keyCode` is deprecated; `e.key` gives a human-readable string

TIPS

- Name event handlers `handle + EventName` by convention (`handleClick`, `handleSubmit`) and callback props `on + EventName` (`onClick`, `onSave`)
- Prefer named handler functions over long inline arrows for complex logic — easier to read, test, and profile
- For forms, consider React Hook Form (Sheet 23) to avoid manually wiring every `onChange` and `value`

SUMMARY

- `onClick={fn}` — camelCase props; pass reference, not a call
- `e.preventDefault()` — stop default browser action on submit
- `e.target.value` — read input value in `onChange`
- `e.key` — readable key name; preferred over deprecated `keyCode`

Controlled Components

REACT AS THE SINGLE SOURCE OF TRUTH

A **controlled component** is a form element whose value is driven by React state. You set `value={stateVar}` on the element and update the state in `onChange`. The DOM element's displayed value always reflects state — the component "controls" the input.

The benefit is that you always know the current value of every field from React state — no need to read the DOM. This enables instant validation, conditional disabling, and formatting-on-type. The cost is that every keystroke fires an `onChange` and a state update. For most forms this is fast enough; for very large forms consider React Hook Form (Sheet 23).

CONTROLLED ELEMENT CHEATSHEET

- `<input value={v} onChange={e => set(e.target.value)} />` — text, email, password, number
- `<input type="checkbox" checked={v} onChange={e => set(e.target.checked)} />`
- `<input type="radio" checked={v === opt} onChange={() => set(opt)} />`
- `<select value={v} onChange={e => set(e.target.value)}>` — single select
- `<select multiple value={arr} onChange={…}>` — multi-select; value is an array
- `<textarea value={v} onChange={e => set(e.target.value)} />` — note: self-closing OK in React

CONTROLLED FORM EXAMPLE

```jsx
import { useState } from 'react';

function SignupForm() {
  const [form, setForm] = useState({ name: '', email: '', role: 'viewer', agree: false });
  const [errors, setErrors] = useState({});

  const set = (field) => (e) =>
    setForm(prev => (
      { ...prev, [field]: e.target.type === 'checkbox' ? e.target.checked : e.target.value }
    ));

  function validate() {
    const errs = {};
    if (!form.name.trim())               errs.name  = 'Name required';
    if (!/\S+@\S+\.\S+/.test(form.email)) errs.email = 'Invalid email';
    if (!form.agree)                     errs.agree = 'Must agree';
    return errs;
  }

  function handleSubmit(e) {
    e.preventDefault();
    const errs = validate();
    if (Object.keys(errs).length) { setErrors(errs); return; }
    console.log('Submit:', form);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input value={form.name} onChange={set('name')} placeholder="Name" />
      {errors.name  && <p>{errors.name}</p>}

      <input type="email" value={form.email} onChange={set('email')} placeholder="Email" />
      {errors.email && <p>{errors.email}</p>}

      <select value={form.role} onChange={set('role')}>
        <option value="viewer">Viewer</option>
        <option value="editor">Editor</option>
        <option value="admin">Admin</option>
      </select>

      <input type="checkbox" id="agree" checked={form.agree} onChange={set('agree')} />
      <label htmlFor="agree">I agree</label>
      {errors.agree && <p>{errors.agree}</p>}

      <button type="submit" disabled={!form.agree}>Sign Up</button>
    </form>
  );
}
```

NOTES

- → `[field]` computed key — one generic setter for all fields; avoids duplicating `onChange` for each field
- → `e.target.checked` — checkboxes use `checked`, not `value`; the generic setter handles both
- → `{ ...prev, [field]: val }` — spread to create new object; React compares references to decide whether to re-render
- → `select value={form.role}` — set the selected option via React state; do not use `selected` on individual `<option>` elements
- → `disabled={!form.agree}` — real-time disable; possible because React always knows the current field values
- → For forms with 10+ fields or complex async validation, React Hook Form (Sheet 23) reduces boilerplate significantly

CONTROLLED VS UNCONTROLLED

FEATURE	CONTROLLED	UNCONTROLLED
Value source	React state	DOM (via ref)
Read value	From state variable	`ref.current.value`
Validation	On every keystroke	On submit only
Format on type	✔	✖
Boilerplate	More (onChange per field)	Less
File input	✖ always uncontrolled	✔

COMMON VALIDATION PATTERNS

CHECK	PATTERN
Required	`!value.trim()`
Min length	`value.length < 8`
Email format	`/\S+@\S+\.\S+/.test(value)`
Show error	`{error && <p>{error}</p>}`
Disable submit	`disabled={!isValid}`

COMMON MISTAKES

- Setting `value` without an `onChange` — React makes the input read-only; the user can't type. Use `defaultValue` for uncontrolled, or add `onChange`
- Using `selected` on individual `<option>` elements — React ignores it; set `value` on the `<select>` instead
- Mutating the form object directly (`form.name = val`) — same reference, no re-render; always spread to create a new object

TIPS

- Use a single `useState` object for all form fields — one state update per field change, consistent shape
- Validate only on blur (not on every keystroke) for a less aggressive UX — set an error only after the user leaves the field
- For file inputs, always use `useRef` — `<input type="file">` cannot be controlled; read `e.target.files` in onChange

SUMMARY

- `value={state}` + `onChange` — the controlled component contract
- `e.target.checked` — for checkboxes; `e.target.value` for all others
- `select value={state}` — controls selected option; not `selected` on option
- File inputs are always uncontrolled — read via `e.target.files`

Uncontrolled Components

LETTING THE DOM MANAGE ITS OWN STATE

An **uncontrolled component** keeps its value in the DOM rather than in React state. You read the value imperatively via a `ref` when you need it — typically on form submit. React does not control the displayed value between interactions.

Uncontrolled inputs are simpler to wire up when you don't need real-time validation or formatting. They are also **required for file inputs** — `<input type="file">` is always uncontrolled because the browser controls which files are selected for security reasons. For most other cases, prefer controlled components (Sheet 21).

CONTROLLED VS UNCONTROLLED AT A GLANCE

- `defaultValue` — sets the initial value of an uncontrolled input; React does not update it after mount
- `defaultChecked` — initial checked state for uncontrolled checkboxes and radios
- `ref.current.value` — read the DOM value imperatively (e.g. on submit)
- `ref.current.files` — read selected files from a file input
- File inputs — always uncontrolled; cannot set `value` programmatically
- Third-party DOM libraries (rich text editors, date pickers) — often easier to integrate as uncontrolled

UNCONTROLLED INPUT PATTERNS

```
import { useRef } from 'react';

// ── Basic uncontrolled form (read on submit) ──────────────
function LoginForm() {
  const emailRef    = useRef(null);
  const passwordRef = useRef(null);

  function handleSubmit(e) {
    e.preventDefault();
    console.log(emailRef.current.value);      // read DOM value on submit
    console.log(passwordRef.current.value);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input type="email" defaultValue="user@example.com" ref={emailRef} />
      <input type="password" defaultValue="" ref={passwordRef} />
      <button type="submit">Login</button>
    </form>
  );
}

// ── File input (always uncontrolled) ──────────────────────
function FileUploader() {
  const fileRef = useRef(null);

  function handleUpload() {
    const file = fileRef.current.files[0];   // FileList — always via ref
    if (!file) return;
    console.log(file.name, file.size, file.type);
    const form = new FormData();
    form.append('file', file);
    fetch('/upload', { method: 'POST', body: form });
  }

  return (
    <>
      <input type="file" accept="image/*" ref={fileRef} />
      <button onClick={handleUpload}>Upload</button>
    </>
  );
}

// ── Reset an uncontrolled form ────────────────────────────
function handleReset(formRef) {
  formRef.current.reset();                   // native DOM reset
}
```

NOTES

- → `defaultValue` — sets the initial DOM value; unlike `value`, React won't override what the user types
- → `ref.current.value` — reads the live DOM value; only meaningful after mount (inside effects or event handlers)
- → `fileRef.current.files` — a `FileList` (not an array); use `Array.from(files)` to iterate
- → `accept="image/*"` — browser filter hint; validate the MIME type server-side too
- → `formRef.current.reset()` — resets all uncontrolled fields to their `defaultValue`; works on the native form element
- → Do not mix `value` (controlled) with a ref read — if you control the value, read it from state, not the DOM

CONTROLLED VS UNCONTROLLED PROPS

ELEMENT	CONTROLLED	UNCONTROLLED
`text input`	`value + onChange`	`defaultValue + ref`
`checkbox`	`checked + onChange`	`defaultChecked + ref`
`select`	`value + onChange`	`defaultValue + ref`
`textarea`	`value + onChange`	`defaultValue + ref`
`file input`	Not possible	`ref.current.files`

WHEN TO USE UNCONTROLLED

SITUATION	REASON
`File input`	Always — browser security prevents controlling
`Simple one-time read on submit`	No need for per-keystroke state overhead
`Integrating a non-React DOM library`	Library manages its own DOM; ref to read result
`Migrating legacy code`	Incremental adoption path
`Real-time validation needed`	Use controlled instead

COMMON MISTAKES

- Setting `value` (not `defaultValue`) on an uncontrolled input without `onChange` — React makes it read-only; the user cannot type
- Reading `ref.current.value` during render — the ref is null on first render; always read inside effects or event handlers
- Trying to programmatically set a file input's value — browsers block this for security; the user must select the file manually every time

TIPS

- Use `Array.from(fileRef.current.files)` to turn the `FileList` into a regular array for mapping or filtering
- For drag-and-drop file upload, listen to `onDrop` on a div and read `e.dataTransfer.files` — same FileList API, no ref needed
- Consider React Hook Form (Sheet 23) for large forms — it uses uncontrolled inputs internally via refs for performance, with a controlled-like API on top

SUMMARY

- `defaultValue / defaultChecked` — sets initial value; DOM manages after
- `ref.current.value` — read value on demand (submit, click)
- `type="file"` — always uncontrolled; read via `ref.current.files`
- Use controlled for real-time validation; uncontrolled for simple submit-only reads

Form Libraries — React Hook Form

WHY USE A FORM LIBRARY?

React Hook Form (RHF) manages form state via uncontrolled inputs under the hood — it attaches refs rather than wiring `onChange` to state — so large forms cause far fewer re-renders than the controlled approach. You register fields, validate on submit (or per field), and receive a `formState.errors` object describing any failures.

RHF integrates with UI libraries (Material UI, Radix, shadcn) via its `Controller` wrapper, which bridges RHF's ref-based system to controlled-only components. **Zod** or **Yup** schemas can be plugged in via `@hookform/resolvers` for schema-based validation.

REACT HOOK FORM EXAMPLES

```
import { useForm, Controller } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';

// ── Basic RHF form ──────────────────────────────────
function SignupForm() {
  const { register, handleSubmit, formState: { errors } } = useForm();

  const onSubmit = (data) => console.log(data); // { email: '…', password: '…' }

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input type="email" placeholder="Email"
        {...register('email', {
          required: 'Email is required',
          pattern: { value: /\S+@\S+\.\S+/, message: 'Invalid email' },
        })} />
      {errors.email && <p>{errors.email.message}</p>}

      <input type="password"
        {...register('password', {
          required: true,
          minLength: { value: 8, message: 'Min 8 chars' },
        })} />
      {errors.password && <p>{errors.password.message}</p>}

      <button type="submit">Sign Up</button>
    </form>
  );
}

// ── Zod schema validation ──────────────────────────
const schema = z.object({
  email:    z.string().email('Invalid email'),
  password: z.string().min(8, 'Min 8 chars'),
});

const { register, handleSubmit, formState: { errors } } =
  useForm({ resolver: zodResolver(schema) });

// ── Controller: for controlled UI-library inputs ──
<Controller
  name="role"
  control={control}
  rules={{ required: true }}
  render={({ field }) => (
    <Select {...field} options={roleOptions} />
      // field has value, onChange, ref
  )}
/>
```

KEY RHF APIS

- `useForm()` — initialises RHF; returns `register`, `handleSubmit`, `formState`, `watch`, `reset`, `setValue`
- `register('name', rules)` — connects a native input to RHF; spreads ref + event props onto the element
- `handleSubmit(fn)` — wraps your submit handler; validates first, calls `fn(data)` only if valid
- `formState.errors` — object keyed by field name; each value has `.message` and `.type`
- `watch('field')` — returns the current value of a field; re-renders on change
- `Controller` — wrapper for controlled UI-library inputs (Select, DatePicker)

NOTES

- `{...register('email', rules)}` — spreads `name`, `ref`, `onChange`, `onBlur` onto the input; no `value` prop — uncontrolled internally
- `handleSubmit(onSubmit)` — validates all fields first; calls `onSubmit(data)` only if there are no errors
- `formState.errors.email.message` — the message string from your rule; undefined if the field is valid
- `zodResolver(schema)` — replaces per-field rules with a single Zod schema; errors appear in `formState.errors` as usual
- `Controller` — needed when a component only accepts `value` / `onChange` (controlled); `field` contains both plus a `ref`
- `watch('email')` — subscribes to a field's value; useful for cross-field validation (confirm password) or conditional fields

BUILT-IN VALIDATION RULES

RULE	VALUE	MEANING
`required`	`true | 'msg'`	Field must not be empty
`minLength`	`{ value: 8, message: '…' }`	Minimum character count
`maxLength`	`{ value: 100, message: '…' }`	Maximum character count
`min / max`	`{ value: 0, message: '…' }`	Numeric range
`pattern`	`{ value: /regex/, message: '…' }`	Regex match
`validate`	`v => v === pwd || 'No match'`	Custom sync or async function

VALIDATION MODE OPTIONS

MODE	VALIDATES WHEN
`onSubmit (default)`	On form submit only
`onBlur`	When field loses focus
`onChange`	On every keystroke (most re-renders)
`onTouched`	First on blur, then on change
`all`	onBlur + onChange combined

COMMON MISTAKES

- Using `watch` for many fields without memoization — each watched field causes a re-render on every change; use `getValues()` inside handlers instead when you don't need live updates
- Forgetting `Controller` for library components — RHF can't attach its ref to a custom Select or DatePicker without it; the field stays unregistered
- Not calling `reset()` after a successful submit — the form retains its values and dirty/touched state from the previous submission

TIPS

- Use `mode: 'onBlur'` for a good UX balance — fields validate when the user leaves them, not on every keystroke
- Pair RHF with Zod for type-safe forms — `z.infer<typeof schema>` gives you the submit data type for free
- Use `defaultValues` in `useForm` to pre-populate edit forms — RHF uses them as the baseline for dirty-field detection

SUMMARY

- `{...register('field', rules)}` — registers a native input with RHF
- `handleSubmit(fn)` — validates then calls fn with data object
- `formState.errors.field.message` — error string for each invalid field
- `Controller` — bridges RHF to controlled UI-library components

Lifting State Up

shared state · single source of truth · prop drill

SHARING STATE BETWEEN COMPONENTS

When two sibling components need to share or react to the same piece of state, the solution is to **lift the state up** to their nearest common ancestor. The ancestor owns the state and passes it down as props, along with a callback prop that children call to request changes.

This preserves React's **single source of truth** principle: the state lives in exactly one place, every component that needs it receives the same value, and all updates go through the same setter. The trade-off is **prop drilling** — intermediate components must pass props they don't directly use. When drilling gets deep (3+ levels), consider Context (Sheet 12) or a state manager (Sheet 26–28).

LIFTING STATE RULES

- Find the **lowest common ancestor** — the nearest component that is a parent of all components that need the state
- Move `useState` to the ancestor — remove it from the children that previously owned it
- Pass the value **down as a prop** — children read but do not own
- Pass the **setter (or a callback) down as a prop** — children request changes by calling it
- Children become **controlled** — their output is fully determined by props
- If drilling reaches 3+ levels, lift to context instead of continuing to drill

LIFTING STATE EXAMPLE

```
// ── BEFORE: each panel owns its own open state ──────────
function PanelA() {
  const [open, setOpen] = useState(false);   // local — PanelB can't see this
  return <div onClick={() => setOpen(o => !o)}>{open ? 'A open' : 'A closed'}</div>;
}

// ── AFTER: parent owns open state; only one panel open ──
function Accordion() {
  const [openId, setOpenId] = useState(null);  // lifted up to common ancestor

  return (
    <>
      <Panel id="a" title="Section A" isOpen={openId === 'a'} onToggle={setOpenId} />
      <Panel id="b" title="Section B" isOpen={openId === 'b'} onToggle={setOpenId} />
    </>
  );
}

function Panel({ id, title, isOpen, onToggle }) {
  return (
    <div>
      <button onClick={() => onToggle(isOpen ? null : id)}>
        {title} {isOpen ? '▲' : '▼'}
      </button>
      {isOpen && <p>Content for {title}</p>}
    </div>
  );
}

// ── Temperature converter: two inputs, one value ────────
function TemperatureConverter() {
  const [celsius, setCelsius] = useState('');   // single source of truth

  const fahrenheit = celsius === '' ? '' : (celsius * 9/5 + 32).toFixed(1);

  return (
    <>
      <input value={celsius} onChange={e =>
            setCelsius(e.target.value)} />  °C
      <input value={fahrenheit} onChange={e =>
            setCelsius(((e.target.value - 32) * 5/9).toFixed(1))} /> °F
    </>
  );
}
```

NOTES

→ `openId === 'a'` — the parent derives each panel's open state from a single value; no panel can be independently open

→ `onToggle={setOpenId}` — the setter is passed down as a callback prop; the child calls it but doesn't own the state

→ `isOpen ? null : id` — toggle logic: if already open, close (null); if closed, open (this id)

→ `celsius` only stored — `fahrenheit` is always derived from it; no sync needed, no risk of the two getting out of step

→ Derived values belong in render, not state — if you can compute B from A, store only A and compute B on render

→ Lifting causes parent re-renders — every child of the ancestor re-renders too; wrap expensive children in `React.memo` if needed

BEFORE AND AFTER LIFTING

	BEFORE	AFTER LIFTING
State lives in	Child A	Parent (common ancestor)
Child B can read	✖	✔ (via prop)
Sibling sync	✖	✔
Single source of truth	✖	✔
Parent re-renders	✖	✔ on change

SHARING STATE: WHICH SOLUTION?

DEPTH	SOLUTION
1–2 levels	Lift state + pass props
3–5 levels	Context (Sheet 12 / 25)
Many components app-wide	Redux Toolkit / Zustand (Sheets 26–28)
Server-synced data	TanStack Query / SWR (Sheets 33–34)

COMMON MISTAKES

- Duplicating state across siblings — two components each holding a copy of the same value will inevitably get out of sync; lift to a single owner
- Storing derived data in state — if `fahrenheit` can be computed from `celsius`, putting both in state means every update requires two `setState` calls
- Over-lifting to a global store prematurely — lift only as far as necessary; global state managers add complexity and should be a last resort

TIPS

- Look for the pattern: "two components need to agree on the same value" — that's always the signal to lift
- Compute derived values during render instead of storing them in state — simpler, always in sync, and no extra `useEffect` needed
- Name callback props `onNoun` or `onVerb` (`onToggle`, `onSelect`) to signal they are events flowing upward, not downward data

SUMMARY

- `Lift state` — move to nearest common ancestor of all consumers
- Pass value down as prop; pass setter as callback prop
- Derived values — compute from state during render; don't duplicate in state
- Deep drilling → Context or state manager instead of more lifting

Context API — Patterns

global state · context + useReducer · provider tre

CONTEXT AS A LIGHTWEIGHT GLOBAL STORE

Pairing `useReducer` with Context gives you a Redux-like global store without any external library. The reducer owns all state transitions; the Context distributes state and dispatch to any component in the tree. This pattern works well for small-to-medium apps with clearly defined actions.

The key performance technique is **splitting state and dispatch into two separate contexts**. The dispatch function is always stable (never changes reference), so components that only dispatch — like buttons — never re-render when state changes. Components that read state re-render only when the part of state they use changes.

CONTEXT + REDUCER PATTERN

- Create **two contexts**: `StateContext` (changes often) and `DispatchContext` (always stable)
- Wrap both Providers around the app; inner Provider first
- Export custom hooks: `useAppState()` and `useAppDispatch()`
- Reducer stays pure and testable — no React knowledge needed in the reducer file
- Memoize the state value object with `useMemo` if it is an object literal to prevent consumer re-renders
- Scale limit: if 10+ components dispatch/read from a single context, consider Zustand or Redux Toolkit

CONTEXT + USEREDUCER GLOBAL STORE

```jsx
// store/AppContext.jsx
import { createContext, useContext, useReducer } from 'react';

const initialState = { user: null, theme: 'light', cart: [] };

function reducer(state, action) {
  switch (action.type) {
    case 'SET_USER':  return { ...state, user: action.payload };
    case 'TOGGLE_THEME': return { ...state, theme: state.theme === 'light' ? 'dark' : 'light' };
    case 'ADD_TO_CART': return { ...state, cart: [...state.cart, action.payload] };
    default: throw new Error(`Unknown action: ${action.type}`);
  }
}

// Two contexts: state (re-renders on change) and dispatch (always stable)
const StateCtx    = createContext(null);
const DispatchCtx = createContext(null);

export function AppProvider({ children }) {
  const [state, dispatch] = useReducer(reducer, initialState);
  return (
    <DispatchCtx.Provider value={dispatch}>
      <StateCtx.Provider value={state}>
        {children}
      </StateCtx.Provider>
    </DispatchCtx.Provider>
  );
}

export const useAppState    = () => useContext(StateCtx);
export const useAppDispatch = () => useContext(DispatchCtx);

// Consuming in a component
function CartButton() {
  const dispatch = useAppDispatch();  // never re-renders from state changes
  return <button onClick={() => dispatch({ type: 'ADD_TO_CART', payload: item })}>Add</button>;
}

function Header() {
  const { user, theme } = useAppState();  // re-renders when state changes
  return <header className={theme}>{user?.name}</header>;
}
```

NOTES

- → `DispatchCtx` wraps outer — dispatch is stable, so its Provider never triggers re-renders; the inner `StateCtx` does
- → `useAppDispatch()` — a component that only dispatches reads from the stable context; it never re-renders when state changes
- → `useAppState()` — reads the whole state object; any state change re-renders all consumers, not just those using the changed field
- → Selector limitation — Context has no built-in selector mechanism; a component reading `state.user` re-renders when `state.cart` changes too
- → `throw new Error` in default — catches action type typos in development immediately
- → For fine-grained subscriptions (skip re-render when unrelated state changes), Zustand (Sheet 28) is a better fit

SPLIT CONTEXT STRATEGY

CONTEXT	VALUE	RE-RENDERS CONSUMERS WHEN
`StateContext`	state object	Any dispatch changes state
`DispatchContext`	dispatch fn	Never (dispatch is stable)

CONTEXT VS REDUX/ZUSTAND

FEATURE	CONTEXT+REDUCER	REDUX/ZUSTAND
`Setup`	Zero dependencies	npm install needed
`DevTools`	✖	✔
`Selective subscription`	✖ (whole context)	✔ (selector)
`Async actions`	Manual (useEffect)	Built-in middleware
`Best for`	Small–medium apps	Large apps, frequent updates

COMMON MISTAKES

- One context for both state and dispatch — components that only dispatch still re-render on every state change; split them
- Storing all app state in a single large context — every consumer re-renders on every field change; split into domain-specific contexts
- Putting an inline object literal as the context value without `useMemo` — creates a new reference every render and re-renders all consumers unnecessarily

TIPS

- Keep the reducer in its own file — it's pure JS, easy to unit test without any React setup
- If you're wrapping your app in 5+ Providers, introduce a single `<AppProviders>` composition component to keep `main.jsx` readable
- Use this pattern as a stepping stone — if the app grows and you need DevTools, time-travel debugging, or selectors, migrate to Redux Toolkit or Zustand with minimal restructuring

SUMMARY

- `StateCtx + DispatchCtx` — split for performance; dispatch never re-renders
- `useReducer` in the Provider — pure transitions, easy to test
- Context has no selectors — all consumers re-render on any state change
- Scale limit → Zustand or Redux Toolkit for large apps

Redux Toolkit — Setup

REDUX TOOLKIT (RTK)

Redux Toolkit is the official, opinionated way to write Redux. It eliminates the boilerplate of vanilla Redux: `createSlice` generates action creators and a reducer from a single object, and `configureStore` sets up Redux DevTools, Immer (for safe "mutations" in reducers), and thunk middleware automatically.

RTK uses **Immer** internally, so reducers can appear to mutate state directly — Immer intercepts the mutations and produces a new immutable state object. You can also return a new state explicitly if preferred. Components read state with `useSelector` and dispatch actions with `useDispatch`.

REDUX TOOLKIT SETUP

```
// npm install @reduxjs/toolkit react-redux

// ── store/counterSlice.js ──────────────────────────────
import { createSlice } from '@reduxjs/toolkit';

const counterSlice = createSlice({
  name: 'counter',
  initialState: { value: 0, step: 1 },
  reducers: {
    increment: (state) => { state.value += state.step; },  // Immer "mutation" — safe
    decrement: (state) => { state.value -= state.step; },
    reset:     (state) => { state.value = 0; },
    setStep:   (state, action) => { state.step = action.payload; },
  },
});

export const { increment, decrement, reset, setStep } = counterSlice.actions;
export default counterSlice.reducer;

// ── store/index.js ─────────────────────────────────────
import { configureStore } from '@reduxjs/toolkit';
import counterReducer from './counterSlice';

export const store = configureStore({
  reducer: { counter: counterReducer },   // add more slices here
});

// ── main.jsx ───────────────────────────────────────────
import { Provider } from 'react-redux';
import { store }    from './store';

createRoot(document.getElementById('root')).render(
  <Provider store={store}><App /></Provider>
);

// ── Counter component ──────────────────────────────────
import { useSelector, useDispatch } from 'react-redux';
import { increment, decrement, reset } from './store/counterSlice';

function Counter() {
  const value    = useSelector(state => state.counter.value);
  const dispatch = useDispatch();
  return (
    <>
      <p>{value}</p>
      <button onClick={() => dispatch(increment())}>+</button>
      <button onClick={() => dispatch(decrement())}>-</button>
      <button onClick={() => dispatch(reset())}>Reset</button>
    </>
  );
}
```

RTK KEY APIS

- `configureStore({ reducer })` — creates the store; auto-enables DevTools and Thunk
- `createSlice({ name, initialState, reducers })` — generates reducer + action creators in one call
- `slice.actions` — exported action creators; dispatch them to trigger reducer cases
- `slice.reducer` — the generated reducer function; pass to `configureStore`
- `useSelector(state => state.slice.field)` — reads from the store; re-renders on change
- `useDispatch()` — returns the store's dispatch function; stable reference

NOTES

- → `state.value += state.step` — looks like mutation but Immer intercepts it and produces a new immutable state; no spread needed inside reducers
- → `action.payload` — RTK automatically puts the first argument of a dispatched action creator into `payload`
- → `configureStore` — automatically sets up Redux DevTools Extension (install the browser extension to use it)
- → `<Provider store={store}>` — makes the store available to all `useSelector` and `useDispatch` calls in the tree
- → `useSelector(state => state.counter.value)` — selects a specific field; component re-renders only when that field changes (strict equality check)
- → `dispatch(increment())` — `increment()` is the action creator (returns `{ type: 'counter/increment' }`); dispatch sends it to the reducer

RTK FILE STRUCTURE

FILE	PURPOSE
`store/index.js`	`configureStore` — combines all slices
`store/counterSlice.js`	`createSlice` — one slice per feature
`main.jsx`	`<Provider store={store}>` wraps the app
`Component.jsx`	`useSelector` + `useDispatch`

USESELECTOR TIPS

PATTERN	EFFECT
`state => state.cart.items`	Re-renders only when items array changes
`state => state.cart`	Re-renders when any cart field changes
`state => state`	Re-renders on any store change — avoid
`createSelector (reselect)`	Memoised derived selector — skips re-render

COMMON MISTAKES

- Mutating state outside a slice reducer — Immer only wraps reducer functions; mutating state in a component or selector causes bugs
- Selecting the whole slice (`state => state.cart`) when you only need one field — the component re-renders on any cart change; select the specific field
- Forgetting `<Provider>` — `useSelector` and `useDispatch` throw immediately without it; wrap the app root

TIPS

- One slice per feature — `cartSlice` , `authSlice` , `uiSlice` — combine them all in `configureStore`
- Export selector functions from the slice file (`export const selectCount = state => state.counter.value`) so component imports stay clean
- For async data fetching within Redux, use `createAsyncThunk` and RTK Query (Sheet 27) rather than dispatching inside `useEffect`

SUMMARY

- `createSlice` — generates reducer + action creators from one object
- `configureStore` — combines slices; enables DevTools + Immer + Thunk
- `useSelector(fn)` — reads store state; re-renders on field change
- `useDispatch()` — returns stable dispatch; call with action creators

Redux Toolkit — Async

createAsyncThunk · extraReducers · pending/fulfill

ASYNC ACTIONS IN REDUX TOOLKIT

`createAsyncThunk` generates a thunk that dispatches three lifecycle actions automatically: `pending` when the async operation starts, `fulfilled` when it resolves, and `rejected` when it throws. You handle these in `extraReducers` using the `builder` API inside your slice.

RTK Query (included in RTK) is a higher-level data-fetching and caching layer. You define an API with endpoints, and RTK Query generates hooks (`useGetPostsQuery`, `useAddPostMutation`) that handle caching, invalidation, loading and error states automatically — without writing thunks or reducers.

ASYNC THUNK & RTK QUERY

```
// ── createAsyncThunk ──────────────────────────────────
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';

export const fetchPosts = createAsyncThunk(
  'posts/fetchAll',                          // action type prefix
  async (_, { rejectWithValue }) => {
    const res = await fetch('/api/posts');
    if (!res.ok) return rejectWithValue(res.status);
    return res.json();                       // becomes action.payload on fulfilled
  }
);

const postsSlice = createSlice({
  name: 'posts',
  initialState: { items: [], status: 'idle', error: null },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchPosts.pending,   (state) => { state.status = 'loading'; })
      .addCase(fetchPosts.fulfilled, (state, action) => {
        state.status = 'succeeded';
        state.items  = action.payload;
      })
      .addCase(fetchPosts.rejected,  (state, action) => {
        state.status = 'failed';
        state.error  = action.payload ?? action.error.message;
      });
  },
});

// Component usage:
const dispatch = useDispatch();
useEffect(() => { dispatch(fetchPosts()); }, [dispatch]);
const { items, status } = useSelector(s => s.posts);

// ── RTK Query (minimal setup) ─────────────────────────
import { createApi, fetchBaseQuery } from '@reduxjs/toolkit/query/react';

export const postsApi = createApi({
  reducerPath: 'postsApi',
  baseQuery: fetchBaseQuery({ baseUrl: '/api' }),
  endpoints: (build) => ({
    getPosts: build.query({ query: () => '/posts' }),
    addPost:  build.mutation({ query: (body) => ({ url: '/posts', method: 'POST', body }) }),
  }),
});

export const { useGetPostsQuery, useAddPostMutation } = postsApi;

// Component: auto loading + caching
function Posts() {
  const { data, isLoading, error } = useGetPostsQuery();
  if (isLoading) return <p>Loading…</p>;
  if (error)     return <p>Error</p>;
  return <ul>{data.map(p => <li key={p.id}>{p.title}</li>)}</ul>;
}
```

CREATEASYNCTHUNK LIFECYCLE

- `pending` — dispatched immediately when thunk is called
- `fulfilled` — dispatched with the resolved value as `action.payload`
- `rejected` — dispatched if the thunk throws; `action.error.message`
- `rejectWithValue(val)` — dispatch rejected with a custom payload instead of the error
- `builder.addCase(thunk.pending, reducer)` — handle each lifecycle in `extraReducers`
- `builder.addMatcher` — handle multiple thunks with one reducer branch

NOTES

- `'posts/fetchAll'` — the type prefix; RTK appends `/pending`, `/fulfilled`, `/rejected` automatically
- `rejectWithValue(res.status)` — puts a custom value in `action.payload` on rejection instead of the raw Error object
- `extraReducers builder` — handles actions generated outside this slice; `addCase` takes the thunk action creator directly
- `state.status = 'loading'` — Immer allows direct mutation inside RTK reducers; no need to spread
- `createApi` — RTK Query; add its reducer and middleware to `configureStore`: `reducer: { [postsApi.reducerPath]: postsApi.reducer }`
- `useGetPostsQuery()` — auto-fetches on mount, caches by endpoint + arg, deduplicates concurrent calls, re-fetches on window focus

CREATEASYNCTHUNK VS RTK QUERY

FEATURE	CREATEASYNCTHUNK	RTK QUERY
Caching	✖ manual	✔ automatic
Loading state	Manual in reducer	Auto via hook
Re-fetch on focus	✖	✔
Invalidation	✖ manual	✔ tags
Custom logic	✔ full control	✖ endpoint-based
Best for	Complex async flows	REST / GraphQL CRUD

RTK QUERY HOOK TYPES

HOOK	RETURNS
`useGetXQuery(arg)`	{ data, isLoading, error, refetch }
`useAddXMutation()`	[triggerFn, { isLoading, error }]
`useUpdateXMutation()`	[triggerFn, { isLoading, error }]
`useDeleteXMutation()`	[triggerFn, { isLoading }]

COMMON MISTAKES

- Throwing inside a thunk without `rejectWithValue` — the error object is serialised differently; use `rejectWithValue` for predictable payload shape
- Forgetting to add RTK Query's reducer and middleware to `configureStore` — hooks throw and caching doesn't work
- Dispatching a thunk inside render — triggers on every render; always dispatch inside `useEffect` or event handlers

TIPS

- Prefer RTK Query over hand-written thunks for standard CRUD — it gives you caching, deduplication, and invalidation for free
- Use `status: 'idle' | 'loading' | 'succeeded' | 'failed'` as a string enum in your slice state — clearer than separate boolean flags
- RTK Query tag invalidation — define `providesTags` on queries and `invalidatesTags` on mutations to auto-refetch after writes

SUMMARY

- `createAsyncThunk` — generates pending/fulfilled/rejected actions automatically
- `extraReducers builder` — handles async lifecycle in the slice
- `rejectWithValue` — custom rejected payload instead of raw Error
- `RTK Query` — auto caching + hooks; prefer for standard API fetching

Zustand

create · set · get · devtools middleware · persist

MINIMAL GLOBAL STATE WITH ZUSTAND

Zustand is a small (~1 kB) state management library with no boilerplate and no Provider required. You define a store with `create`, co-locating state and actions in one object. Components subscribe to a slice of that store via a **selector function** — they only re-render when the selected value changes, giving fine-grained subscriptions without any extra work.

Unlike Redux, Zustand stores live outside the React tree and can be read from anywhere — including non-component code (utility functions, event handlers). Middleware like `devtools` and `persist` are composable and easy to add.

ZUSTAND KEY CONCEPTS

- `create((set, get) => ({…}))` — defines store; returns a hook
- `set(patch)` — merges patch into state (shallow); triggers subscribed components
- `set(fn)` — functional form: `set(state => ({ count: state.count + 1 }))`
- `get()` — reads current state inside an action without subscribing
- `useStore(selector)` — component subscribes only to the selected slice
- No Provider — store is a module-level singleton; import the hook anywhere

ZUSTAND STORE PATTERNS

```
// npm install zustand
import { create } from 'zustand';
import { devtools, persist } from 'zustand/middleware';

// ── Basic store ──────────────────────────────────────────
const useCounterStore = create((set) => ({
  count: 0,
  increment: ()  => set((s) => ({ count: s.count + 1 })),
  decrement: ()  => set((s) => ({ count: s.count - 1 })),
  reset:     ()  => set({ count: 0 }),
  setCount:  (n) => set({ count: n }),
}));

// ── Component usage — selector = fine-grained subscription ─
function Counter() {
  const count     = useCounterStore((s) => s.count);       // re-renders on count change only
  const increment = useCounterStore((s) => s.increment);   // stable — action never changes
  return <button onClick={increment}>{count}</button>;
}

// ── Store with async action ──────────────────────────────
const usePostsStore = create((set) => ({
  posts: [],
  loading: false,
  fetchPosts: async () => {
    set({ loading: true });
    const data = await fetch('/api/posts').then(r => r.json());
    set({ posts: data, loading: false });
  },
}));

// ── persist middleware (saves to localStorage) ──────────
const useThemeStore = create(
  persist(
    (set) => ({
      theme: 'light',
      toggle: () => set((s) => ({ theme: s.theme === 'light' ? 'dark' : 'light' })),
    }),
    { name: 'theme-storage' }          // localStorage key
  )
);
```

NOTES

- `set((s) => ({ count: s.count + 1 }))` — functional form reads current state safely; `set({ count: 0 })` merges a patch directly
- `useCounterStore((s) => s.count)` — selector; component re-renders only when `count` changes; comparing by strict equality
- `useCounterStore((s) => s.increment)` — actions are stable (same reference across renders) so selecting them doesn't cause re-renders
- Async actions — plain async functions inside the store; call `set` before and after the async operation; no thunk middleware needed
- `persist(store, { name })` — wraps the store creator; auto-hydrates from localStorage on load, auto-saves on every `set`
- Read store outside React — `useCounterStore.getState().count`; call actions with `useCounterStore.getState().increment()`

ZUSTAND VS REDUX TOOLKIT

FEATURE	ZUSTAND	REDUX TOOLKIT
Bundle size	~1 kB	~16 kB
Provider needed	✖	✔
Boilerplate	Minimal	Slice + store + Provider
DevTools	Via middleware	Built-in
Async actions	Regular async functions	createAsyncThunk
Selectors	Inline in hook call	useSelector

ZUSTAND MIDDLEWARE

MIDDLEWARE	PURPOSE
devtools	Redux DevTools integration
persist	Auto-save to localStorage / sessionStorage
immer	Mutate state directly in set callbacks
subscribeWithSelector	Subscribe to store outside React (vanilla JS)

COMMON MISTAKES

- Selecting the entire store object (`useStore(s => s)`) — every state change re-renders the component; always select the minimum needed slice
- Calling `set` with stale state without the functional form — `set({ count: count + 1 })` reads the closure value; use `set(s => …)` for correctness
- Storing non-serialisable values (class instances, functions) with `persist` — `JSON.stringify` drops them silently; use `partialize` to exclude them

TIPS

- Split large stores into slices by combining multiple `create` calls or using the slice pattern: each feature exports a partial store that is merged in a root store
- Add the `devtools` middleware in development only — wrap with `process.env.NODE_ENV === 'development' ? devtools(store) : store`
- Zustand works without React — great for sharing state with non-React parts of an app (vanilla event handlers, third-party libraries)

SUMMARY

- `create((set, get) => ({…}))` — define state + actions in one object
- `useStore(s => s.field)` — selector; re-renders only when field changes
- No Provider — store is a module singleton; import hook anywhere
- `persist` middleware — auto localStorage sync with one wrapper

React Router — Setup & Routes

CLIENT-SIDE ROUTING WITH REACT ROUTER V6

React Router v6 is the standard routing library for React SPAs. It maps URL paths to components, enabling navigation without full-page reloads. The router reads the browser's URL, matches it against a <Routes> tree, and renders the matching <Route element> component.

Wrap the app in <BrowserRouter> (uses the History API) or <HashRouter> (uses the URL hash — no server config needed). Declare routes with <Routes> + <Route>; React Router picks the best match automatically. An `index` route renders at the parent path with no additional segment.

ROUTER TYPES

- `BrowserRouter` — uses `/path` URLs; requires server to serve `index.html` for all paths
- `HashRouter` — uses `/#/path`; works with any static host; not ideal for SEO
- `MemoryRouter` — in-memory; for tests and React Native
- `createBrowserRouter` — data router API (v6.4+); enables loaders, actions, and error boundaries per route
- `path="*"` — wildcard; matches anything not caught above — use for 404 pages
- `index` route — renders at the parent's exact path; no extra segment

ROUTER SETUP

```
// npm install react-router-dom

// ── main.jsx ──────────────────────────────────────────
import { BrowserRouter } from 'react-router-dom';

createRoot(document.getElementById('root')).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);

// ── App.jsx — route declarations ──────────────────────
import { Routes, Route } from 'react-router-dom';
import Layout from './Layout';
import Home from './pages/Home';
import About from './pages/About';
import UserDetail from './pages/UserDetail';
import NotFound from './pages/NotFound';

function App() {
  return (
    <Routes>
      <Route path="/" element={<Layout />}>        {/* layout route */}
        <Route index element={<Home />} />          {/* / — index route */}
        <Route path="about" element={<About />} />  {/* /about */}
        <Route path="users/:id" element={<UserDetail />} />  {/* /users/42 */}
        <Route path="*" element={<NotFound />} />   {/* 404 catch-all */}
      </Route>
    </Routes>
  );
}

// ── Layout.jsx — renders matched child via Outlet ─────
import { Outlet } from 'react-router-dom';

function Layout() {
  return (
    <div>
      <nav>…</nav>
      <main>
        <Outlet />   {/* matched child route renders here */}
      </main>
    </div>
  );
}
```

NOTES

- → `<BrowserRouter>` — wrap at the app root; provides routing context to all descendants
- → `<Route path="/" element={<Layout />}>` — layout route; its children are nested routes rendered via `<Outlet>`
- → `index` — renders at the parent's exact path; here `/` renders `<Home>`
- → `path="users/:id"` — relative to parent; resolves to `/users/:id`; no leading slash on children
- → `path="*"` — matches any URL not matched by siblings; always put it last
- → `<Outlet />` — placeholder in the layout where the matched child route component renders

ROUTE PROPS

PROP	TYPE	PURPOSE
`path`	`string`	URL pattern to match
`element`	`JSX`	Component to render on match
`index`	`boolean`	Default child at parent path
`errorElement`	`JSX`	Render on loader/action error
`loader`	`async fn`	Data router: fetch before render

PATH PATTERN SYNTAX

PATTERN	MATCHES
`/users`	Exact path /users
`/users/:id`	/users/42, /users/abc — id is a param
`/files/*`	/files/a/b/c — splat param
`*`	Anything — use last for 404
`(index)`	Parent path exactly (no extra segment)

COMMON MISTAKES

- Using absolute paths on nested routes (`path="/about"` inside a parent) — causes double-matching issues; child routes should use relative paths without a leading slash
- Forgetting `<Outlet />` in a layout route — nested child routes never render; the layout shows but the child content is missing
- Not configuring the server to return `index.html` for all paths with `BrowserRouter` — direct URL access or refresh returns a 404 from the server

TIPS

- Use layout routes to share a persistent shell (nav, sidebar) across multiple pages without re-mounting on navigation
- Use `createBrowserRouter` + `RouterProvider` (the data router API) for new projects — it enables loaders, actions, and per-route error boundaries
- Always include a `path="*"` catch-all at the end of your top-level routes — a blank screen is worse than a friendly 404

SUMMARY

- `<BrowserRouter>` — wraps app; provides routing context
- `<Routes><Route path element /></Routes>` — declares URL-to-component mapping
- `index` — default child route at parent path
- `<Outlet />` — where nested child routes render inside a layout

React Router — Navigation & Params

NAVIGATING AND READING URL DATA

React Router provides declarative navigation (`<Link>`, `<NavLink>`) and imperative navigation (`useNavigate`) for all navigation needs. URL data flows into components via `useParams` (path segments) and `useSearchParams` (query strings).

`<Link>` renders an `<a>` tag but intercepts the click to perform a client-side navigation instead of a full page reload. `<NavLink>` adds an `active` class (or a class you specify) when its `to` path matches the current URL — ideal for navigation menus. `useNavigate` returns a function for programmatic navigation after form submissions, auth checks, or timer callbacks.

NAVIGATION HOOK APIS

- `useNavigate()` — returns `navigate(path, options)`; options: `{ replace, state, relative }`
- `navigate(-1)` — go back one step in history; `navigate(1)` goes forward
- `navigate('/path', { replace: true })` — replaces history entry (no back button)
- `useParams()` — returns object of URL params from `:paramName` segments
- `useSearchParams()` — returns `[searchParams, setSearchParams]`; like `useState` for query strings
- `useLocation()` — returns current location object: `{ pathname, search, hash, state }`

NAVIGATION & PARAMS EXAMPLES

```
import { Link, NavLink, useNavigate, useParams, useSearchParams } from 'react-router-dom';

// ── Link and NavLink ──────────────────────────────
<Link to="/about">About</Link>

<NavLink
  to="/dashboard"
  className={({ isActive }) => isActive ? 'nav-link active' : 'nav-link'}
>
  Dashboard
</NavLink>

// ── useNavigate: after form submit ────────────────
function LoginForm() {
  const navigate = useNavigate();

  async function handleSubmit(e) {
    e.preventDefault();
    await login(email, password);
    navigate('/dashboard', { replace: true });  // can't go back to login
  }
  return <form onSubmit={handleSubmit}>…</form>;
}

// ── useParams: read :id from URL ──────────────────
// Route: <Route path="users/:id" element={<UserDetail />} />
function UserDetail() {
  const { id } = useParams();           // /users/42 → { id: '42' }
  const { data } = useFetch(`/api/users/${id}`);
  return <p>{data?.name}</p>;
}

// ── useSearchParams: read and update query string ─
function SearchPage() {
  const [searchParams, setSearchParams] = useSearchParams();
  const query = searchParams.get('q') ?? '';

  return (
    <input
      value={query}
      onChange={e => setSearchParams({ q: e.target.value })}
      placeholder="Search…"
    />
  );
  // URL updates to ?q=react as user types
}
```

NOTES

- `<Link to="/about">` — renders an `<a>`; uses `history.pushState` to navigate without page reload
- `({ isActive }) => …` — NavLink passes an object to `className` and `style` function props; use it to apply active styles
- `replace: true` — replaces the current history entry; the user cannot press Back to return to the previous page
- `useParams()` — all param values are strings; parse numbers with `Number(id)` before using as an API argument
- `setSearchParams({ q: val })` — sets the whole query string; use functional form `prev => …` to merge instead of replace
- `navigate(-1)` — equivalent to `window.history.back()`; works within the app's history stack

LINK VS NAVLINK VS NAVIGATE()

API	USE WHEN
`<Link to="/path">`	Standard navigation in JSX
`<NavLink to="/path">`	Nav menu — needs active styling
`navigate('/path')`	Programmatic — after form submit, auth check
`navigate(-1)`	Go back (like browser back button)
`navigate(path, { replace })`	Redirect without adding to history

USESEARCHPARAMS METHODS

METHOD	EXAMPLE
`searchParams.get('key')`	`?q=react → 'react'`
`searchParams.getAll('tag')`	`?tag=a&tag=b → ['a','b']`
`searchParams.has('key')`	`boolean`
`setSearchParams({ q: 'x' })`	`Replaces entire query string`
`setSearchParams(prev => …)`	`Functional update of params`

COMMON MISTAKES

- Using `<a href="/path">` instead of `<Link>` — causes a full page reload and discards all React state
- Treating URL params as numbers without parsing — `useParams()` returns strings; `/users/42` gives `id = "42"`, not `42`
- Calling `navigate` during render (outside a handler or effect) — causes an infinite loop; always call inside event handlers or `useEffect`

TIPS

- Use `useSearchParams` to keep UI state (filters, sort, page) in the URL — it makes the current view shareable and back/forward navigable
- Pass `state` in `navigate` to send data between routes without query strings: `navigate('/confirm', { state: { orderId } })`; read with `useLocation().state`
- Wrap `<NavLink>` className in a function rather than a string — it receives `{ isActive, isPending }` to handle both normal and transition states

SUMMARY

- `<Link to>` — declarative navigation; no page reload
- `<NavLink>` — Link + active class for nav menus
- `useParams()` — reads `:param` segments as strings
- `useSearchParams()` — read/write query string like useState

React Router — Advanced

Outlet · nested routes · loader · action · errorEl

DATA ROUTER & LAZY ROUTES

React Router v6.4+ introduced the **data router API** (`createBrowserRouter` + `RouterProvider`) which adds loader and action functions directly to route definitions. A loader fetches data before the route renders — no more useEffect data fetching in page components. An action handles form mutations server-side style.

React.lazy + React.Suspense enable **route-level code splitting**: each page component is a separate JS chunk loaded only when first visited. This keeps the initial bundle small. The errorElement prop replaces a whole route subtree with an error UI when a loader, action, or render throws.

DATA ROUTER HOOKS

- `useLoaderData()` — reads the value returned by the route's `loader` function
- `useActionData()` — reads the value returned by the route's `action` function after a form submit
- `useNavigation()` — `{ state: 'idle' | 'loading' | 'submitting' }` — global navigation state
- `useRouteError()` — reads the thrown error inside an `errorElement`
- `<Form method="post">` — RR form; submits to the route's `action` function instead of the server
- `defer()` + `<Await>` — stream slow data while fast data renders immediately

NOTES

- → `createBrowserRouter(routes)` — replaces `<BrowserRouter>`; takes an array of route objects instead of JSX
- → `loader({ params, request })` — receives route params and the full Request; throw a Response to trigger errorElement
- → `useLoaderData()` — reads whatever the loader returned; TypeScript-infer the type with a generic
- → `<Form method="delete">` — React Router intercepts the submit and calls the route's `action` function; no `e.preventDefault()` needed
- → `lazy(() => import('./Page'))` — the chunk is fetched only when the route is first visited; pair with `<Suspense>` for the fallback
- → `errorElement` — replaces the route's element when its loader, action, or render throws; access the error with `useRouteError()`

LOADER VS USEEFFECT FETCH

FEATURE	LOADER	USEEFFECT FETCH
When it runs	Before component renders	After component renders
Loading flash	✖ (data ready on mount)	✔ (empty → populated)
Error boundary	errorElement on route	Manual try/catch + state
Parallel loads	✔ automatic	✖ manual Promise.all
Works without JS	✔ (with SSR)	✖

ROUTE CODE SPLITTING

APPROACH	API
Lazy import	`React.lazy(() => import('./Page'))`
Suspense fallback	`<Suspense fallback={<Spinner />}>`
RR lazy prop	`lazy: () => import('./Page')`
Preload on hover	Call import() on mouseenter of Link

DATA ROUTER & LAZY ROUTES

```
import { createBrowserRouter, RouterProvider, Outlet,
         useLoaderData, useRouteError, Form } from 'react-router-dom';
import { lazy, Suspense } from 'react';

// ── Lazy page components ──────────────────────────────
const Dashboard = lazy(() => import('./pages/Dashboard'));
const UserDetail = lazy(() => import('./pages/UserDetail'));

// ── Loader function (runs before render) ──────────────
async function userLoader({ params }) {
  const res = await fetch(`/api/users/${params.id}`);
  if (!res.ok) throw new Response('Not Found', { status: 404 });
  return res.json();                    // available via useLoaderData()
}

// ── Action function (handles Form submit) ─────────────
async function deleteAction({ params }) {
  await fetch(`/api/users/${params.id}`, { method: 'DELETE' });
  return { success: true };             // available via useActionData()
}

// ── Router definition ─────────────────────────────────
const router = createBrowserRouter([
  {
    path: '/',
    element: <Layout />,
    errorElement: <ErrorPage />,
    children: [
      { index: true, element: <Suspense fallback={<p>…</p>}><Dashboard /></Suspense> },
      {
        path: 'users/:id',
        loader: userLoader,
        action: deleteAction,
        errorElement: <ErrorPage />,
        element: <Suspense fallback={<p>…</p>}><UserDetail /></Suspense>,
      },
    ],
  },
]);

// ── Page component reads loader data ──────────────────
function UserDetail() {
  const user = useLoaderData();         // no useEffect needed
  return (
    <>
      <h1>{user.name}</h1>
      <Form method="delete">
        <button type="submit">Delete</button>
      </Form>
    </>
  );
}

// ── Error boundary component ──────────────────────────
function ErrorPage() {
  const error = useRouteError();
  return <p>Error: {error.statusText ?? error.message}</p>;
}

// ── App entry ─────────────────────────────────────────
<RouterProvider router={router} />
```

COMMON MISTAKES

- Mixing `<BrowserRouter>` with data-router hooks (`useLoaderData`) — data hooks only work inside a `createBrowserRouter` tree; they throw otherwise
- Forgetting `<Suspense>` around lazy routes — React throws an error; always wrap lazy components in a Suspense boundary with a fallback
- Returning non-serialisable values from a loader — loaders that use `defer()` must return serialisable data; class instances and functions are silently dropped

TIPS

- Move all data fetching into loaders — page components become pure display components with zero loading state management
- Place `errorElement` on the root route as a catch-all, then add specific ones on routes with loaders that might 404
- Use `useNavigation().state === 'loading'` in the layout to show a global progress bar during route transitions

SUMMARY

- `loader` — fetches data before the route renders; no useEffect needed
- `useLoaderData()` — reads loader return value in the component
- `errorElement` — error boundary per route; read error with `useRouteError()`
- `React.lazy + Suspense` — split each route into its own JS chunk

Fetching Data with useEffect

THE MANUAL DATA FETCHING PATTERN

Fetching data inside `useEffect` is the baseline pattern every React developer should know, even if libraries like TanStack Query or SWR (Sheets 33–34) are preferred for production. The pattern requires three pieces of state — **data, loading, and error** — and a cleanup function using `AbortController` to cancel the in-flight request when the component unmounts or the dependency changes.

The critical rule: `useEffect` cannot be `async` directly. Declare the async function inside the effect and call it immediately. Use an `AbortController` signal to prevent "state update on unmounted component" errors when navigation happens before a slow request completes.

COMPLETE FETCH PATTERN

```
import { useState, useEffect } from 'react';

function useFetch(url) {
  const [data,    setData]    = useState(null);
  const [loading, setLoading] = useState(true);
  const [error,   setError]   = useState(null);

  useEffect(() => {
    // ❶ Reset state for new URL
    setLoading(true);
    setError(null);

    // ❷ AbortController for cleanup
    const controller = new AbortController();

    // ❸ Async fn declared inside — effect itself stays sync
    async function fetchData() {
      try {
        const res = await fetch(url, { signal: controller.signal });
        if (!res.ok) throw new Error(`HTTP ${res.status}: ${res.statusText}`);
        const json = await res.json();
        setData(json);
      } catch (err) {
        if (err.name !== 'AbortError') setError(err.message); // ❹ ignore abort
      } finally {
        setLoading(false);                    // ❺ always clear spinner
      }
    }

    fetchData();

    return () => controller.abort();        // ❻ cleanup on unmount / url change
  }, [url]);

  return { data, loading, error };
}

// Usage
function PostList() {
  const { data: posts, loading, error } = useFetch('/api/posts');

  if (loading) return <p>Loading…</p>;
  if (error)   return <p>Error: {error}</p>;
  return (
    <ul>
      {posts.map(p => <li key={p.id}>{p.title}</li>)}
    </ul>
  );
}
```

FETCH PATTERN CHECKLIST

- Three state vars: `data` (null), `loading` (true), `error` (null)
- Declare async fn *inside* effect — never make the effect callback itself async
- Always check `res.ok` — a 404 does not throw; it resolves with `ok: false`
- Use `AbortController` + cleanup `return () => controller.abort()`
- Catch `AbortError` separately — it's not a real error, just a cancelled request
- Set `loading: false` in `finally` — runs whether fetch succeeded or failed

NOTES

- ❶ Reset state — when `url` changes, clear stale data and show spinner again before the new response arrives
- ❷ `AbortController` — one per effect run; tied to this specific fetch; does not affect other in-flight requests
- ❸ `async function fetchData()` inside effect — the only valid way to use `await` inside `useEffect`
- ❹ `err.name !== 'AbortError'` — an abort is intentional cleanup, not a real error; swallow it silently
- ❺ `finally` — guarantees `loading` is cleared whether the fetch succeeded, errored, or threw unexpectedly
- ❻ Cleanup — `controller.abort()` fires when component unmounts or `url` dep changes; prevents stale state updates

FETCH STATE MACHINE

STATE	LOADING	DATA	ERROR
`Initial`	`true`	`null`	`null`
`Success`	`false`	`[…]`	`null`
`Error`	`false`	`null`	`'msg'`
`Re-fetching`	`true`	`prev data`	`null`

MANUAL FETCH VS QUERY LIBRARY

FEATURE	USEEFFECT	TANSTACK QUERY
`Caching`	✖	✔
`Deduplication`	✖	✔
`Re-fetch on focus`	✖	✔
`Pagination helpers`	✖	✔
`Zero deps`	✔	✖
`Learning curve`	Low	Medium

COMMON MISTAKES

- Making the effect callback `async` directly — React expects the effect to return a cleanup function or nothing; an async function returns a Promise, which React ignores as a cleanup
- Not checking `res.ok` — a 404 or 500 response resolves (doesn't throw); without the check you silently try to parse an error body as JSON
- Missing the AbortController cleanup — navigating away while a slow fetch is in-flight causes a state-update-on-unmounted-component warning and potential bugs

TIPS

- Extract this pattern into a `useFetch` custom hook immediately — you'll need it in multiple components and it's one of the cleanest hook extractions
- For production apps with many data requirements, switch to TanStack Query (Sheet 33) — it handles caching, background refresh, and pagination that `useEffect` fetch cannot
- Show stale data while re-fetching (keep previous `data` during `loading: true`) for a better UX than blanking the page on every refetch

SUMMARY

- `data / loading / error` — three state vars cover the full fetch lifecycle
- Async fn inside effect — never make the effect callback itself async
- `AbortController` — cancel in-flight fetch in the cleanup return
- `res.ok` check — 4xx/5xx responses resolve; you must throw manually

TanStack Query (React Query)

ASYNC STATE MANAGEMENT FOR SERVER DATA

TanStack Query (formerly React Query) manages *server state* — data that lives on the server and must be fetched, cached, and synchronised. It tracks loading, error, and success states automatically, deduplicates concurrent requests for the same key, and re-fetches in the background to keep data fresh.

The two core hooks are `useQuery` (read data) and `useMutation` (write data). Both are configured with a `QueryClient` provided at the app root. After a mutation, call `queryClient.invalidateQueries` to tell TanStack Query to re-fetch any cached queries that depend on the changed data.

TANSTACK QUERY SETUP & USAGE

```
// npm install @tanstack/react-query

// ── main.jsx: provide QueryClient ──────────────────────
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';

const queryClient = new QueryClient({
  defaultOptions: { queries: { staleTime: 1000 * 60 } }, // 1 min stale time
});

<QueryClientProvider client={queryClient}>
  <App />
</QueryClientProvider>

// ── useQuery: fetch and cache data ─────────────────────
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';

function Posts() {
  const { data: posts, isLoading, error } = useQuery({
    queryKey: ['posts'],                    // cache key
    queryFn:  () => fetch('/api/posts').then(r => r.json()),
  });

  if (isLoading) return <p>Loading…</p>;
  if (error)     return <p>Error: {error.message}</p>;
  return <ul>{posts.map(p => <li key={p.id}>{p.title}</li>)}</ul>;
}

// ── Dependent query (wait for userId) ──────────────────
const { data: profile } = useQuery({
  queryKey: ['profile', userId],
  queryFn:  () => fetch(`/api/users/${userId}`).then(r => r.json()),
  enabled:  !!userId,                       // skip until userId is truthy
});

// ── useMutation: write + invalidate ────────────────────
function AddPost() {
  const qc = useQueryClient();
  const mutation = useMutation({
    mutationFn: (newPost) => fetch('/api/posts', {
      method: 'POST', body: JSON.stringify(newPost),
      headers: { 'Content-Type': 'application/json' },
    }).then(r => r.json()),
    onSuccess: () => qc.invalidateQueries({ queryKey: ['posts'] }),
  });

  return (
    <button
      onClick={() => mutation.mutate({ title: 'New Post' })}
      disabled={mutation.isPending}>
      {mutation.isPending ? 'Saving…' : 'Add Post'}
    </button>
  );
}
```

KEY CONCEPTS

- `queryKey` — array identifying a query; same key = same cache entry; include all variables the fetcher depends on
- `queryFn` — async function returning data (or throwing); TanStack Query tracks the Promise
- `staleTime` — ms before data is considered stale and eligible for background re-fetch (default: 0)
- `gcTime` — ms after last subscriber before cache entry is garbage collected (default: 5 min)
- `enabled` — boolean; set to `false` to skip the query until a condition is met
- `invalidateQueries` — marks cache entries as stale and re-fetches active ones

NOTES

- → `queryKey: ['posts']` — array; adding a variable (`['posts', id]`) creates a separate cache entry per value
- → `staleTime: 60_000` — data stays fresh for 1 min; TanStack Query won't re-fetch on component mount within that window
- → `enabled: !!userId` — conditional query; skips the fetch until `userId` is defined; useful for chained requests
- → `invalidateQueries({ queryKey: ['posts'] })` — marks all cache entries matching the key as stale and re-fetches any that have active subscribers
- → `isFetching` vs `isLoading` — `isLoading` is true only on the very first fetch with no cache; `isFetching` is true on any fetch including background refreshes
- → `mutateAsync` — returns a Promise; use when you need to `await` the result inside a form submit handler

USEQUERY RETURN VALUES

PROPERTY	TYPE	MEANING
`data`	`T | undefined`	Resolved query data
`isLoading`	`boolean`	First fetch, no cached data yet
`isFetching`	`boolean`	Any fetch in progress (incl. background)
`isError`	`boolean`	Last fetch threw an error
`error`	`Error`	The thrown error object
`refetch`	`fn`	Manually trigger a re-fetch

USEMUTATION RETURN VALUES

PROPERTY	MEANING
`mutate(vars)`	Trigger the mutation (fire and forget)
`mutateAsync(vars)`	Returns a Promise — await the result
`isPending`	Mutation is in flight
`isSuccess / isError`	Terminal states after settle
`onSuccess callback`	Called with data after success
`onError callback`	Called with error after failure

COMMON MISTAKES

- Using an unstable query key — `queryKey: [{ id }]` with an object literal creates a new reference every render and re-fetches constantly; use primitives in the key array
- Not including all fetcher dependencies in the queryKey — if the fetcher uses `userId`, the key must include it; otherwise stale data is served when the user changes
- Calling `mutate` and expecting immediate data in the same function — `mutate` is async; read the updated data from the re-fetched query, not from the mutation return

TIPS

- Install the `@tanstack/react-query-devtools` package and render `<ReactQueryDevtools />` in development — shows all cache entries and their status in a panel
- Co-locate query key factories in one file: `postKeys = { all: ['posts'], detail: (id) => ['posts', id] }` — prevents key typos and makes invalidation precise
- Use `placeholderData: keepPreviousData` on paginated queries — shows the previous page while the next page loads instead of a loading spinner

SUMMARY

- `useQuery({ queryKey, queryFn })` — fetch, cache, and subscribe to server data
- `useMutation({ mutationFn, onSuccess })` — write data; invalidate on success
- `invalidateQueries` — marks cache stale and triggers re-fetch
- `enabled` — skip query until condition is met; essential for chained requests

SWR

STALE-WHILE-REVALIDATE DATA FETCHING

SWR (by Vercel) is a lightweight React hook library for data fetching. Its name comes from the HTTP cache strategy: return *stale* data immediately (fast), then re-fetch (*revalidate*) in the background and update the UI. This gives the perception of instant responses while keeping data fresh.

SWR is simpler than TanStack Query — it has a single `useSWR` hook with a key, a fetcher, and options. The global `SWRConfig` provider sets defaults. For mutations, call the local or global `mutate` to manually update the cache or trigger a re-fetch. SWR integrates especially well with Next.js (same team).

SWR KEY BEHAVIOURS

- **Stale-while-revalidate** — returns cached data immediately, then fetches fresh data in the background
- **Deduplication** — multiple `useSWR` calls with the same key share one request
- **Revalidate on focus** — re-fetches when the tab regains focus (configurable)
- **Revalidate on reconnect** — re-fetches when network comes back online
- `key = null | false` — disables the fetch; use for conditional/dependent queries
- `mutate(key, data, options)` — global mutate; updates or invalidates any cached key

SWR PATTERNS

```
// npm install swr
import useSWR, { mutate as globalMutate, SWRConfig } from 'swr';

// ── Global fetcher (set once in SWRConfig) ──────────────
const fetcher = (url) => fetch(url).then(r => {
  if (!r.ok) throw new Error(`HTTP ${r.status}`);
  return r.json();
});

// main.jsx — wrap app
<SWRConfig value={{ fetcher, revalidateOnFocus: false }}>
  <App />
</SWRConfig>

// ── Basic usage ─────────────────────────────────────────
function Profile({ userId }) {
  const { data, error, isLoading } = useSWR(`/api/users/${userId}`);

  if (isLoading) return <p>Loading…</p>;
  if (error)     return <p>Error: {error.message}</p>;
  return <p>{data.name}</p>;
}

// ── Conditional fetch (wait for userId) ─────────────────
const { data } = useSWR(userId ? `/api/users/${userId}` : null);

// ── Optimistic update + mutate ──────────────────────────
function LikeButton({ postId, likes }) {
  const { mutate } = useSWR(`/api/posts/${postId}`);

  async function handleLike() {
    // Optimistic: update cache immediately
    mutate({ likes: likes + 1 }, { revalidate: false });
    try {
      await fetch(`/api/posts/${postId}/like`, { method: 'POST' });
    } catch {
      mutate(); // revert: trigger re-fetch on error
    }
  }
  return <button onClick={handleLike}>♥ {likes}</button>;
}

// ── Global mutate: invalidate from outside the component ──
await fetch('/api/posts', { method: 'POST', body: … });
globalMutate('/api/posts');  // re-fetch the posts list anywhere
```

NOTES

- → `SWRConfig` — sets the global fetcher once; all `useSWR` calls inherit it; override per-call by passing a second fetcher arg
- → `key = null` — SWR skips the fetch; use for conditional/dependent queries (like TanStack Query's `enabled`)
- → `mutate({ likes: likes + 1 }, { revalidate: false })` — optimistic update: set cache data immediately without re-fetching; revert by calling `mutate()` with no args
- → `globalMutate('/api/posts')` — invalidates the cache entry for that key from anywhere (outside a component, after a form submit)
- → `isLoading` vs `isValidating` — `isLoading` true only when no cache exists; `isValidating` true during any background refresh
- → `refreshInterval: 5000` — poll every 5 s; useful for live dashboards, chat, or stock prices

USESWR RETURN VALUES

PROPERTY	MEANING
`data`	Cached or fetched data (undefined until first response)
`error`	Error thrown by fetcher
`isLoading`	No cached data + fetch in progress
`isValidating`	Any fetch in progress (incl. background refresh)
`mutate(data?)`	Update local cache and/or trigger re-fetch

COMMON SWR OPTIONS

OPTION	DEFAULT	PURPOSE
`revalidateOnFocus`	`true`	Re-fetch on tab focus
`revalidateOnReconnect`	`true`	Re-fetch on network reconnect
`refreshInterval`	`0`	Polling interval in ms (0 = off)
`dedupingInterval`	`2000`	Suppress duplicate requests within ms
`fallbackData`	`undefined`	Initial data before first fetch
`onSuccess(data)`	—	Callback after successful fetch

COMMON MISTAKES

- Creating the fetcher function inside a component — new reference on every render causes SWR to think the fetcher changed and re-fetch; define it outside the component
- Using an object or array as the SWR key without stability — SWR compares keys with JSON.stringify, so objects work, but large/nested objects slow down the comparison; prefer string keys
- Forgetting to revert on optimistic update failure — update the cache immediately but catch errors and call `mutate()` (no args) to re-fetch the real data

TIPS

- Use an array key when the fetch depends on multiple values: `useSWR(['/api/data', userId, page])` — SWR serialises it for the cache
- Set `revalidateOnFocus: false` globally for dashboards where background polling is preferred over focus-triggered re-fetches
- SWR and TanStack Query are interchangeable for most use cases; SWR is simpler; TanStack Query has more built-in features (infinite queries, paginated helpers, devtools)

SUMMARY

- `useSWR(key, fetcher)` — stale-while-revalidate; returns data/error/isLoading
- `key = null` — disables fetch; use for conditional queries
- `mutate(data, opts)` — optimistic update; no-arg call triggers re-fetch
- `SWRConfig` — set global fetcher and defaults once at app root

React.memo & Memoization

SKIPPING UNNECESSARY RE-RENDERS

`React.memo` is a higher-order component that wraps a function component and **skips re-rendering if props are shallowly equal** to the previous render. By default, React re-renders a child whenever its parent renders, even if none of the child's props changed. `React.memo` prevents this by doing a `Object.is` comparison on each prop.

`React.memo` only helps when the wrapped component's render is genuinely expensive, or when it renders a long list and even one skipped render makes a difference. For it to work, every prop passed to the memo'd component must be **referentially stable** — use `useCallback` for function props and `useMemo` for object/array props.

REACT.MEMO IN PRACTICE

```
import { memo, useState, useCallback, useMemo } from 'react';

// ── Wrap component with memo ──────────────────────────
const ProductCard = memo(function ProductCard({ product, onBuy }) {
  console.log('ProductCard rendered:', product.id);
  return (
    <div>
      <p>{product.name} — ${product.price}</p>
      <button onClick={() => onBuy(product.id)}>Buy</button>
    </div>
  );
});

// ── Parent must stabilise every prop ──────────────────
function Shop({ products }) {
  const [cart, setCart] = useState([]);

  // ✔ useCallback stabilises the function reference
  const handleBuy = useCallback(
    (id) => setCart(prev => [...prev, id]),
    []   // setCart is stable; empty deps OK
  );

  // ✔ useMemo stabilises an object prop (if you need one)
  const options = useMemo(() => ({ currency: 'USD', tax: 0.1 }), []);

  return (
    <ul>
      {products.map(p => (
        <ProductCard key={p.id} product={p} onBuy={handleBuy} />
      ))}
    </ul>
  );
}

// ── Custom comparison (skip re-render unless id changes) ──
const UserBadge = memo(
  function UserBadge({ user }) { return <span>{user.name}</span>; },
  (prev, next) => prev.user.id === next.user.id  // true = skip render
);

// ── What NOT to do ────────────────────────────────────
// ✖ inline object — new reference every render, memo never skips
<ProductCard product={products[0]} options={{ tax: 0.1 }} onBuy={id => buy(id)} />
```

REACT.MEMO RULES

- Shallow comparison — primitive props (string, number, bool) are compared by value; objects and functions by reference
- A new object/function reference as a prop **breaks memo** even if the content is identical
- Pair with `useCallback` for callback props and `useMemo` for object/array props
- Second arg: custom comparison — `React.memo(Comp, (prev, next) => equal)` ; return `true` to skip render
- `React.memo` does not help if the component's own state or context changes — it only gates on props
- Profile first — adding memo unnecessarily increases code complexity and adds the comparison overhead

NOTES

- → `memo(Component)` — wraps the component; React compares old and new props shallowly before calling the function
- → `useCallback(fn, [])` — same function reference across renders; without it, `onBuy` is a new function on every parent render and memo always re-renders
- → `prev.user.id === next.user.id` — custom comparator; return `true` to skip render (props are "equal"), `false` to re-render
- → `{ tax: 0.1 }` inline — creates a new object on every render; `Object.is` returns false; memo is bypassed every time
- → Memo is not free — the comparison itself takes time; only worthwhile when the render cost exceeds the comparison cost
- → React DevTools Profiler — use it to measure render counts and durations before and after applying memo

WHEN REACT.MEMO HELPS

SCENARIO	HELPFUL?
Expensive render (chart, table)	✔
Long list item component	✔
Parent re-renders frequently	✔
All props are primitives	✔ easy to stabilise
Component renders in <1ms	✖ overhead not worth it
Props include unstable objects/fns	✖ memo never skips

MEMOIZATION TOOLKIT

TOOL	STABILISES
React.memo	The component render itself
useCallback	Function prop references
useMemo	Object / array prop references
useState setter	Always stable — no memo needed
useReducer dispatch	Always stable — no memo needed
Context value	Wrap with useMemo to stabilise

COMMON MISTAKES

- Wrapping a component in `memo` but passing an inline function or object prop — the unstable prop defeats memo on every render; stabilise props first
- Using a custom comparator that returns `true` too aggressively — the component will never update even when it should; be precise about what "equal" means
- Applying memo to every component by default — adds complexity and comparison cost without benefit when renders are already cheap

TIPS

- Apply memo bottom-up — start with the most frequently rendered leaf components (list items, cards) before wrapping parent components
- Use React DevTools Profiler's "Highlight updates" mode to visually identify which components re-render on each interaction
- If a component has no props at all (pure display), wrap it in memo — it will never re-render from parent renders, with zero cost

SUMMARY

- `memo(Comp)` — skips re-render when props are shallowly equal
- Requires stable props — use `useCallback` for fns, `useMemo` for objects
- Custom comparator: second arg — return `true` to skip, `false` to re-render
- Profile before applying — memo adds overhead; only worthwhile for expensive renders

Code Splitting & Lazy Loading

SPLITTING THE BUNDLE AT BOUNDARIES

By default, bundlers (Vite, Webpack) combine all JS into a single file. **Code splitting** divides the bundle into smaller chunks loaded on demand. `React.lazy()` + a dynamic `import()` tells the bundler to create a separate chunk for that component, loaded only when the component first renders.

A `<Suspense>` boundary must wrap any lazy component. It renders the `fallback` prop (a spinner, skeleton, or loading message) while the chunk is downloading, then swaps in the real component. The most impactful place to split is at **route level** — each page becomes its own chunk and the initial bundle only contains the landing page code.

CODE SPLITTING RULES

- `React.lazy(() => import('./Comp'))` — the import must resolve to a module with a `default` export
- `<Suspense fallback={…}>` — required ancestor of the lazy component; no ancestor = thrown error
- The fallback renders while the chunk *downloads*, not while the component renders
- Multiple lazy components under one `<Suspense>` — all share the same fallback; show a single spinner
- Named exports — `lazy(() => import('./Comp').then(m => ({ default: m.MyComp })))`
- Preload a chunk before it's needed — call `import('./Page')` on hover/mousedown

NOTES

- → `lazy(() => import('./Page'))` — bundler creates a separate chunk; the Promise is tracked by Suspense
- → `<Suspense fallback>` at the route level — wraps all pages; the nav bar stays mounted while the page chunk loads
- → `{open && <Suspense>…}` — Suspense inside a condition; the chunk isn't even requested until `open` is true
- → `.then(m => ({ default: m.BarChart }))` — normalises a named export to a default export shape that `React.lazy` requires
- → `onMouseEnter={preload}` — calling `import()` on hover kicks off the network request 100–200 ms before the click; the chunk arrives faster
- → Check bundle sizes with `vite build --report` or the Rollup Bundle Visualiser plugin to identify the best split points

GOOD CODE SPLIT BOUNDARIES

BOUNDARY	BENEFIT
`Route / page components`	Largest impact; each page = one chunk
`Heavy modal / drawer`	Loaded only when user opens it
`Rich text / chart library`	Large library loaded only on the page that needs it
`Admin panel`	Only users who visit admin pay the download cost
`Small utility component`	Too small — chunk overhead not worth it

SUSPENSE BOUNDARY PLACEMENT

WHERE	EFFECT
`App root`	One global fallback for all lazy components
`Layout (around <Outlet>)`	Page-level spinner; nav stays visible
`Per-component`	Granular; different fallbacks for different parts
`Missing`	React throws — lazy component crashes the app

LAZY LOADING PATTERNS

```jsx
import { lazy, Suspense, useState } from 'react';

// ── Route-level code splitting ──────────────────────
const HomePage   = lazy(() => import('./pages/HomePage'));
const Dashboard  = lazy(() => import('./pages/Dashboard'));
const AdminPanel = lazy(() => import('./pages/AdminPanel'));

function App() {
  return (
    <Suspense fallback={<div className="page-spinner">Loading…</div>}>
      <Routes>
        <Route path="/"          element={<HomePage />} />
        <Route path="/dashboard" element={<Dashboard />} />
        <Route path="/admin"     element={<AdminPanel />} />
      </Routes>
    </Suspense>
  );
}

// ── Named export — wrap to make it a default ────────
const BarChart = lazy(() =>
  import('./charts').then(m => ({ default: m.BarChart }))
);

// ── Lazy modal — load only when opened ──────────────
const HeavyModal = lazy(() => import('./HeavyModal'));

function Page() {
  const [open, setOpen] = useState(false);
  return (
    <>
      <button onClick={() => setOpen(true)}>Open Modal</button>
      {open && (
        <Suspense fallback={<p>Loading modal…</p>}>
          <HeavyModal onClose={() => setOpen(false)} />
        </Suspense>
      )}
    </>
  );
}

// ── Preload on hover ────────────────────────────────
const preload = () => import('./pages/Dashboard');
      // kicks off the download

<Link to="/dashboard" onMouseEnter={preload}>Dashboard</Link>
```

COMMON MISTAKES

- Defining the lazy component inside a render function — a new lazy import is created on every render; always define at the module level outside the component
- Forgetting a `<Suspense>` ancestor — React throws immediately when the lazy chunk starts loading; wrap early
- Lazy-loading tiny components — the chunk round-trip overhead (~20ms) can exceed the parse time saved; only split components that are genuinely large

TIPS

- Start with route-level splitting only — it usually cuts the initial bundle by 40–60% and requires the least effort
- Place a `<Suspense>` inside the layout component (around `<Outlet>`) so the nav and header remain visible during page transitions
- Pair with an `errorElement` (React Router) or an Error Boundary component to handle chunk load failures (network error, CDN timeout)

SUMMARY

- `React.lazy(() => import('./Page'))` — creates a separate JS chunk
- `<Suspense fallback>` — required ancestor; shows fallback during download
- Route-level splitting — highest impact; one chunk per page
- Preload on hover — call `import()` early to reduce perceived latency

Render Optimization

HOW REACT DECIDES WHAT TO RE-RENDER

React's **reconciliation** algorithm compares the virtual DOM tree from the previous render to the new one. It updates only the DOM nodes that actually changed. Two rules drive this: (1) elements of different types are destroyed and rebuilt; (2) elements of the same type are updated in place. The `key` prop tells React which list items are the same across renders.

React 18 introduced **automatic batching**: multiple state updates in a single event handler — or even in a timeout or Promise — are batched into one re-render. This means calling `setA()` + `setB()` triggers one render, not two. Use `flushSync` from `react-dom` to opt out of batching when you need an immediate DOM update.

OPTIMIZATION CHECKLIST

- **Profile first** — React DevTools Profiler shows render counts, durations, and what triggered each render
- **Stable keys** — use data IDs as list keys; index keys cause full re-mounts on reorder
- **Stable references** — avoid inline objects/arrays/functions as props; they create new references every render
- **State colocation** — keep state as close to its consumer as possible; high state causes wide re-renders
- **Avoid context for hot paths** — every consumer re-renders on any context value change
- **Virtualise long lists** — use `react-window` or `TanStack Virtual` for 100+ item lists

OPTIMIZATION PATTERNS

```
import { useState, useCallback, Profiler } from 'react';
import { flushSync } from 'react-dom';

// ── Automatic batching (React 18) ──────────────────────
// Both setters fire → ONE re-render (not two)
function handleClick() {
  setCount(c => c + 1);
  setFlag(f => !f);
}

// ── flushSync: opt out of batching ─────────────────────
flushSync(() => setCount(c => c + 1));  // DOM updated synchronously
// Use sparingly — e.g. before reading a layout measurement

// ── State colocation: keep state near its consumer ─────
// ✖ high state — every child re-renders on count change
function App() {
  const [count, setCount] = useState(0);
  return <><Counter count={count} onInc={setCount} /><HeavySection /></>;
}

// ✔ colocated — HeavySection never re-renders
function CounterSection() {
  const [count, setCount] = useState(0);  // state stays inside this subtree
  return <Counter count={count} onInc={setCount} />;
}
function App() {
  return <><CounterSection /><HeavySection /></>;
}

// ── Profiler: measure render cost ──────────────────────
function onRender(id, phase, actualDuration) {
  console.log(`${id} [${phase}]: ${actualDuration.toFixed(2)}ms`);
}
<Profiler id="ProductList" onRender={onRender}>
  <ProductList products={products} />
</Profiler>

// ── Virtualise long lists ──────────────────────────────
// npm install react-window
import { FixedSizeList } from 'react-window';
<FixedSizeList height={400} itemCount={10000} itemSize={50} width="100%">
  {({ index, style }) => <div style={style}>Row {index}</div>}
</FixedSizeList>
```

NOTES

- → **Batching** — React 18 batches all state updates by default, including those inside setTimeout and Promises; previously only event handlers were batched
- → `flushSync` — forces a synchronous DOM update before returning; only needed when you need to read a layout value immediately after a state change
- → **Colocation** — moving state down into a subtree is the single most effective optimisation; it narrows the re-render blast radius without any memo overhead
- → `Profiler onRender` — receives `id`, `phase` ('mount'|'update'), `actualDuration` (ms this render took), and `baseDuration` (estimated time without memo)
- → `FixedSizeList` — only renders the visible rows; 10 000 items render as fast as 10 items; essential for any list the user scrolls
- → Profile → colocate → memo — always in that order; premature memo without profiling adds complexity for no gain

WHAT CAUSES A RE-RENDER

TRIGGER	RE-RENDERS
`setState / dispatch called`	Component + all descendants
`Parent re-renders`	Child (unless memo'd with stable props)
`Context value changes`	All useContext(Ctx) consumers
`forceUpdate (class)`	Component + descendants
`Same state value set`	Bails out (Object.is check)

PROFILING & MEASUREMENT TOOLS

TOOL	USE FOR
`React DevTools Profiler`	Render counts, durations, flame graph
`<Profiler> component`	Programmatic render timing in code
`why-did-you-render`	Log unnecessary re-renders to console
`Chrome Performance tab`	Long tasks, layout thrashing, paint cost

COMMON MISTAKES

- Optimising before profiling — most renders are fast enough; add memo/useCallback only where the Profiler shows actual performance problems
- Lifting state too high — a counter at the App root re-renders the entire tree on every click; colocate state in the smallest subtree that needs it
- Rendering 500+ unvirtualised list items — each item is a real DOM node; layout and paint cost grows linearly; virtualise once the list exceeds ~100 items

TIPS

- Use the React DevTools Profiler's "Record why each component rendered" setting — it tells you exactly which prop or state changed to trigger a render
- Install `@welldone-software/why-did-you-render` in development to get console warnings whenever a component re-renders with equal props
- Batch related state into one `useState` object or use `useReducer` — fewer state vars means fewer potential re-render triggers

SUMMARY

- **Reconciliation** — React diffs virtual DOM; same type = update in place
- **Batching** — React 18 batches all setState calls into one render
- **Colocation** — move state down to narrow re-render blast radius
- **Virtualise** — use react-window for lists with 100+ items

CSS Modules & Global Styles

SCOPED CSS WITHOUT A RUNTIME

CSS Modules are plain `.css` files with a special naming convention (`*.module.css`). Vite and Create React App automatically scope every class name to the component that imports it — the bundler rewrites `.card` to something like `.card_x7f3a`, so styles never leak to other components.

You import the module as a JS object (`styles`) and apply class names via `className={styles.card}`. Global styles (resets, fonts, design tokens) live in a plain `.css` file imported once in `main.jsx`. CSS custom properties (variables) work inside module files and are the recommended way to share values like colours and spacing across components.

CSS MODULES KEY POINTS

- File must end in `.module.css` for the bundler to apply scoping
- Import as a default: `import styles from './Card.module.css'`
- `styles.className` — returns the hashed class name string
- Hyphened names — access as `styles['nav-link']` or rename to camelCase in the CSS
- `composes: base from './base.module.css'` — inherit classes across modules
- `:global(.cls)` — opt out of scoping for a specific selector (e.g. third-party library class)

CSS MODULES IN PRACTICE

```
/* ── Card.module.css ──────────────────────────────── */
.card {
  border: 1px solid var(--color-border);  /* CSS var from global */
  border-radius: 8px;
  padding: 16px;
}

.card.active {          /* compound selector */
  border-color: var(--color-primary);
}

.title {
  font-size: 1.25rem;
  font-weight: 700;
}

:global(.third-party-class) {  /* opt out of scoping */
  color: red;
}

/* ── Card.jsx ─────────────────────────────────────── */
import styles from './Card.module.css';
import clsx from 'clsx';                // npm install clsx

function Card({ title, isActive, children }) {
  return (
    <div className={clsx(styles.card, { [styles.active]: isActive })}>
      <h2 className={styles.title}>{title}</h2>
      {children}
    </div>
  );
}

/* ── global.css (imported once in main.jsx) ──────── */
/* :root { --color-primary: #5c6bc0; --color-border: #e0e0e0; } */
/* *, *::before, *::after { box-sizing: border-box; } */
/* body { margin: 0; font-family: sans-serif; } */

/* ── Inline styles (dynamic values) ──────────────── */
<div style={{ color: theme.primary, fontSize: `${size}px` }}>…</div>
```

NOTES

- `styles.card` — resolves to a hashed string like `"Card_card__x7f3a"` at runtime; collision-free by design
- `clsx(a, { b: bool })` — utility that joins class names and conditionally includes them; cleaner than template literals with ternaries
- `var(--color-primary)` — CSS custom properties flow through the cascade normally; define them in `:root` in your global CSS file
- `:global(.cls)` — tells the bundler not to scope this selector; needed when a third-party library adds a class name you need to target
- `style={{ fontSize: `${size}px` }}` — use inline styles for values computed at runtime (user preferences, animation progress, dynamic colours)
- CSS Modules do not support nesting by default in older setups; Vite supports it natively with modern CSS

CSS STYLING OPTIONS IN REACT

APPROACH	SCOPED?	RUNTIME COST
CSS Modules	✔	None — build time
Inline styles	✔ (element)	None — JS objects
Global CSS	✖	None
Styled Components	✔	Small — style injection
Tailwind CSS	✔ (atomic)	None — utility classes

COMBINING CLASS NAMES

PATTERN	EXAMPLE
Two module classes	`` `${styles.card} ${styles.active}` ``
Conditional class	`isActive ? styles.active : ''`
clsx library	`clsx(styles.card, { [styles.active]: isActive })`
Global + module	`` `container ${styles.card}` ``

COMMON MISTAKES

- Naming the file `Card.css` instead of `Card.module.css` — bundler treats it as a global stylesheet and classes are not scoped
- Using a hyphenated class name with dot notation (`styles.nav-link`) — JS identifier syntax error; use bracket notation `styles['nav-link']` or rename to camelCase
- Putting all styles in inline style objects — no CSS cascade, no pseudo-selectors (`:hover` , `:focus`), no media queries; CSS Modules handle these natively

TIPS

- Install `clsx` (or `classnames`) immediately — combining conditional class names with template literals gets unwieldy fast
- Define all design tokens (colours, spacing, font sizes) as CSS custom properties in `:root` in global CSS — modules inherit them without any extra tooling
- Co-locate the module file with its component (`Card.jsx` + `Card.module.css` in the same folder) — easy to find, easy to delete together

SUMMARY

- `*.module.css` — bundler scopes all class names to the importing component
- `styles.className` — returns the hashed, unique class name string
- `clsx(styles.a, { [styles.b]: cond })` — clean conditional class combining
- `:root` CSS vars in global.css — shared design tokens accessible in every module

Styled Components

CSS-IN-JS WITH STYLED COMPONENTS

Styled Components is a CSS-in-JS library that lets you write actual CSS inside JavaScript using tagged template literals. Each call to `styled.element` returns a new React component with those styles applied and a unique class name injected at runtime. Styles are scoped to the component by default.

The key advantage is **dynamic styles from props** — you can interpolate functions into the template literal that receive the component's props and return CSS values. `ThemeProvider` makes a theme object available to all styled components in its tree via context. `createGlobalStyle` injects CSS into `<head>` for resets and global rules.

STYLED COMPONENTS EXAMPLES

```
// npm install styled-components
import styled, { ThemeProvider, createGlobalStyle, css } from 'styled-components';

// ── Basic styled component ──────────────────────────
const Card = styled.div`
  border: 1px solid #ddd;
  border-radius: 8px;
  padding: 16px;

  &:hover {              /* & = the component itself */
    border-color: #5c6bc0;
  }
`;

// ── Dynamic styles from props ───────────────────────
const Button = styled.button`
  padding: 8px 16px;
  border-radius: 4px;
  background: ${props => props.$primary ? '#5c6bc0' : 'white'};
  color:      ${props => props.$primary ? 'white'   : '#333'};
  border: 1px solid #5c6bc0;
`;
// Usage: <Button $primary>Save</Button> or <Button>Cancel</Button>
// Transient props ($ prefix) are not forwarded to the DOM element

// ── Extending a styled component ────────────────────
const DangerButton = styled(Button)`
  background: #e53935;
  border-color: #e53935;
`;

// ── ThemeProvider ───────────────────────────────────
const theme = { primary: '#5c6bc0', radius: '6px' };

const ThemedCard = styled.div`
  border-radius: ${p => p.theme.radius};
  background: ${p => p.theme.primary}22;
`;

<ThemeProvider theme={theme}>
  <ThemedCard>Uses theme values</ThemedCard>
</ThemeProvider>

// ── Global styles ───────────────────────────────────
const GlobalStyle = createGlobalStyle`
  *, *::before, *::after { box-sizing: border-box; }
  body { margin: 0; font-family: sans-serif; }
`;
// Render once: <><GlobalStyle /><App /></>
```

KEY PATTERNS

- `styled.div`css`` — creates a div with scoped styles; any HTML tag works
- `styled(Component)`css`` — extends any component (including other styled ones)
- `${props => props.primary ? 'blue' : 'grey'}` — dynamic CSS from props
- `ThemeProvider` — passes a theme object to all styled components as `props.theme`
- `createGlobalStyle` — injects global CSS (resets, fonts, CSS variables)
- `.attrs({ type: 'text' })` — set default HTML attributes on the styled element

NOTES

- → `&:hover` — the ampersand refers to the generated class name; supports all CSS selectors including pseudo-elements and media queries
- → `$primary` — transient prop (dollar prefix); styled-components does not forward it to the DOM, avoiding invalid HTML attribute warnings
- → `styled(Button)`css`` — creates a new component with the base styles plus the extended CSS; all Button props are inherited
- → `p.theme.primary` — theme flows through context automatically to every styled component inside ThemeProvider; no manual prop passing
- → `createGlobalStyle` — injects a `<style>` tag into `<head>`; render the component once in the tree to activate it
- → Styled components defined at module level — never define them inside a component function; a new component type is created on every render, causing remounts

STYLED COMPONENTS API

API	PURPOSE
`styled.div`css``	Create a styled
`styled(Comp)`css``	Extend any component's styles
`css helper`	Share CSS snippets between components
`ThemeProvider`	Provide theme to all styled children
`createGlobalStyle`	Inject global CSS rules
`keyframes`	Define CSS animation keyframes

STYLED COMPONENTS VS CSS MODULES

FEATURE	STYLED COMPONENTS	CSS MODULES
`Props-based styles`	✔ native	✖ needs inline
`Runtime cost`	Small (style injection)	None (build time)
`SSR support`	Needs ServerStyleSheet	Automatic
`TypeScript`	Props typed via generics	Via CSS Modules types
`Co-location`	✔ in same .jsx file	✔ in same folder

COMMON MISTAKES

- Defining styled components inside a React component function — a new component type is created on every render, causing the DOM node to unmount and remount
- Forwarding non-standard props to DOM elements (e.g. `primary` on a button) — browser warns about unknown attributes; use the `$` transient prop prefix
- Overusing prop-based styles for static variants — an `if/else` in template literals for two variants is harder to read than two separate styled components

TIPS

- Use the **Babel plugin for styled-components** — it adds display names and source maps, making components identifiable in DevTools by their variable name
- Keep styled component definitions at the bottom of the file (after the component functions) or in a separate `styles.js` file for large components
- For SSR (Next.js), use `ServerStyleSheet` from styled-components to collect styles and inject them in `_document.jsx` to avoid flash of unstyled content

SUMMARY

- `styled.div`css`` — scoped styles in a tagged template literal
- `${p => p.$primary ? … : …}` — dynamic CSS from props; use `$` prefix for transient props
- `ThemeProvider` — passes theme to all styled descendants via context
- Always define styled components at module level, never inside render

Tailwind CSS in React

UTILITY-FIRST CSS IN REACT

Tailwind CSS is a utility-first framework: instead of writing CSS classes, you compose low-level utility classes directly in `className`. Tailwind scans your source files for class names and generates only the CSS you use — the production stylesheet is tiny. There is no runtime cost; all styles are in a single CSS file loaded at page start.

The main challenge with Tailwind in React is **conditional and dynamic class names**. The `clsx` library (or the `cn` utility from shadcn/ui that combines clsx with Tailwind Merge) handles conditional classes cleanly and also resolves Tailwind class conflicts — e.g. merging p-4 and p-8 correctly discards the first.

TAILWIND + REACT RULES

- Full class names must appear in source — never build class names with string concatenation (`'text-' + color`) or Tailwind won't include them
- Use `clsx` or `cn` for conditional classes — template literals get messy with multiple conditions
- Tailwind Merge (`twMerge`) resolves conflicting utility classes — use it when composing components that accept className overrides
- Dark mode — add `dark:` prefix; configure `darkMode: 'class'` in `tailwind.config.js` to toggle via JS
- Arbitrary values — `w-[372px]` , `text-[#5c6bc0]` — one-off values that aren't in the default scale
- Extend the theme in `tailwind.config.js` for brand colours, fonts, and custom spacing

TAILWIND IN REACT COMPONENTS

```
// npm install -D tailwindcss @tailwindcss/vite
// Add to vite.config.js: plugins: [tailwindcss()]
// Add to main.css: @import "tailwindcss";

// ── cn utility (shadcn/ui pattern) ──────────────────
import { clsx } from 'clsx';
import { twMerge } from 'tailwind-merge';

function cn(...inputs) {
  return twMerge(clsx(inputs));  // merge + resolve conflicts
}

// ── Button component with variants ──────────────────
function Button({ variant = 'primary', disabled, className, children, ...rest }) {
  return (
    <button
      className={cn(
        'px-4 py-2 rounded-md font-medium transition-colors',
        variant === 'primary' && 'bg-blue-600 text-white hover:bg-blue-700',
        variant === 'danger'  && 'bg-red-600 text-white hover:bg-red-700',
        variant === 'outline' && 'border border-gray-300 text-gray-700 hover:bg-gray-50',
        disabled && 'opacity-50 cursor-not-allowed',
        className  // allow caller to override
      )}
      disabled={disabled}
      {...rest}
    >
      {children}
    </button>
  );
}

// ── Dark mode (class strategy) ──────────────────────
// tailwind.config.js: darkMode: 'class'
// Toggle: document.documentElement.classList.toggle('dark')
<div className="bg-white dark:bg-gray-900 text-gray-900 dark:text-white">
  Content adapts to dark mode
</div>

// ── Do NOT build class names dynamically ────────────
// ✖ Tailwind can't find 'text-red-500' in source → not generated
const color = 'red';
<p className={`text-${color}-500`}>Wrong</p>

// ✔ Use full class names in conditionals
<p className={error ? 'text-red-500' : 'text-green-500'}>Correct</p>
```

NOTES

- → `cn(...inputs)` — combines `clsx` (conditional joining) with `twMerge` (conflict resolution); e.g. `cn('p-4', 'p-8')` → `'p-8'`
- → `variant === 'primary' && '…'` — returns the class string or `false` ; `clsx` filters out falsy values automatically
- → `className` prop — pass it last in `cn()` so caller overrides win after `twMerge` resolves conflicts
- → `dark:bg-gray-900` — only applied when the `dark` class is on `<html>` ; toggle with JS or match `prefers-color-scheme` media query
- → `` `text-${color}-500` `` — string interpolation hides class names from Tailwind's static scanner; the class is never generated
- → Arbitrary values — `w-[372px]` generates a one-off utility; useful for pixel-perfect designs outside the default scale

COMMON TAILWIND CLASS GROUPS

CSS PROPERTY	TAILWIND EXAMPLE
display	flex · grid · hidden · block
flexbox	items-center · justify-between · gap-4
spacing	p-4 · px-6 · m-auto · mt-2
sizing	w-full · h-screen · max-w-lg
typography	text-lg · font-bold · text-gray-700
background	bg-white · bg-blue-500 · bg-opacity-50
border	border · rounded-lg · border-gray-200

TAILWIND MODIFIERS

MODIFIER	EXAMPLE	MEANING
hover:	hover:bg-blue-600	On mouse hover
focus:	focus:ring-2	On keyboard focus
dark:	dark:bg-gray-900	In dark mode
sm: md: lg:	md:flex-row	Responsive breakpoints
disabled:	disabled:opacity-50	Disabled state
group-hover:	group-hover:text-blue-500	Parent group hovered

COMMON MISTAKES

- Building class names with string interpolation — Tailwind's scanner won't find the interpolated class and it won't be in the final CSS
- Not using Tailwind Merge when composing — `twMerge('p-4', 'p-8')` resolves to `'p-8'` ; without it both classes end up in the DOM and whichever has higher specificity wins unpredictably
- Forgetting to configure `content` paths in `tailwind.config.js` — Tailwind only scans the files you list; classes in unlisted files are never generated

TIPS

- Install the **Tailwind CSS IntelliSense** VS Code extension — autocompletes class names, shows the generated CSS on hover, and flags unknown classes
- Use the `cn` utility pattern for every component that accepts a `className` prop — it makes the component composable and conflict-safe
- Define component-level variants in a `cva` (class-variance-authority) call — structured variant management for design systems

SUMMARY

- Full class names in source — never interpolate; Tailwind scanner is static
- `cn(clsx + twMerge)` — conditional classes + conflict resolution
- `dark:` modifier + `darkMode: 'class'` config — JS-toggled dark mode
- Arbitrary values `w-[372px]` — one-off values outside the default scale

Testing Library — Basics

TESTING FROM THE USER'S PERSPECTIVE

React Testing Library (RTL) renders components into a real DOM (via jsdom) and encourages querying elements the way a user would interact with them — by visible text, role, label, or placeholder — rather than by CSS class or implementation detail. This makes tests resilient to refactors.

The core functions are `render` (mount a component), `screen` (query the rendered output), and **assertions** from `@testing-library/jest-dom`. Queries come in three families: `getBy` (throws if not found), `queryBy` (returns null if not found), and `findBy` (returns a Promise — for async elements).

RTL BASICS

```
// npm install -D @testing-library/react @testing-library/jest-dom vitest jsdom
// vitest.config.js: environment: 'jsdom', setupFiles: ['./src/setup.ts']
// setup.ts: import '@testing-library/jest-dom';

import { render, screen } from '@testing-library/react';
import { describe, it, expect } from 'vitest';
import Greeting from './Greeting';

// ── Basic render + query ──────────────────────────────
describe('Greeting', () => {
  it('displays the user name', () => {
    render(<Greeting name="Ada" />);

    // getByRole — preferred; also tests accessibility
    const heading = screen.getByRole('heading', { name: /ada/i });
    expect(heading).toBeInTheDocument();

    // getByText — for non-interactive text
    expect(screen.getByText(/welcome/i)).toBeVisible();
  });

  it('shows a login link when not authenticated', () => {
    render(<Greeting />);

    // queryBy — use when element might not exist
    expect(screen.queryByRole('heading')).toBeNull();
    expect(screen.getByRole('link', { name: /log in/i })).toBeInTheDocument();
  });
});

// ── findBy — for elements that appear asynchronously ──
it('shows results after loading', async () => {
  render(<SearchResults query="react" />);

  // findBy waits (default 1s) for element to appear
  const item = await screen.findByRole('listitem');
  expect(item).toHaveTextContent('React docs');
});

// ── Common role names ─────────────────────────────────
// button, link, heading, textbox, checkbox, radio,
// combobox (select), listitem, list, dialog, alert
```

QUERY PRIORITY (RTL GUIDE)

- `getByRole` — best; queries by ARIA role + accessible name; tests accessibility too
- `getByLabelText` — for form fields linked to a label
- `getByPlaceholderText` — for inputs without a label (use sparingly)
- `getByText` — for non-interactive elements: paragraphs, headings, spans
- `getByDisplayValue` — for the current value of an input/select/textarea
- `getByTestId` — last resort; add `data-testid` attribute only when no semantic query works

NOTES

- → `screen.getByRole('heading', { name: /ada/i })` — matches any heading whose accessible name matches the regex; case-insensitive
- → `/ada/i` — regex is preferred over exact strings; tests pass even if surrounding text changes
- → `queryBy` — returns null instead of throwing; use when asserting an element is absent: `expect(screen.queryByText('Error')).not.toBeInTheDocument()`
- → `findBy` — internally polls with `waitFor`; use for elements that appear after a fetch, timer, or state update
- → `toBeInTheDocument()` — from `@testing-library/jest-dom`; must be imported in setup; not available in plain Jest/Vitest
- → RTL automatically cleans up the DOM after each test (`afterEach`); no manual cleanup needed with Vitest + jsdom

QUERY FAMILY COMPARISON

PREFIX	NOT FOUND	MULTIPLE FOUND	ASYNC?
`getBy`	Throws error	Throws error	✖
`queryBy`	Returns null	Throws error	✖
`findBy`	Rejects (timeout)	Rejects	✔
`getAllBy`	Throws error	Returns array	✖
`queryAllBy`	Returns []	Returns array	✖
`findAllBy`	Rejects	Returns array	✔

JEST-DOM MATCHERS

MATCHER	ASSERTS
`toBeInTheDocument()`	Element exists in the DOM
`toBeVisible()`	Element is visible to the user
`toBeDisabled()`	Button/input is disabled
`toHaveTextContent('txt')`	Element contains the text
`toHaveValue('val')`	Input has this value
`toHaveClass('cls')`	Element has the CSS class

COMMON MISTAKES

- Using `getByTestId` for everything — it tests implementation details, not user-visible behaviour; reach for `getByRole` and `getByText` first
- Using `getBy` for an element that might be absent — `getBy` throws immediately; use `queryBy` when testing that something does *not* exist
- Forgetting `await` on `findBy` queries — the test passes trivially because the Promise is never awaited and the assertion is skipped

TIPS

- Use `screen.debug()` to print the current DOM to the console when a query isn't finding what you expect
- Run `screen.getByRole('...')` with no options first to see what roles are available — helpful when the exact name is unclear
- Follow the RTL query priority: role → label → placeholder → text → display value → test id; it nudges you toward accessible markup

SUMMARY

- `render(<Comp />)` — mounts into jsdom; `screen` queries the output
- `getBy` — throws if absent; `queryBy` — null; `findBy` — async/await
- `getByRole` — best query; checks ARIA roles + accessible names
- `toBeInTheDocument / toBeVisible / toHaveTextContent` — jest-dom matchers

Testing Library — Interactions

SIMULATING REAL USER INTERACTIONS

userEvent (from `@testing-library/user-event`) simulates real browser events — it fires the full sequence a real user would trigger (pointerdown, mousedown, click, focus, etc.) and properly moves focus. Prefer it over the lower-level `fireEvent` for all interactions.

`userEvent.setup()` creates a session that tracks state between interactions (clipboard, focus, keyboard modifiers). Always call it at the top of your test and use the returned instance's methods. `waitFor` lets you assert on async state changes after an interaction. `act` is used automatically by RTL but you need it manually when testing state updates outside React's event system.

INTERACTION TESTING PATTERNS

```
import { render, screen, waitFor } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { describe, it, expect, vi } from 'vitest';

// ── Click test ──────────────────────────────────────────
it('increments count on click', async () => {
  const user = userEvent.setup();
  render(<Counter />);

  const btn = screen.getByRole('button', { name: /increment/i });
  await user.click(btn);

  expect(screen.getByText('1')).toBeInTheDocument();
});

// ── Form typing + submit ────────────────────────────────
it('submits the form with the entered values', async () => {
  const user     = userEvent.setup();
  const onSubmit = vi.fn();
  render(<LoginForm onSubmit={onSubmit} />);

  await user.type(screen.getByLabelText(/email/i), 'ada@example.com');
  await user.type(screen.getByLabelText(/password/i), 'secret123');
  await user.click(screen.getByRole('button', { name: /log in/i }));

  expect(onSubmit).toHaveBeenCalledWith({
    email: 'ada@example.com',
    password: 'secret123',
  });
});

// ── waitFor: assert after async state change ────────────
it('shows success message after save', async () => {
  const user = userEvent.setup();
  render(<SaveButton />);

  await user.click(screen.getByRole('button', { name: /save/i }));

  // waitFor polls until assertion passes or times out (1s default)
  await waitFor(() => {
    expect(screen.getByText(/saved!/i)).toBeInTheDocument();
  });
});

// ── fireEvent for low-level or custom events ────────────
import { fireEvent } from '@testing-library/react';
fireEvent.change(input, { target: { value: 'new value' } });
fireEvent.keyDown(el, { key: 'Escape' });
```

USEREVENT VS FIREEVENT

- `userEvent` — fires complete event sequences; moves focus; simulates real browser behaviour
- `fireEvent` — fires a single synthetic event; useful for events userEvent doesn't support (custom events, drag)
- `userEvent.setup()` — call once per test; returns `user` instance with all methods
- `await user.click(el)` — all userEvent methods are async; always `await` them
- `waitFor(() => …)` — polls the assertion until it passes or times out (default 1 s)
- `waitForElementToBeRemoved` — waits for an element to disappear from the DOM

NOTES

- → `userEvent.setup()` — creates a session; call once per test, not in `beforeEach` if possible; ensures focus state tracks across interactions
- → `await user.type(el, 'text')` — types each character individually, triggering keydown/keypress/keyup and onChange per key; more realistic than fireEvent.change
- → `vi.fn()` — Vitest mock function; pass as a prop; assert it was called with `toHaveBeenCalledWith`
- → `waitFor(() => expect(…))` — retries the assertion up to the timeout; necessary for state updates driven by Promises or timers
- → `fireEvent.change(input, { target: { value } })` — the manual escape hatch; fires a single change event without the full keyboard event sequence
- → RTL wraps userEvent in `act` automatically; you rarely need to call `act` manually when using `userEvent` and `waitFor`

USEREVENT METHODS

METHOD	SIMULATES
`user.click(el)`	Full click sequence with pointer events
`user.dblClick(el)`	Double-click
`user.type(el, 'text')`	Type text character by character
`user.clear(el)`	Select all and delete
`user.keyboard('{Enter}')`	Press a key by name
`user.tab()`	Press Tab to move focus
`user.hover(el)`	Mouse enter + move events
`user.selectOptions(el, vals)`	Select option(s) in a <select>

KEYBOARD() SPECIAL KEYS

KEY STRING	KEY PRESSED
`{Enter}`	Enter / Return
`{Escape}`	Esc
`{Tab}`	Tab
`{Space}`	Space bar
`{Backspace}`	Backspace
`{ArrowDown}`	Down arrow
`{Shift>}{Tab}{/Shift}`	Shift+Tab (reverse focus)

COMMON MISTAKES

- Forgetting `await` on userEvent methods — they return Promises; without await the test moves on before the interaction completes and assertions fail
- Using `fireEvent.change` instead of `user.type` — fireEvent skips the keyboard events that real inputs depend on; some validation hooks won't trigger
- Asserting synchronously after an async action — state from a fetch or timer hasn't updated yet; always use `waitFor` or `findBy` after async interactions

TIPS

- Mock fetch with `vi.stubGlobal('fetch', vi.fn().mockResolvedValue(…))` — keeps tests fast and deterministic without hitting the network
- Use `waitForElementToBeRemoved(() => screen.getByText(/loading/i))` to wait for a spinner to disappear before asserting the final state
- Group related assertions with `describe` blocks and descriptive `it` strings — they serve as living documentation of component behaviour

SUMMARY

- `const user = userEvent.setup()` — create once per test; always await methods
- `user.click / type / keyboard / tab` — full event sequence; prefer over fireEvent
- `waitFor(() => expect(…))` — poll assertion after async state changes
- `vi.fn()` — mock callbacks; assert with `toHaveBeenCalledWith`

Vitest & Jest Setup

TEST RUNNERS FOR REACT PROJECTS

Vitest is the recommended test runner for Vite-based React projects. It reuses your Vite config, supports ESM natively, and is significantly faster than Jest for large test suites. Its API is almost identical to Jest — `describe`, `it`, `expect`, `vi` (Vitest's mock API, equivalent to `jest`).

Jest remains common in CRA and Next.js projects. Both support the same matchers, mock APIs, and setup patterns. The main difference is the mock namespace (`vi` vs `jest`) and configuration location. Vitest reads from `vite.config.js`; Jest uses `jest.config.js` or the `jest` key in `package.json`.

TEST STRUCTURE

- `describe('name', () => {})` — groups related tests; can be nested
- `it('should …', () => {})` — individual test case; alias: `test()`
- `beforeEach / afterEach` — setup/teardown before/after every test in the block
- `beforeAll / afterAll` — once before/after all tests in the block
- `it.only / it.skip` — run only or skip specific tests during debugging
- `expect(val).matcher()` — assertion; throws if it fails, test passes otherwise

VITEST SETUP & PATTERNS

```
// vite.config.js — add test section
export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
    setupFiles:  ['./src/test/setup.ts'],
    globals:     true,    // no need to import describe/it/expect
  },
});

// src/test/setup.ts
import '@testing-library/jest-dom';   // adds toBeInTheDocument etc.

// ── Unit test: pure function ──────────────────────────
import { describe, it, expect } from 'vitest';
import { formatPrice } from './utils';

describe('formatPrice', () => {
  it('formats whole dollars', ()     => expect(formatPrice(10)).toBe('$10.00'));
  it('formats cents',        ()     => expect(formatPrice(9.5)).toBe('$9.50'));
  it('throws on negative',   ()     => expect(() => formatPrice(-1)).toThrow());
});

// ── Mock function ─────────────────────────────────────
import { vi, it, expect } from 'vitest';

it('calls the callback with the result', () => {
  const callback = vi.fn();
  runWithResult(42, callback);
  expect(callback).toHaveBeenCalledWith(42);
  expect(callback).toHaveBeenCalledTimes(1);
});

// ── Module mock ───────────────────────────────────────
vi.mock('./api', () => ({
  fetchUser: vi.fn().mockResolvedValue({ id: 1, name: 'Ada' }),
}));

// ── beforeEach cleanup ────────────────────────────────
import { beforeEach, afterEach } from 'vitest';

beforeEach(() => { vi.clearAllMocks(); });
afterEach(()  => { vi.restoreAllMocks(); });
```

NOTES

- → `environment: 'jsdom'` — simulates a browser DOM; required for React Testing Library; Vitest uses Node by default
- → `globals: true` — imports `describe`, `it`, `expect` automatically; if false, import them explicitly from 'vitest'
- → `vi.fn()` — creates a tracked function; records calls, arguments, and return values; inspect with `toHaveBeenCalled*` matchers
- → `vi.mock('./api')` — hoisted to the top of the file by Vitest; replaces the module with your factory for the whole file
- → `vi.clearAllMocks()` — resets call counts and arguments between tests; call in `beforeEach` to prevent test order dependencies
- → `vi.spyOn(console, 'error')` — wraps an existing method; useful for suppressing expected console errors in tests

COMMON EXPECT() MATCHERS

MATCHER	ASSERTS
`toBe(val)`	Strict equality (Object.is)
`toEqual(val)`	Deep equality (objects, arrays)
`toBeTruthy() / toBeFalsy()`	Truthy or falsy
`toContain(item)`	Array includes item / string contains substring
`toThrow('msg')`	Function throws (optionally matching message)
`toHaveBeenCalled()`	Mock function was called at least once
`toHaveBeenCalledWith(…args)`	Mock called with specific arguments
`toHaveBeenCalledTimes(n)`	Mock called exactly n times

VI / JEST MOCK APIS

API	PURPOSE
`vi.fn()`	Create a mock function
`vi.fn().mockReturnValue(v)`	Return a fixed value
`vi.fn().mockResolvedValue(v)`	Return a resolved Promise
`vi.fn().mockRejectedValue(e)`	Return a rejected Promise
`vi.mock('module')`	Auto-mock an entire module
`vi.spyOn(obj, 'method')`	Spy on an existing method
`vi.clearAllMocks()`	Reset call history (not implementation)

COMMON MISTAKES

- Using `toBe` for object comparison — `toBe` uses `Object.is` (reference equality); use `toEqual` for deep structural comparison
- Not clearing mocks between tests — a mock called in test 1 retains its call count in test 2; always call `vi.clearAllMocks()` in `beforeEach`
- Forgetting `async` / `await` in an async test — the test completes before the Promise resolves; assertions are skipped and the test passes incorrectly

TIPS

- Use `it.todo('description')` to document planned tests — they show as pending in the output without failing the suite
- Run `vitest --ui` for an interactive browser-based test UI — easier to debug than the terminal output for large suites
- Keep unit tests (pure functions) and integration tests (components with RTL) in separate `describe` blocks or files — different expectations for speed and mocking needs

SUMMARY

- `describe / it / expect` — standard test structure; Vitest API mirrors Jest
- `vi.fn()` — mock function; tracks calls; assert with `toHaveBeenCalledWith`
- `vi.mock('module', factory)` — replace entire module for a test file
- `vi.clearAllMocks()` in beforeEach — prevents call-count bleed between tests

Testing Hooks & Async

TESTING CUSTOM HOOKS AND ASYNC BEHAVIOUR

`renderHook` (from `@testing-library/react`) mounts a hook in an isolated component so you can test its return value and behaviour directly without building a full UI. It returns a `result` ref whose `.current` always reflects the hook's latest returned value.

For hooks that trigger state updates (from async operations, timers, or effects), wrap the triggering call in `act()` and `await` it. Mock `fetch` with `vi.stubGlobal` or `vi.fn()` to keep tests fast and offline. Use `waitFor` to poll assertions until async state settles.

RENDERHOOK PATTERN

- `renderHook(() => useMyHook(args))` — mounts the hook; returns `{ result, rerender, unmount }`
- `result.current` — the hook's return value; always up to date after state updates
- `rerender({ newArgs })` — re-invoke with new arguments to test dependency changes
- `act(() => { … })` — wrap any code that causes state updates; RTL does this automatically for userEvent but not for manual calls
- `waitFor` — poll an assertion until it passes; use for async state in hooks
- `unmount()` — triggers cleanup; test that subscriptions and timers are cleared

NOTES

- → `renderHook(() => useCounter(0))` — the factory function runs inside a minimal component; `result.current` tracks the latest return
- → `act(() => { result.current.increment() })` — synchronous act; flushes all React state updates before the next assertion
- → `waitFor(() => expect(result.current.loading).toBe(false))` — polls until the assertion passes; works for async state after fetch completes
- → `vi.stubGlobal('fetch', vi.fn())` — replaces the global `fetch` with a mock; restore with `vi.unstubAllGlobals()` in `afterEach`
- → `vi.spyOn(AbortController.prototype, 'abort')` — tracks whether abort was called; verifies the hook's cleanup function fired
- → `vi.advanceTimersByTimeAsync(300)` — fast-forwards fake timers without real waiting; essential for debounce/throttle hook tests

MOCKING FETCH

PATTERN	CODE
Global stub	`vi.stubGlobal('fetch', vi.fn())`
Mock resolved JSON	`.mockResolvedValue({ ok: true, json: () => Promise.re`
Mock error	`.mockResolvedValue({ ok: false, status: 404 })`
Mock network error	`.mockRejectedValue(new Error('Network error'))`
Restore after tests	`vi.unstubAllGlobals() in afterEach`

ASYNC TESTING TOOLS

TOOL	USE WHEN
`await act(async () => …)`	Trigger async state update manually
`waitFor(() => expect(…))`	Poll assertion until async state resolves
`findBy queries`	Wait for element to appear in DOM
`waitForElementToBeRemoved`	Wait for element to leave DOM (loading state)
`vi.useFakeTimers()`	Control setTimeout / setInterval manually

HOOK & ASYNC TESTING PATTERNS

```
import { renderHook, act, waitFor } from '@testing-library/react';
import { vi, it, expect, beforeEach, afterEach } from 'vitest';
import { useFetch } from './useFetch';
import { useCounter } from './useCounter';

// ── Test a synchronous custom hook ──────────────────────
it('useCounter increments and resets', () => {
  const { result } = renderHook(() => useCounter(0));

  act(() => { result.current.increment(); });
  expect(result.current.count).toBe(1);

  act(() => { result.current.reset(); });
  expect(result.current.count).toBe(0);
});

// ── Test a hook with async fetch ────────────────────────
beforeEach(() => {
  vi.stubGlobal('fetch', vi.fn().mockResolvedValue({
    ok: true,
    json: () => Promise.resolve([{ id: 1, title: 'React' }]),
  }));
});

afterEach(() => { vi.unstubAllGlobals(); });

it('useFetch returns data after loading', async () => {
  const { result } = renderHook(() => useFetch('/api/posts'));

  // Initially loading
  expect(result.current.loading).toBe(true);

  // Wait for the fetch to complete and state to update
  await waitFor(() => {
    expect(result.current.loading).toBe(false);
  });

  expect(result.current.data).toEqual([{ id: 1, title: 'React' }]);
  expect(result.current.error).toBeNull();
});

// ── Test cleanup (abort on unmount) ─────────────────────
it('aborts fetch on unmount', () => {
  const abortSpy = vi.spyOn(AbortController.prototype, 'abort');
  const { unmount } = renderHook(() => useFetch('/api/posts'));
  unmount();
  expect(abortSpy).toHaveBeenCalled();
});

// ── Test with fake timers ───────────────────────────────
it('useDebounce delays update', async () => {
  vi.useFakeTimers();
  const { result, rerender } = renderHook(({ v }) => useDebounce(v, 300), {
    initialProps: { v: 'a' },
  });
  rerender({ v: 'ab' });
  expect(result.current).toBe('a');          // still old value
  await act(() => vi.advanceTimersByTimeAsync(300));
  expect(result.current).toBe('ab');         // debounced value
  vi.useRealTimers();
});
```

COMMON MISTAKES

- Reading `result.current` before wrapping the state-triggering call in `act` — the state hasn't applied yet; the value is still the previous one
- Not restoring fetch mocks after tests — a stubbed global leaks into subsequent tests; always use `vi.unstubAllGlobals()` in `afterEach`
- Using fake timers and forgetting `vi.useRealTimers()` afterward — subsequent tests that rely on real timers (like `waitFor`) hang or fail

TIPS

- Prefer testing hooks through their consuming component with RTL when possible — it exercises the full integration and is less brittle than renderHook
- For hooks that need a Context Provider, pass it via the `wrapper` option: `renderHook(() => useMyHook(), { wrapper: MyProvider })`
- Test the error path explicitly — mock a failed fetch and assert that `result.current.error` has the right message; happy path only tests are incomplete

SUMMARY

- `renderHook(() => useHook())` — test hooks directly; read via `result.current`
- `act(() => …)` — flush state updates before asserting
- `vi.stubGlobal('fetch', vi.fn())` — mock network; restore in afterEach
- `vi.useFakeTimers()` — control time; `advanceTimersByTimeAsync` to skip waits

Build, Deploy & Next Steps

npm run build · Vite build · Vercel · Netlify · en

FROM DEVELOPMENT TO PRODUCTION

`npm run build` runs Vite's production build: it bundles, tree-shakes, minifies, and outputs static files into `dist/`. The output is a **Single Page Application (SPA)** — a single `index.html` plus hashed JS/CSS chunks. It can be hosted on any static file server.

For apps that need **server-side rendering (SSR)**, static site generation (SSG), file-system routing, or better SEO, a **framework** built on React is the right choice. **Next.js** is the most popular — it adds SSR, API routes, and image optimisation on top of React. **Remix** focuses on web fundamentals (forms, loaders, progressive enhancement) and works well at the edge.

BUILD, PREVIEW & DEPLOY

```
# ── Vite build ──────────────────────────────────────────
npm run build          # outputs to dist/
npm run preview        # serve dist/ locally on :4173

# dist/ contents after build:
# dist/index.html
# dist/assets/index-BxYtf3Jw.js    (hashed — safe to cache forever)
# dist/assets/index-Cd12kMnP.css

# ── Netlify: _redirects file for SPA fallback ──────────
# public/_redirects:
# /*    /index.html   200

# ── Vercel: vercel.json for SPA fallback ───────────────
# {
#   "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
# }

# ── Environment variables ──────────────────────────────
# .env.local (Vite — gitignored)
VITE_API_URL=https://api.example.com
VITE_STRIPE_KEY=pk_test_…

# In code:
# const api = import.meta.env.VITE_API_URL;

# Set in Vercel/Netlify dashboard under "Environment Variables"
# Never commit secrets — use .env.example with keys but no values

# ── Next.js scaffold (SSR/SSG alternative) ─────────────
npx create-next-app@latest my-app

# ── Remix scaffold ─────────────────────────────────────
npx create-remix@latest my-app

# ── GitHub Actions: auto-deploy to Vercel ──────────────
# .github/workflows/deploy.yml:
# on: push: branches: [main]
# jobs: deploy: uses: vercel/actions…
```

DEPLOYMENT CHECKLIST

- `npm run build` — run locally first; fix any type or lint errors before pushing
- `npm run preview` — serve the `dist/` folder locally; catches asset-path bugs before deployment
- Set environment variables in the hosting dashboard — never commit secrets to git
- Configure the host to serve `index.html` for all paths (SPA fallback) — prevents 404s on direct URL access
- Add a `public/404.html` that redirects to `index.html` for GitHub Pages (no server config available)
- Enable HTTPS — required for service workers, geolocation, and camera APIs

NOTES

- → `dist/assets/index-BxYtf3Jw.js` — content-hashed filename; set `Cache-Control: max-age=31536000` on the assets folder; `index.html` must never be cached
- → `/* /index.html 200` in `_redirects` — Netlify serves `index.html` for all 404 paths so React Router can handle them client-side
- → `VITE_` prefix — required; only vars with this prefix are embedded in the browser bundle; all others stay server-side
- → Never commit `.env` — add it to `.gitignore`; commit `.env.example` with key names and placeholder values for teammates
- → Next.js vs plain Vite — if you need SSR for SEO or faster initial paint, start with Next.js; for internal tools and authenticated apps, a plain SPA is fine
- → React Server Components (Next.js 13+) — run on the server, ship zero JS to the client; the next major paradigm shift in the React ecosystem

HOSTING OPTIONS FOR REACT SPAS

HOST	DEPLOY COMMAND / METHOD	SPA FALLBACK
Vercel	Push to GitHub → auto-deploy	✔ automatic
Netlify	Push or drag-and-drop dist/	✔ via _redirects
GitHub Pages	gh-pages branch or Actions	✖ needs workaround
Cloudflare Pages	Git integration or Wrangler CLI	✔ automatic
AWS S3 + CloudFront	aws s3 sync dist/ s3://bucket	✔ via CF error page

REACT FRAMEWORKS AT A GLANCE

FRAMEWORK	KEY FEATURES
Next.js	SSR, SSG, App Router, API routes, Image opt, Vercel-native
Remix	Loaders, actions, forms, edge-first, web standards
Gatsby	GraphQL data layer, static-first, plugin ecosystem
Astro	Islands architecture; ships zero JS by default; use React where needed
Vite (plain)	SPA only; no SSR; simplest starting point

COMMON MISTAKES

- Not configuring the SPA fallback on the host — direct URL access or page refresh returns a 404 from the server because the path doesn't map to a real file
- Caching `index.html` aggressively — users get stale JS bundle references; set `Cache-Control: no-cache` on `index.html` and let the hashed assets be cached long-term
- Using environment variables without the `VITE_` prefix — they are undefined at runtime with no error; Vite silently excludes them from the bundle

TIPS

- Run `npm run build && npm run preview` before every deployment — the preview build catches base-path issues, missing env vars, and broken asset paths that only appear in production
- Use Vercel or Netlify for new projects — zero-config deploys, automatic HTTPS, preview deployments on every PR, and SPA fallback out of the box
- Learn Next.js after mastering React — App Router, Server Components, and edge functions are where the ecosystem is heading; the React fundamentals you've learned apply directly

SUMMARY

- `npm run build` → `dist/` — static SPA ready to host anywhere
- SPA fallback — configure host to serve index.html for all 404s
- `VITE_` env vars — client-side only; set secrets in hosting dashboard
- Next.js / Remix — the next step for SSR, routing, and server-side data